

NTFS 文档

理查德·鲁森

尤瓦尔·弗莱德

NTFS 文档

作者:理查德·鲁森和尤瓦尔·弗莱德尔

抽象的

这是技术文档,旨在帮助程序员。

它最初是为了配合Linux NTFS 驱动程序而编写的[<http://linux-ntfs.sourceforge.net/>]。

最新版本可在线获取,网址为: <http://linux-ntfs.sourceforge.net/ntfs/index.html>,并且可以……

下载自: http://sourceforge.net/project/showfiles.php?group_id=13956

我们相信信息是正确的。我们认为我们知道自身知识的不足之处。但我们也可能出错。请注意。

有关常见问题的简单答案,请访问[<http://linux-ntfs.sourceforge.net/info/ntfs.html>]。

建议阅读 NTFS 常见问题解答。

目录

1. 序言.....	1
1. NTFS 文档前言.....	1
2. 关于 NTFS 文档	1
3. 表格图例.....	3
4. 音量布局	4
2. NTFS 属性	7
1. 概述	7
2. 属性 - \$STANDARD_INFORMATION (0x10)	7
3. 属性 - \$ATTRIBUTE_LIST (0x20)	9
4. 属性 - \$FILE_NAME (0x30)	11
5. 属性 - \$OBJECT_ID (0x40)	13
6. 属性 - \$SECURITY_DESCRIPTOR (0x50)	14
7. 属性 - \$VOLUME_NAME (0x60)	22
8. 属性 - \$VOLUME_INFORMATION (0x70)	23
9. 属性 - \$DATA (0x80)	24
10. 属性 - \$INDEX_ROOT (0x90)	25
11. 属性 - \$INDEX_ALLOCATION (0xA0)	28
12. 属性 - \$BITMAP (0xB0)	29
13. 属性 - \$REPARSE_POINT (0xC0)	30
14. 属性 - \$EA_INFORMATION (0xD0)	32
15. 属性 - \$EA (0xE0)	33
16. 属性 - \$LOGGED_UTILITY_STREAM (0x100)	34
3. NTFS 文件	35
1. 概述	35
2. NTFS 文件:\$MFT (0)	36
3. NTFS 文件:\$MFTMirr (1)	37
4. NTFS 文件:\$LogFile (2)	38
5. NTFS 文件:\$Volume (3)	43
6. NTFS 文件:\$AttrDef (4)	43
7. NTFS 文件: . (根目录) (5)	46
8. NTFS 文件:\$Bitmap (6)	47
9. NTFS 文件:\$Boot (7)	48
10. NTFS 文件:\$BadClus (8)	50
11. NTFS 文件:\$Secure (9)	51
12. NTFS 文件:\$UpCase (10)	54
13. NTFS 文件:\$Extend (11)	55
14. NTFS 文件:\$ObjId (任意)	55
15. NTFS 文件:\$配额 (NT:9,2K:任意)	57
16. NTFS 文件:\$Reparse (任意)	59
17. NTFS 文件:\$UsnJrnl (任意)	60
4. NTFS 概念	64
1. 概述	64
2. 概念 - 属性标头	64
3. 概念 - 属性 ID	68
4. 概念 - B*树	68
5. 概念 - 集群.....	72
6. 概念 - 整理	73
7. 概念 - 压缩	74
8. 概念 - 数据运行	77
9. 概念 - 目录.....	84
10. 概念 - 文件.....	86
11. 概念 - 文件记录.....	88
12. 概念 - 文件参考	92

13. 概念 - 文件名命名空间.....	92
14. 概念 - 修复	93
15. 概念 - 索引标题.....	96
16. 概念 - 索引记录.....	97
17. 概念 - 链接	99
18. 概念 - 重启	99
19. 概念 - SID	99
20. 概念 - 稀疏.....	103
5. 尾声.....	104
1. 待办事项.....	104
2. 未解答的问题.....	105
3. 历史.....	106
附录一. 许可证.....	110
1. GNU 自由文档许可证.....	110
词汇表.....	115

表格列表

1.1. 尺寸字段表图例	3
1.2. 索引表示例.....	4
1.3. 各操作系统的 NTFS 卷版本.....	4
1.4. 新格式化的 NTFS 卷的布局	4
2.1. 标准 NTFS 属性	7
2.2. \$STANDARD_INFORMATION (0x10) 属性的布局	8
2.3 文件权限.....	8
2.4. \$ATTRIBUTE_LIST (0x20) 属性的布局.....	10
2.5. \$FILE_NAME (0x30) 属性的布局	11
2.6 文件标志	12
2.7. \$OBJECT_ID (0x40) 属性的布局.....	13
2.8. \$SECURITY_DESCRIPTOR (0x50) 属性的布局.....	14
2.9. \$SECURITY_DESCRIPTOR (0x50) 属性头的布局.....	15
2.10. ACL 的布局	16
2.11. ACE 的布局	16
2.12. ACE 类型.....	16
2.13. ACE 标志.....	17
2.14. ACE 审核标志.....	17
2.15. ACE 访问掩码.....	17
2.16. SID 内容	18
2.17. SID 示例	18
2.18. 安全描述符控制标志	18
2.19. \$VOLUME_NAME (0x60) 属性的布局.....	22
2.20. \$VOLUME_INFORMATION (0x70) 属性的布局	23
2.21. 音量标志.....	23
2.22. 卷版本号.....	24
2.23. \$DATA (0x80) 属性的布局	24
2.24. \$INDEX_ROOT (0x90) 属性的布局:索引根.....	26
2.25. \$INDEX_ROOT (0x90) 属性的布局:索引头	26
2.26. 索引标志.....	26
2.27. 常用索引	27
2.28. \$INDEX_ALLOCATION (0xA0) 属性的布局.....	28
2.29. \$INDEX_ALLOCATION (0xA0) 属性中的数据条目布局.....	28
2.30. 数据录入标志.....	29
2.31. \$BITMAP (0xB0) 属性的布局	30
2.32. \$REPARSE_POINT (0xC0) 属性的布局 (Microsoft 重新分析点)	30
2.33. \$REPARSE_POINT (0xC0) 属性 (第三方重解析点)的布局	30
2.34.符号链接重新分析数据 2.35.卷链接重解析数	31
据 2.36.重新解析标签标	31
志.....	31
2.37. \$EA_INFORMATION (0xD0) 属性的布局	33
2.38. \$EA (0xE0) 属性的布局	33
2.39. EA 旗帜.....	33
2.40. \$LOGGED_UTILITY_STREAM (0x100) 属性的布局	34
3.1 卷上的文件布局.....	35
3.2. \$MFT 属性.....	36
3.3. \$MFT 起始时的示例记录	36
3.4. \$MFTMirr 属性	37
3.5. \$MFTMirr 的布局	38
3.6. 日志文件属性.....	38
3.7. 交易量属性.....	43
3.8. \$AttrDef 属性	43
3.9. \$AttrDef 的布局	44

3.10. \$AttrDef 排序规则..... 44

3.11. \$AttrDef 标志..... 45

3.12. Windows NT 中的 \$AttrDef 示例 3.13. Windows 2000/XP 中的 \$AttrDef 示例 46

3.14. 点 (.) 属性 46

3.15. 点的布局 (.) 47

3.16. 位图属性..... 47

3.17. \$Bitmap 的布局..... 48

3.18. 启动属性 48

3.19. \$Boot 的布局 49

3.20. \$BadClus 属性 50

3.21. 安全属性..... 51

3.22. \$Secure:\$SDS 的布局 52

3.23. \$Secure:\$SDH 的布局 52

3.24. \$Secure:\$SII 的布局 53

3.25. \$UpCase 属性 54

3.26. \$UpCase 的布局..... 54

3.27. 扩展属性..... 55

3.28. \$ObjId 属性 55

3.29. \$ObjId:\$O 的布局..... 56

3.30. \$ObjId 标志 56

3.31. 配额属性 57

3.32. 配额布局:\$O 57

3.33. 配额布局:\$Q 58

3.34. \$配额标志 58

3.35. \$重新解析属性..... 59

3.36. \$Reparse:\$R 的布局 60

3.37. \$UsnJrnl 属性..... 60

3.38. \$UsnJrnl:\$J 的布局 61

3.39. \$UsnJrnl:\$Max 的布局 61

3.40. \$UsnJrnl 原因标志 62

3.41. \$UsnJrnl 源信息标志 62

4.1. NTFS 概念..... 64

4.2. 驻留未命名属性头的布局..... 65

4.3. 居民命名属性头的布局..... 65

4.4. 非驻留未命名属性头的布局..... 66

4.5. 非驻留命名属性头的布局..... 66

4.6. 属性标志..... 67

4.7. 默认集群大小..... 72

4.8. 排序类型..... 73

4.9. 标准索引的默认排序规则类型..... 73

4.10. 数据运行布局 77

4.11. 已解析数据运行 :示例 1 - 正常、未碎片化的文件 81

4.12. 已解析数据运行 :示例 2 - 正常、碎片化文件 82

4.13. 解析后的数据运行 :示例 3 - 正常文件、乱码文件 82

4.14. 已解析数据运行 :示例 4 - 稀疏、未碎片化文件..... 83

4.15. 解析后的数据运行 :示例 5 - 压缩的、未碎片化的文件..... 84

4.16. 目录记录属性..... 85

4.17. 文件记录属性..... 86

4.18. 虚构的命名数据流 87

4.19. 命名数据流的摘要信息..... 88

4.20. 汇总信息内容命名数据流..... 88

4.21. 文件记录的布局..... 89

4.22. 文件记录标志..... 90

4.23. 文件引用的布局..... 92

4.24. 修复示例 :之前..... 94

4.25. 修复示例 :在..... 95 之后

4.26. 标准索引顶层的布局..... 96

4.27. 常用索引列表	97
4.28. 索引记录头的布局.....	97
4.29. 常见且广为人知的SIDs.....	100
4.30. 标识符授权机构.....	100
4.31. 相对标识符	101
4.32. 域用户.....	101
4.33. 域组.....	102
4.34. 域名别名	102
4.35. 通用知名SID.....	102
4.36. NT 众所周知的 SIDs	103
122. 计量单位	128

第一章 序言

1. NTFS 文档前言

这是 NTFS 文档的 0.5 版本,它是Linux-NTFS 项目的一部分。
[<http://linux-ntfs.sourceforge.net/>]

这是技术文档,旨在帮助程序员。

它最初是为了配合Linux NTFS 驱动程序而编写的[<http://linux-ntfs.sourceforge.net/>]。

最新版本可在线获取,网址为: <http://linux-ntfs.sourceforge.net/ntfs/index.html>,并且可以……
下载自: http://sourceforge.net/project/showfiles.php?group_id=13956

有关常见问题的简单答案,请访问[<http://linux-ntfs.sourceforge.net/info/ntfs.html>]。

建议阅读 NTFS 常见问题解答。

2. 关于 NTFS 文档

2.1 概述

NTFS 是 Windows NT、2000 和 XP 的文件系统。它支持几乎所有 POSIX 功能和所有 HFS 功能。以及所有 HPFS 功能。

- 它可以处理大容量 (高达 2⁴⁶ GB) 存储单位。
- 它内置了数据压缩功能。
- 它使用日志文件来记录交易。
- 字节顺序: 磁盘上的所有数据都是小端字节序。

2.2 文档布局

- 第一章 - 序言: 是对文档的描述信息。
- 第 2 章 - 文件: 是元数据文件的列表。
- 第 3 章 - 属性: 是元数据属性的列表。
- 第 4 章 - 概念: 是一个既不是文件也不是属性的对象列表。
- 第 5 章 - 结语: 包含有关文档的更多信息。
- 附录 I - 许可: 是文档分发所依据的许可。
- 术语表: 是对技术术语的解释说明

2.3. 准确性

微软并未发布任何关于 NTFS 的文档。这些文档一部分是通过仔细阅读所有 SDK 和 Windows 帮助文档整理而成,但主要还是通过对文件系统进行逆向工程得出的。

我们相信信息是正确的。我们认为我们知道自身知识的不足之处。但我们也可能出错。请注意。

2.4. 接触点

您可以在 SourceForge [<http://sourceforge.net/>] 的开放论坛上发布问题,网址为 http://sourceforge.net/forum/forum.php?forum_id=44084

如果你想更多地参与 Linux 项目,那么你可以加入其中一个邮件列表(两个邮件列表的邮件量都很低)。

- NTFS 通用列表: <http://tiger.informatik.hu-berlin.de/cgi-bin/mailman/listinfo/linux-ntfs>
- 这里还有一个更技术性的链接: <http://lists.sourceforge.net/lists/listinfo/linux-ntfs-dev>

如果您有任何问题、建议或更正,请给我发电子邮件。

理查德·鲁森

2.5. 许可证

版权所有 (C) 1996-2004 Richard Russon。

版权所有 (C) 2005 Yuval Fledel。

根据 GNU 自由文档许可证 1.1 版或自由软件基金会发布的任何后续版本的条款,允许复制、分发和/或修改本文档;

- 不变封面为谢谢
- 封面文字内容为关于 NTFS 文档的介绍
- 而且没有封底文字。

许可证副本包含在题为“GNU 自由文档许可证”的部分中。

2.6. 谢谢

非常感谢以下人员在准备这些文件时提供的帮助。

- 阿尔伯特·卡哈兰
- 亚历克斯·伊奥内斯库
- 安东·阿尔塔帕尔马科夫
- Bram Moolenaar (为 vim 而作)
- 达蒙·卡萨莱

- 大卫·迪拉德
- Domagoj Pensa
- 海伦·卡斯特
- 马丁·冯·勒维斯
- 奥洛夫·沃尔加斯特
- 拉尼·阿萨夫
- 雷吉斯·杜舍纳
- 理查德·鲁森
- 尤瓦尔·弗莱德

3. 表格图例

3.1 概述

本文档中的表格并不完全一致。以下是表格说明,展示了如何……
涵盖了各个领域。

3.2. 脚注

任何带有脚注标记 (例如 (a)、(e))的表格字段,其下方都会立即显示更详细的描述。
桌子。

3.3. 尺寸字段

在 NTFS 文件系统中,并非所有字段的大小都是固定的。有些字段的大小取决于另一个字段的值,有些字段的大小取决于……
该字段的内容。

尺寸字段中的所有数字均为十进制格式。例如 12 (十二),42 (四十二)。

表 1.1. 尺寸字段表图例

钥匙	姓名	描述
12	固定的	该字段长度为 12 字节,大小固定。
P8	填充	P8 表示将字段填充到 8 字节边界。此字段的大小可以是 0 到 7 个字节。P4 表示 4 字节对齐,等等 (a)
在	多变的	该字段的长度取决于其内容。例如: SID。要了解其长度,必须解码该结构。
S	交叉引用	交叉引用表明,尺寸在表格的其他位置有定义。尺寸可以用除 P 或 V 以外的任何字母表示。

(a)任何固定大小的填充都将以固定大小显示。

3.4 指数

如果表格代表索引,则键和数据将显示如下:

表 1.2. 索引表示例

抵消	尺寸	描述	
0x00	2	数据偏移	
0x02	2	数据大小	
0x04	4	钥匙	SID
0x08	4	数据	所有者 ID
0x0C	4	数据	哈希

3.5 操作系统

请注意,这些字段的用途并不完全相同。NT 表示旧字段,而 2K 和 XP 则表示新字段。
表示新增字段。

表 1.3. 各操作系统的 NTFS 卷版本

你	NTFS	描述
空白的	全部	适用于所有版本的 Windows
NT	1.2	仅在 Windows NT 系统中使用
2K	3.0	Windows 2000 及更高版本
XP	3.1	初次使用 Windows XP

重复组?
链接到 help/tables 中的 padding8、padding 和其他表格功能
始终如一地使用填充/对齐字段

4. 音量布局

4.1 概述

一个新格式化的 NTFS 卷看起来会像这样:

表 1.4. 新格式化的 NTFS 卷的布局

B	M	自由空间	更多的	自由空间
这	F		元	
这	T		数据	

T				
---	--	--	--	--

4.2 注释

4.2.1 其他信息

在 NTFS 文件系统中,一切皆为文件。这些文件的索引是主文件表 (MFT)。MFT 列出了位于磁盘开头的引导扇区文件 (\$Boot)。\$Boot 还列出了 MFT 的存放位置。MFT也列出了自己。

在磁盘中心,我们找到了一些元数据文件。其中比较重要的两个是 \$MFTMirr 和 \$LogFile。MFT 镜像文件是 MFT 前 4 条记录的精确副本。如果 MFT 损坏,则可以通过找到镜像文件来恢复卷。LogFile 文件记录了所有等待写入磁盘的事件。如果机器崩溃,则使用 LogFile 文件将磁盘恢复到正常状态。

卷的末尾隐藏着引导扇区 (簇 0)的副本。唯一引用它的元数据文件是 \$Bitmap,而该文件仅表明该簇正在使用中。

4.2.2. MFT 区

为了防止主文件表 (MFT) 碎片化,Windows 会在其周围维护一个缓冲区。在其他磁盘空间用完之前,不会在这个缓冲区中创建新文件。缓冲区大小可配置,可以是磁盘空间的 12.5%、25%、37.5% 或 50%。每次磁盘空间用完时,缓冲区大小都会减半。

MFT 区域预留未存储在磁盘上。MFT 区域 (MFT 预留空间)1 = 12.5% 2 = 25.0% 3 = 37.5% 4 = 50.0% 此信息存储在磁盘的哪个位置?卷?MFT?引导分区?

这是磁盘开头的“系统文件”空间。

NtfsMftZoneReservation

链接到 MFT 和位图

- 簇大小:512 字节、1k、2k、4k、8k、16k、32k、64k
- 非常灵活,所有系统文件都可以重新定位,除了 \$Boot 文件。
- 支持名为数据流的数据流
- 文件的属性可以跨越多个 MFT 记录,这些记录不一定是连续的,也不一定按顺序排列。

一切皆属性,包括数据。

- 文件名以 Unicode 编码存储。
- 日志文件系统

- 压缩
- 安全
- 硬链接
- 加密
- LCN 与 VCN

第二章 NTFS 属性

1. 概述

每个 MFT 文件记录都是由属性构成的。

可能的属性列表在 \$AttrDef 中定义。

表 2.1. 标准 NTFS 属性

操作系统	姓名
0x10	标准信息
0x20	属性列表
0x30	\$FILE_NAME
0x40 NT	\$VOLUME_VERSION
0x40 2K	\$OBJECT_ID
0x50	\$SECURITY_DESCRIPTOR
0x60	\$VOLUME_NAME
0x70	成交量信息
0x80	\$DATA
0x90	\$INDEX_ROOT
0xA0	索引分配
0xB0	位图
0xC0 NT	\$SYMBOLIC_LINK
0xC0 2K	\$REPARSE_POINT
0xD0	\$EA_INFORMATION
0xE0	每股
0xF0 NT	\$PROPERTY_SET
0x100 2K	\$LOGGED_UTILITY_STREAM

1.1 注释

1.1.1 其他信息

\$PROPERTY_SET、\$SYMBOLIC_LINK 和 \$VOLUME_VERSION 在 NTFS v1.2 中存在,但未使用。它们在 NTFS v3.0 (Win2K 使用的版本)中不再存在。

每个 MFT 记录都有一个标准标头,后跟一个属性列表(按属性值升序排列)。ID)和一个结束标记。结束标记只有四个字节:0xFFFFFFFF。

2. 属性 - \$STANDARD_INFORMATION (0x10)

2.1 概述

在旧版本的 NTFS 文件系统中,此属性仅包含 DOS 文件权限和文件信息。
次数。

Windows 2000 引入了四个新字段,用于引用配额、安全性、文件大小和
日志信息。

根据 \$AttrDef 的定义,此属性的最小大小为 48 字节,最大大小为 72 字节。

2.2. 属性（驻留者）的布局

表 2.2. \$STANDARD_INFORMATION (0x10) 属性的布局

抵消	尺寸	你	描述
~	~		标准属性标头
0x00	8		C 时间 - 文件创建
0x08	8		时间 - 文件已更改
0x10	8		M 时间 - MFT 已更改
0x18	8		R 时间 - 文件读取
0x20	4		DOS 文件权限
0x24	4		版本数量上限
0x28	4		版本号
0x2C	4		类 ID
0x30	4	2K	所有者 ID
0x34	4	2K	安全 ID
0x38	8	2K	配额收费
0x40	8	2K	更新序列号 (USN)

2.2.1. 文件权限

在DOS术语中也称为属性。

表 2.3 文件权限

旗帜	描述
0x0001	只读
0x0002	隐
0x0004	系统
0x0020	档案
0x0040	设备
0x0080	普通的
0x0100	暂时的
0x0200	稀疏文件
0x0400	重解析点

旗帜	描述
0x0800	压缩
0x1000	离线
0x2000	未编入索引的内容
0x4000	加密

- 最大版本数
文件允许的最大版本数。零表示禁用版本编号。
- 版本号
此文件的版本（如有）。如果最大版本数为零,则版本号为零。
- 类 ID
从双向类 ID 索引中获取类 ID。
- 所有者 ID
文件所有者的 ID。此 ID 是文件 \$Quota 的 \$O 和 \$Q 索引中的一个键。如果此 ID 为零,则配额功能已禁用。
- 安全 ID
这不应与安全标识符混淆。安全 ID 是文件 \$Secure 中 \$SII 索引和 \$SDS 数据流的一个键。
- 配额收费
此文件占用的用户配额字节数。这应该是所有数据流的总数据大小。如果为零,则表示配额已禁用。
- 更新序列号 (USN)
文件最后更新序列号。这是指向文件 \$UsnJrnl 的直接索引。如果为零,则 USN 日志功能将被禁用。

2.3 注释

2.3.1 其他信息

如果将 NTFS 卷从 v1.2 升级到 v3.0,则此属性在被访问之前不会升级（延长）。

2.3.2 问题

Version 字段和 Class Id 是否会被使用？

3. 属性 - \$ATTRIBUTE_LIST (0x20)

3.1 概述

当属性很多而 MFT 记录空间不足时,所有可以省略的属性都将无法使用。
被认定为非居民的患者将被迁出MFT。如果仍然没有足够的床位,那么……
需要 \$ATTRIBUTE_LIST 属性。其余属性将放入新的 MFT 记录中。
\$ATTRIBUTE_LIST 属性描述了它们的位置。这个属性非常罕见。

3.2 属性布局

在标准标头之后,此属性包含一个可变长度记录列表,用于描述类型和
该文件所有其他属性在主文件表 (MFT) 中的位置。每条记录按 8 字节对齐。
边界。

列表按以下方式排序:

- 1. 属性类型
- 2. 属性名称 (如有)
- 3. 序列号

注意:它不会列出自身。

表 2.4. \$ATTRIBUTE_LIST (0x20) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00	4	类型
0x04	2	记录长度
0x06	1	名称长度 (N)
0x07	1	名称偏移量 (a)
0x08	8	起始 VCN (b)
0x10	8	属性的基本文件引用
0x18	2	属性 ID (c)
0x1A	2N	Unicode 名称 (如果 N > 0)

- (a)如果名称不存在,则此属性指向该属性还是零?
 - (b) 起始 VCN,如果属性是驻留属性,则起始 VCN 为零。
 - (c)每个属性都有一个唯一的标识符
 - (a)它始终指向名称所在的位置 (0x1A)
 - 0x04 记录分配 (8 字节对齐)
 - (c)似乎总是零,请检查
 - (c) 不,它只会在给定属性类型首次出现时显示。
- 不确定是否应该按序列号排序。VCN 肯定可以。

3.3 注释

3.3.1. \$AttrDef

它可以是居民或非居民。此属性没有最小或最大尺寸限制。

3.3.2 其他信息

偏移量 0x07 处只有一个字节长,这对于属性来说很不寻常。

如果此属性为非驻留属性,则数据运行必须适合一个 MFT 记录。

如果文件存在以下情况,则可能需要 \$ATTRIBUTE_LIST:

- 包含大量硬链接 (存在大量文件名属性)。
- 变得非常分散,因此数据运行超出 MFT 记录。
- 具有复杂的安全描述符 (不适用于 NTFS v3.0+)
- 有许多命名流,例如数据流。

3.3.3.待办事项

8 VCN 最低_vcn;
此属性值部分的最小虚拟集群编号。当一个属性无法放入一个 MFT 记录中,需要分配多个 MFT 记录来存储该属性时,此值不为零。在后一种情况下,每个记录包含该属性的一个范围,并且属性列表条目也只有一个范围。注意:这绝对是一个有符号值!Windows 驱动程序在比较此值时使用 cmp (由 jg 提供),因此将其视为有符号值。

24 个 __u16 实例;
如果 lowest_vcn = 0,则为被引用属性的实例;否则为 0
属性列表用于在 MFT 中完整描述文件扩展名 FILE 记录时,以便查找该文件的任何文件属性。

由于该文件流可能会增长,因此该文件属性可能是非驻留的。

非居民属性的范围 (如果存在)紧随其后
在初始范围之后,它们按最低 VCN 排序,并具有它们的实例。

4. 属性 - \$FILE_NAME (0x30)

4.1 概述

此属性存储文件属性的名称,并且始终驻留。

根据 \$AttrDef 的定义,此属性的最小大小为 68 字节,最大大小为 578 字节。
相当于文件名长度最大为 255 个 Unicode 字符。

4.2. 属性 (驻留者)的布局

表 2.5. \$FILE_NAME (0x30) 属性的布局

NTFS属性		
抵消	尺寸	描述
~	~	标准属性标头
0x00	8	文件引用到父目录。
0x08	8	C 时间 - 文件创建
0x10	8	时间 - 文件已更改
0x18	8	M 时间 - MFT 已更改
0x20	8	R 时间 - 文件读取
0x28	8	文件分配大小
0x30	8	文件实际大小
0x38	4	标志,例如:目录、压缩、隐藏
0x3c	4	EA 和 Reparse 使用
0x40	1	文件名长度 (字符数)
0x41	1	文件名命名空间 0x42 2L Unicode 文件名 (非空终止符)

4.2.1 文件引用

这是指向父目录基本记录的文件引用。

4.2.2 文件大小

文件分配大小是指文件占用的磁盘空间大小。它将是文件大小的倍数。
集群大小。文件的实际大小是未命名数据属性的大小。这个数字将
出现在目录列表中。

注意:仅当起始 VCN 为零时,才会显示“实际大小”。请参阅标准属性头。
更多信息。

4.2.3. 旗帜

表 2.6. 文件标志

旗帜	描述
0x0001	只读
0x0002	隐
0x0004	系统
0x0020	档案
0x0040	设备
0x0080	普通的
0x0100	暂时的
0x0200	稀疏文件
0x0400	重解析点
0x0800	压缩
0x1000	离线
0x2000	未编入索引的内容
0x4000	加密

旗帜	描述
0x10000000	目录（从 MFT 记录中的相应位复制）
0x20000000	索引视图（从 MFT 记录中的相应位复制）

4.3 注释

4.3.1 其他信息

NTFS 通过创建包含多个文件名属性的文件来实现 POSIX 风格的硬链接。每个文件名属性都有其自身的详细信息和父级。当删除硬链接文件时,其文件名会从 MFT 记录中移除。只有当最后一个链接被移除时,该文件才会被真正删除。

注意 :除父目录外,所有字段仅在文件名更改时更新。在此之前,它们只是会过时。但是,\$STANDARD_INFORMATION 属性将始终保留最新。

注意 :如果文件包含扩展属性 (EA),则 EA 字段将包含所需的缓冲区大小。

注意 :如果文件是重分析点,则重分析字段将给出其类型。

5. 属性 - \$OBJECT_ID (0x40)

5.1 概述

对象 ID 是在 Windows 2000 中引入的。每个 MFT 记录都被分配一个唯一的 GUID。此外,一条记录可能还包含出生卷 ID、出生对象 ID 和域 ID,所有这些 ID 都…… GUID。

根据 \$AttrDef 的定义,此属性没有最小大小,但最大为 256 字节。

5.2 属性的布局

表 2.7. \$OBJECT_ID (0x40) 属性的布局

抵消	尺寸	姓名	描述
~	~	标准属性标头	
0x00	16	GUID 对象 ID	文件的唯一标识符
0x10	16	GUID 出生量 ID	文件创建所在的卷
0x20	16	GUID 出生对象 ID	文件原始对象 ID
0x30	16	GUID 域 ID	对象创建所在的域

5.2.1. 出生量 ID

出生卷 ID 是分配对象 ID 的卷的对象 ID。它永远不会改变。

5.2.2. 出生对象 ID

出生对象 ID 是分配给此 MFT 记录的第一个对象 ID。例如,如果对象 ID 为由于某种原因发生了更改,此字段将反映对象 ID 的原始值。

5.2.3. 域名 ID

域 ID 目前未使用,但计划在本地网络环境中使用。
这台机器是 Windows 2000 域的一部分。它可以在 Windows 2000 高级服务器中使用。
托管域。

5.3 注释

5.3.1 其他信息

此属性的长度可能只有 16 字节（一个 GUID 的大小）。

即使未使用出生卷、出生对象和域 ID,它们也可能存在,但其中一项或多项可能无效。
更多可能为零。

需要一些使用所有字段的示例。

6. 属性 - \$SECURITY_DESCRIPTOR (0x50)

6.1 概述

标准属性标头？

安全描述符可概括如下：

- 一个头部（可能是标志）,后面跟着一个或两个 ACL 和两个 SID。
- 第一个 ACL 包含审计信息,可能不存在。
- 第二个 ACL 包含权限（谁可以做什么）。
- 每个 ACL 包含一个或多个 ACE。
- 每个 ACE 都包含一个 SID。
- 最后两个 SID 显示对象的拥有者（用户和组）。

表 2.8. \$SECURITY_DESCRIPTOR (0x50) 属性的布局

成分			描述
标题			各种结构的偏移
审计 ACL	高手	SID	审计 ACL 的 ACE
权限 ACL	高手	SID	权限 ACL 的 ACE
	高手	SID	
	高手	SID	
SID（用户）			此物品的所有者
SID（组）			

安全描述符对于防止未经授权访问文件至关重要。它存储以下信息：

- 文件所有者
- 所有者授予其他用户的权限
- 哪些操作需要记录（审计）

6.2 属性的布局

6.2.1. 注释

6.2.1.1. 尺寸

根据 \$AttrDef 的定义,此属性没有最小或最大大小。

6.3. 流的布局

6.3.1. 问题

- 目录的 ACE 如何继承？
- 如何将 ACE 集成到外观正常的 Unix 文件系统中？
- 如何将文件权限与 PAM 或 SMB 关联起来？
- 我们能否以某种方式使用 NT 认证？

6.3.2.待办事项

- 确定哪些标准权限和特定权限与哪些文件系统活动相关,例如
FILE_APPEND_DATA 允许用户扩展文件,但不能创建新文件。
- 做些实验,看看我们看到的零是否是填充,以及类似标志的字段是否是标志。
- 尝试使用通用读取/写入/执行/全部标志。

6.3.3. 标题

表 2.9. \$SECURITY_DESCRIPTOR (0x50) 属性头的布局

抵消	尺寸	描述
0x00	1	修订版 (a)
0x01	1	填充
0x02	2	控制标志 (b)
0x04	4	用户 SID 偏移量
0x08	4	组 SID 偏移量

NTFS属性

抵消	尺寸	描述
0x0C	4	偏移至 SACL
0x10	4	偏移至 DACL

(a) 暂定为 0x1

(b) 通常为 0x4 (DACL 存在) 或 0x14 (DACL 存在且 SACL 存在)。请参阅下方的标志。

(c) 这指的是审计 ACL。

(d) 这指的是权限访问控制列表 (ACL)。

6.4. 前交叉韧带

表 2.10. ACL 的布局

抵消	尺寸	描述
0x00	1	前交叉韧带翻修术
0x01	1	填充 (0x00)
0x02	2	前交叉韧带尺寸
0x04	2	ACE 计数
0x06	2	填充 (0x0000)

访问控制列表 (ACL) 包含一个或多个 ACE。

我的机器上, ACL 版本当前为 0x02。

Win32 API 表明, 0x01 和 0x06 包含用于对齐的填充 0x00。

6.5. ACE

表 2.11. ACE 的布局

抵消	尺寸	描述
0x00	1	类型
0x01	1	旗帜
0x02	2	尺寸
0x04	4	访问掩码
0x08	在	SID

6.5.1. 类型

目前 (在 NT 系统中) 已实现的类型有:

表 2.12. ACE 类型

NTFS属性

价值	描述
0x00	已允许访问
0x01	拒绝访问
0x02	系统审计

6.5.2. 旗帜

Flags 是一个位域。Flags 的可能值取决于 Type 的值。当应用于目录时, Access Allowed 或 Access Denied 可以具有标志。

表 2.13. ACE 标志

价值	描述
0x01	对象继承ACE
0x02	容器继承ACE
0x04	不要传播“继承ACE”
0x08	仅继承ACE

如果类型为系统审计,则标志可以是:

表 2.14. ACE 审计标志

价值	描述
0x40	成功审计
0x80	失败审计

6.5.3. 访问掩码/访问权限

访问掩码/权限是一个位域,列举了所有 (不允许的)操作。

表 2.15. ACE 访问掩码

位 (范围)	意义	描述/示例
0-15	对象特定访问权限	读取数据、执行、追加数据
16 - 22	标准访问权限	删除、写入 ACL、写入所有者
23	可以访问安全 ACL	
24 - 27	预订的	
28	通用全部 (读、写、执行)	以下所有内容
29	通用执行	执行程序所需的一切
30	通用写入	写入文件所需的一切
31	通用阅读	读取文件所需的一切

6.6. SID（安全标识符）

典型的SID格式如下:S-1-5-21-646518322-1873620750-619646970-1110

它由 “Spqrstuv”组成

表 2.16. SID 内容

S	安全
p	修订号（当前为 1）
q	NT 权威机构。该数字分为 6 个字节（48 位大端序数字）。
房车	NT 下属机构（可能有很多）

磁盘上SID的存储方式如下：

12月:S-1-5-21-646518322-1873620750-619646970-1110

十六进制:S-1-5-15-26891632-6fad2f0e-24ef0ffa-456（5 个子机构）

表 2.17. SID 示例

0x00	01	05	00	00	00	00	00	05
0x08	15	00	00	00	32	16	89	26
0x10	0e	2f	广告	6f	但	0f	如果	24
0x18	56	04	00	00				

注意:这是一个可变长度的结构。组成该结构的下属机构数量可能更多,也可能更少。
结构更大,或更小。

6.6.1. 安全描述符控制标志

表 2.18. 安全描述符控制标志

旗帜	描述
0x0001	业主违约
0x0002	组违约
0x0004	DACL 呈报
0x0008	DACL 违约
0x0010	SACL 呈现
0x0020	SACL 违约
0x0100	DACL 自动继承要求
0x0200	SACL 自动继承要求
0x0400	DACL 自动继承
0x0800	SACL 自动继承
0x1000	DACL 保护
0x2000	SACL 保护

旗帜	描述
0x4000	RM 控制有效
0x8000	自亲

6.6.1.1. 所有者违约

设置此布尔标志时,表示 Owner 字段指向的 SID 是由去故障机制提供的,而不是由安全描述符的原始提供者显式提供的。

这可能会影响SID在所有者继承权方面的处理方式。

6.6.1.2. 组默认

此布尔标志设置后,表示“组”字段中的 SID 是由默认机制提供的,而不是由安全描述符的原始提供程序显式提供的。这可能会影响 SID 在继承主组时的处理方式。

6.6.1.3. 拿着礼物

此布尔标志位设置后,表示安全描述符包含一个自主访问控制列表 (ACL)。如果此标志位已设置,且安全描述符的 Dacl 字段为空,则表示显式指定了一个空 ACL。

6.6.1.4. 默认声明

设置此布尔标志时,表示 Dacl 字段指向的 ACL 是由故障消除机制提供的,而不是由安全描述符的原始提供程序显式提供的。

这可能会影响前交叉韧带 (ACL)的遗传处理。如果未设置 DaclPresent 标志,则忽略此标志。

6.6.1.5. SACL 存在

此布尔标志位设置后,表示安全描述符包含由 Sacl 字段指向的系统 ACL。如果此标志位已设置,且安全描述符的 Sacl 字段为空,则表示指定了一个空 (但实际存在)的 ACL。

6.6.1.6. SACL 默认

设置此布尔标志时,表示 Sacl 字段指向的 ACL 是由故障消除机制提供的,而不是由安全描述符的原始提供程序显式提供的。

这可能会影响对前交叉韧带 (ACL)遗传性的处理。如果未设置 SaclPresent 标志,则忽略此标志。

6.6.1.7. 自相对

此布尔标志位设置后,表示安全描述符采用自相对形式。在这种形式下,安全描述符的所有字段在内存中都是连续的,所有指针字段都表示为从安全描述符开头算起的偏移量。

SID结构是一种可变长度的结构,用于唯一标识用户或组。SID代表安全标识符。

SID 的标准文本表示形式为:SRISS...

其中: - 第
一个“S”是字面字符“S”,用于标识后面的数字为 SID。

- R 是 SID 的修订级别,以数字序列表示。
可以是十进制或十六进制 (如果是后者,则以 “0x”为前缀)。
- I 是 48 位标识符_授权,以数字形式表示,如上文 R 所示。
- S... 是一个或多个子权限值,以数字形式表示,如上所述。

示例 SID;本地管理员组的域相对 SID
Windows NT/2k:
S-1-5-32-544

这相当于一个 SID,其值为:

修订版 = 1,
子机构数量 = 2,
identifier_authority = {0,0,0,0,0,5}, SECURITY_NT_AUTHORITY
子权威[0] = 32,
子机构[1] = 544

SECURITY_BUILTIN_DOMAIN_RID
DOMAIN_ALIAS_RID_ADMINS

ACE 类型
ACCESS_MIN_MS_ACE_TYPE = 0
允许的 ACE 类型 = 0
ACCESS_DENIED_ACE_TYPE = 1
系统审计 ACE 类型 = 2
系统报警ACE类型 = 3 截至 Win2k 尚未实现。
ACCESS_MAX_MS_V2_ACE_TYPE = 3

ACCESS_ALLOWED_COMPOUND_ACE_TYPE = 4
ACCESS_MAX_MS_V3_ACE_TYPE = 4

以下内容仅适用于 Windows 2000。
ACCESS_MIN_MS_OBJECT_ACE_TYPE = 5
ACCESS_ALLOWED_OBJECT_ACE_TYPE = 5
ACCESS_DENIED_OBJECT_ACE_TYPE = 7 = 6
SYSTEM_AUDIT_OBJECT_ACE_TYPE
SYSTEM_ALARM_OBJECT_ACE_TYPE = 8
ACCESS_MAX_MS_OBJECT_ACE_TYPE = 8

ACCESS_MAX_MS_V4_ACE_TYPE = 8

这个版本适用于WinNT和2k。
ACCESS_MAX_MS_ACE_TYPE = 8

用于审计和继承的 ACE 标志 (8 位)

SUCCESSFUL_ACCESS_ACE_FLAG 仅用于系统审计和告警 ACE。
用于指示 (在 Windows 系统中!)成功生成消息的类型
访问。

FAILED_ACCESS_ACE_FLAG 仅用于系统审计和告警 ACE 类型
表示 (在 Windows 系统中!)对于访问失败会生成一条消息。

继承标志。
对象继承ACE、容器继承ACE、不传 = 0x01
播继承ACE、仅继承ACE、已继承ACE、有 = 0x02
效继承标志 = 0x04
= 0x08
= 0x10 仅限 Win2k
= 0x1f

审计标记。
成功访问 ACE 标志 = 0x40
访问失败标志 = 0x80

访问掩码定义了访问权限。

标准权利。

删除 = 0x00010000
读取控制 = 0x00020000
WRITE_DAC = 0x00040000
WRITE_OWNER = 0x00080000
同步 = 0x00100000

STANDARD_RIGHTS_REQUIRED = 0x000f0000

STANDARD_RIGHTS_READ = 0x00020000
标准写入权限 0x00020000
STANDARD_RIGHTS_EXECUTE = 0x00020000

标准权利 = 0x001f0000

访问控制系统 ACL 和最大允许访问类型。
ACCESS_SYSTEM_SECURITY = 0x01000000
最大允许值 = 0x02000000

通用权利。
通用全部 通用执行 通 = 0x10000000
用写入 通用读取 = 0x20000000
= 0x40000000
= 0x80000000

对象 ACE 标志 (32 位)。
ACE_OBJECT_TYPE_PRESENT = 1
ACE_INHERITED_OBJECT_TYPE_PRESENT = 2

ACL CONSTANTS

当前版本。
ACL_REVISION = 2
ACL_REVISION_DS = 4

修订历史。
ACL_REVISION1 = 1
MIN_ACL_REVISION = 2
ACL_REVISION2 = 2
ACL_REVISION3 = 3
ACL_REVISION4 = 4
MAX_ACL_REVISION = 4

绝对安全描述符。不包含所有者和组 SID,也不包含其他信息。
安全描述符内部的 sacl 和 dacl ACL。相反,它包含
指向内存中这些结构的指针。显然,绝对安全。
描述符仅对内存中的安全性表示有用。
描述符。在磁盘上,使用自相对安全描述符。

属性:安全描述符 (0x50)。标准自相对安全
描述符。

注意:始终居住。
注意:NTFS 3.0 及更高版本中未使用此功能,因为安全描述符集中存储。
在 FILE_\$Secure 中,使用 security_id 找到了正确的描述符。
来自标准信息属性。

在 NTFS 3.0 及更高版本中,所有安全描述符都存储在 FILE_\$Secure 中。只有一个
存储每个唯一安全描述符的引用实例。

FILE_\$Secure 不包含任何未命名的数据属性,即其长度为零。
不过,它确实包含两个指数 (\$SDH 和 \$SII)以及一个命名数据。
流 (\$SDS)。

每个唯一的安全描述符都被分配一个唯一的安全标识符
(security_id,不要与 SID 混淆)。security_id 是唯一的。

该 NTFS 卷用作 \$SII 索引的索引,\$SII 索引将 security_id 映射到 \$SDS 数据属性中安全描述符的存储位置。\$SII 索引按 security_id 升序排序。

每个安全描述符都会计算一个简单的哈希值。该哈希值用作 \$SDH 索引的索引,该索引将安全描述符哈希值映射到 \$SDS 数据属性中安全描述符的存储位置。

\$SDH 索引按安全描述符哈希值排序,并存储在 B+ 树中。当搜索 \$SDH (目的是确定新的安全描述符是否已存在于 \$SDS 数据流中)时,如果找到匹配的哈希值,但安全描述符不匹配,则继续在 \$SDH 索引中搜索下一个匹配的哈希值。

当找到精确匹配项时,会从找到的 \$SDH 索引条目中读取与 \$SDS 属性中的安全描述符对应的 security_id,并将其存储在应用该安全描述符的文件/目录的 \$STANDARD_INFORMATION 属性中。\$STANDARD_INFORMATION 属性存在于所有基本 MFT 记录 (即所有文件和目录)中。

如果未找到匹配项,则为安全描述符分配一个新的唯一 security_id,并将其添加到 \$SDS 数据属性中。然后,将 \$SDS 数据属性中引用此安全描述符的条目添加到 \$SDH 和 \$SII 索引中。

注意:即使没有任何内容再引用某个条目,条目也永远不会从 FILE_{\$Secure 中删除。

\$SDS 数据流包含安全描述符,这些描述符按 16 字节边界对齐,并按 security_id 排序,排列成 B+ 树状结构。安全描述符不能跨越 256KiB 边界 (此限制由 Windows 缓存管理器强制执行)。每个安全描述符都包含在一个 SDS_ENTRY 结构中。

此外,每个安全描述符在 \$SDS 流中存储两次,它们之间的固定偏移量为 0x40000 字节 (256kib,Windows 缓存管理器的最大大小);也就是说,如果 SDS_ENTRY 指定偏移量为 0x51d0,则安全描述符的第一个副本将位于 \$SDS 数据流中的偏移量 0x51d0 处,第二个副本将位于偏移量 0x451d0 处。

\$SII 索引。排序规则类型为 COLLATION_NTFS_ULONGLONG。
\$SDH 索引。排序规则为 COLLATION_NTFS_SECURITY_HASH。

7. 属性 - \$VOLUME_NAME (0x60)

7.1 概述

此属性仅包含卷的名称。

根据 \$AttrDef 的定义,此属性的最小大小为 2 字节,最大大小为 256 字节。这相当于卷名称的最大长度为 127 个 Unicode 字符。

7.2 属性的布局

表 2.19. \$VOLUME_NAME (0x60) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头

NTFS属性		
抵消	尺寸	描述
0x00		Unicode名称

7.3. 注释

卷名不以 Unicode NULL 结尾。其名称长度与属性的大小相同。
如标头中所述。

卷的序列号存储在 \$Boot 中。

8. 属性 - \$VOLUME_INFORMATION (0x70)

8.1 概述

显示卷的版本和状态。

根据 \$AttrDef 中的定义,此属性的最小和最大大小均为 12 字节。

8.2 属性的布局

表 2.20. \$VOLUME_INFORMATION (0x70) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00	8	总是零?
0x08	1	主版本号
0x09	1	次版本号
0x0A	2	旗帜
0x0C	4	总是零?

8.2.1. 旗帜

表 2.21. 音量标志

价值	描述
0x0001	肮脏的
0x0002	调整日志文件大小
0x0004	升级山
0x0008	安装在 NT4 上
0x0010	删除美国海军正在执行的任务
0x0020	修复对象 ID
0x8000	由 chkdsk 修改

8.3 注释

8.3.1. 脏旗

当脏标志被设置时,Windows NT 必须在卷上执行 chkdsk /F 命令。
下一只靴子。

8.3.2. 版本号

表 2.22. 卷版本号

操作系统	版本
Windows NT	1.2
Windows 2000	3.0
Windows XP	3.1

8.3.3 其他信息

卷的序列号存储在 \$Boot 中。

9. 属性 - \$DATA (0x80)

9.1 概述

此属性包含文件的数据。文件的大小即为其未命名数据流的大小。

根据 \$AttrDef 的定义,此属性没有最小或最大大小。

9.2 属性的布局

表 2.23. \$DATA (0x80) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00		任何数据

9.3. 注释

9.3.1. Windows 使用的通用数据流

- [未命名]
- {4c8cc155-6c1e-11d1-8e41-00c04fb9386d}
- ^EDocumentSummaryInformation

- ^ESebiesnrMkudrfcolaamtykdDa
- ^E摘要信息
- \$MountMgr数据库
- 差
- \$SDS
- \$J
- 最高

9.3.2 其他信息

通常情况下,目录没有数据属性,文件的数据属性也没有名称。

必须具有 (至少为空的)未命名数据属性

NTFS 的一个优势在于:由于一个文件可以拥有多个数据属性,因此可以轻松实现 HFS 文件系统。HFS 的文件由两部分组成 (在 HFS 术语中也称为“分支”) :资源部分和数据部分。数据部分使用默认的未命名数据属性,而资源部分则使用名为“resource”的数据属性。

10. 属性 - \$INDEX_ROOT (0x90)

10.1 概述

这是实现索引 (例如目录)的 B+ 树的根节点。此文件属性始终存在。

常住居民。

10.2 属性的布局

请点击下方链接

\$INDEX_ROOT

- 标准属性标头
- 索引根
- 索引标题
- 索引条目

- 索引条目
- ...

10.2.1. 索引根

表 2.24. \$INDEX_ROOT (0x90) 属性的布局:索引根

抵消	尺寸	描述
~	~	标准属性标头
0x00	4	属性类型
0x04	4	排序规则
0x08	4	索引分配条目的大小 (字节)
0x0C	1	每个索引记录的聚类
0x0D	3	填充 (对齐到 8 字节)

10.2.2. 索引标题

表 2.25. \$INDEX_ROOT (0x90) 属性的布局:索引头

抵消	尺寸	描述
0x00	4	到第一个索引条目的偏移量
0x04	4	索引条目的总大小
0x08	4	索引条目的分配大小
0x0C	1	旗帜
0x0D	3	填充 (对齐到 8 字节)

10.2.3. 旗帜

表 2.26. 索引标志

旗帜	描述
0x00	小索引 (适合索引根)
0x01	大型索引 (需要分配索引)

设置 0x00 的标志位很愚蠢,请移除

大索引标志指示文件属性索引分配和位图是否存在 (当索引足够小,可以完全存储在根节点中 (缺少这两个文件属性))。

10.3. 注释

10.3.1. 尺寸

根据 \$AttrDef 的定义,此属性没有最小或最大大小。

10.3.2. 索引条目的顺序

这是一个长度可变的索引条目序列。该序列以一个特殊字符结尾。
索引条目,其最后一个条目标志已设置。

这是索引的头部信息,描述了 INDEX_ENTRY 记录。
索引头之后是索引项。索引头和索引项共同构成索引项。
编制完整的索引。

接下来是一系列索引条目 (INDEX_ENTRY 结构)
如索引头所述。

当一个目录足够小,可以放入索引根目录时,就会出现这种情况。
这是描述目录的唯一属性。当目录过大时,
另一方面,为了适应索引根目录,还需要两个额外的属性。
存在:索引分配属性,包含 B+ 的子节点
目录树 (见下文)以及一个位图属性,用于描述哪个虚拟目录
索引分配属性中的簇编号 (vcn) 正在被使用
索引块。

注意:根目录 (FILE_\$root)包含它自身的条目。

```
结构体 {  
    ATTR_TYPES 类型;  
    索引属性的类型。对于目录,类型为 $FILENAME,否则为零。  
    用于视图索引。不允许使用其他值。  
    COLLATION_RULES 排序规则;索引条目。如果 type 用于排序的排序规则  
    为 $FILENAME,则此值必须为 COLLATION_FILENAME。  
  
    __u32 bytes_per_index_block;  
    每个索引块的字节大小 (在索引分配属性中)。  
  
    __u8 clusters_per_index_block;  
    每个索引块的簇大小 (在索引分配属性中),索引块的大小大于或等于一个簇的大小,否则这将是日志。  
  
    大小 (例如 MFT 记录大小和索引的编码方式)  
    记录大小 (在引导扇区中找到)。必须是 2 的幂。  
} INDEX_ROOT;
```

10.4. 常用索引列表

表 2.27. 常用指数

姓名	索引	使用者
130美元	文件名	目录
\$SDH	安全描述符	安全
捐赠	安全 ID	安全
\$O	对象 ID	\$ObjId
\$O	所有者 ID	配额
\$Q	配额	配额

姓名	索引	使用者
\$R	重解析点	\$Reparse

索引之间有哪些共享元素？
与索引根无关

11. 属性 - \$INDEX_ALLOCATION (0xA0)

11.1 概述

这是索引（例如目录）的基本组成部分。这是所有子节点的存储位置。
B+ 树中实现索引（例如目录）的文件属性。该文件属性始终是非驻留的。

根据 \$AttrDef 的定义,此属性没有最小或最大大小。

此属性永远不会驻留 应使用索引根代替。

11.2. 属性的布局

它只是属于该索引的所有索引缓冲区的序列。

表 2.28. \$INDEX_ALLOCATION (0xA0) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00	...	数据运行

11.2.1. 索引条目

至少分成两张桌子

表 2.29. \$INDEX_ALLOCATION (0xA0) 中的数据条目布局
属性

抵消	尺寸	描述
~	~	标准属性标头
仅当最后一个条目标志未设置时,下一个字段才有效。		
0x00	8	文件引用
0x08	2	L = 索引项的长度
0x0A	2	M = 河流长度

NTFS属性		
抵消	尺寸	描述
0x0C	1	旗帜
仅当未设置最后一个条目标志时,才会显示下一个字段。		
0x10	M	溪流
仅当设置了子节点标志时,下一个字段才会出现。		
L-8	8	索引分配属性中子节点的 VCN

11.2.2. 旗帜

表 2.30. 数据录入标志

旗帜	描述
0x01	索引条目指向一个子节点
0x02	节点中的最后一个索引条目

最后条目标志用于指示索引条目序列的结束。虽然它不代表有效文件,但可以指向子节点。

11.3. 注释

11.3.1. 河流长度

索引条目索引的驻留文件属性的标头部分中偏移量为 10 的字段的副本。
但为什么这两个字段的大小不一样呢？

11.3.2. 流

由索引条目索引的驻留文件属性流的副本（例如,对于目录,文件名称属性）。

始终是非居民（反正成为居民也没什么意义！）。

这是一个索引块数组。每个索引块都以一个开头。
INDEX_BLOCK 结构包含索引头,后跟一系列索引块。
索引条目（INDEX_ENTRY 结构）,如 INDEX_HEADER 所述。

创建索引块时,我们将更新序列数组放置在此处。
偏移量,也就是索引条目开始之前的偏移量。这样做也是合理的。
否则,我们可能会遇到更新序列数组的问题。
它本身包含一个扇区的最后两个字节,这意味着
多扇区传输保护不起作用,因为无法保护数据。
因为覆盖它之后就无法恢复了……
读取时使用ntfs记录头中的数据。

12. 属性 - \$BITMAP (0xB0)

12.1 概述

此文件属性是一系列比特,每个比特代表一个实体的状态。

根据 \$AttrDef 的定义,此属性没有最小或最大大小。

12.2 属性的布局

此属性目前在两个地方使用:索引(例如目录)和 \$MFT。注意:索引条目 FILE 记录中还包含标志,用于显示它们是否正在使用。

在索引中,位字段指示哪些索引项正在使用。每个位代表索引的一个虚拟内容编号 (VCN) 分配。

在 \$MFT 中,位字段显示哪些 FILE 记录正在使用。

表 2.31. \$BITMAP (0xB0) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00		位域

13. 属性 - \$REPARSE_POINT (0xC0)

13.1 概述

根据 \$AttrDef 中的定义,此属性没有最小大小,但最大大小为 16384 字节。

13.2. 属性布局 (Microsoft 重解析点)

表 2.32. \$REPARSE_POINT (0xC0) 属性的布局 (Microsoft 重解析点)

抵消	尺寸	描述
~	~	标准属性标头
0x00	4	重新解析类型 (和标志)
0x04	2	重新分析数据长度
0x06	2	填充 (对齐到 8 字节)
0x08	在	重新解析数据 (a)

13.3. 属性布局 (第三方重解析点)

表 2.33. \$REPARSE_POINT (0xC0) 属性的布局 (第三方 重解析点)

抵消	尺寸	描述
~	~	标准属性标头

抵消	尺寸	描述
0x00	4	重新解析类型（和标志）
0x04	2	重新分析数据长度
0x06	2	填充（对齐到 8 字节）
0x08	16	维修指南
0x18	在	重新解析数据 (a)

(a) 重新解析数据的结构取决于重新解析类型。
三个已定义的重新解析数据（符号链接、卷链接和 RSS)+通用重新解析。

13.3.1. 符号链接重新解析数据

表 2.34。符号链接重新分析数据

抵消	尺寸	描述
0x00	2	替代名称偏移
0x02	2	替代名称长度
0x04	2	打印名称偏移
0x08	2	打印姓名长度
0x10	在	路径缓冲区

13.3.2. 卷链接重新解析数据

表 2.35。卷链接重新解析数据

抵消	尺寸	描述
0x00	2	替代名称偏移
0x02	2	替代名称长度
0x04	2	打印名称偏移
0x08	2	打印姓名长度
0x10	在	路径缓冲区

13.3.3. 重新解析标签标志

这些只是预定义的重解析标志。

表 2.36. 重新解析标签标志

旗帜	描述
0x20000000	别名
0x40000000	高延迟
0x80000000	微软

旗帜	描述
0x68000005	国家安全局
0x68000006	NSS恢复
0x68000007	六
0x68000008	DFS
0x88000003	登山点
0xA8000004	HSM
0xE8000000	符号链接

13.4. 注释

13.4.1. 其他信息

重解析点标签定义了重解析点的类型。它还包含几个标志,用于进一步描述重解析点。

重解析点标签是一个无符号 32 位值,分为三个部分:

- 1. 最低有效 16 位 (即位 0 到 15)指定类型重解析点。
- 2. 此后的 13 位 (即第 16 位到第 28 位)保留供将来使用。
- 3. 最高三位是描述重分析点的标志。

它们的定义如下:

位 29:名称代理位。如果设置,则文件名是系统中另一个对象的别名。

位 30:高延迟位。如果设置,则访问数据的第一个字节将速度要慢。(例如,数据存储在磁带驱动器上。)第 31 位:Microsoft 位。如果设置,则该标签归 Microsoft 所有。用户定义的标签此处必须使用零。

系统文件 FILE_Extend/\$Reparse 包含一个名为 \$R 的索引,其中列出了卷上的所有重分析点。索引条目的键定义如下。请注意,索引条目没有关联的索引数据。

索引条目按索引键 file_id 排序。排序规则为 COLLATION_NTDFS_ULONGS。待修复:验证 reparse_tag 是否不是主键/根本不是键。(AIA)

14. 属性 - \$EA_INFORMATION (0xD0)

14.1 概述

用于在 NTFS 文件系统下实现 HPFS 扩展属性,这些属性由 OS/2 和 Windows NT 服务器的 OS/2 客户端的信息子系统使用。由于其数据流可能会增长,因此该文件属性可能是非驻留的。

根据 \$AttrDef 的定义,此属性的最小和最大大小均为 8 字节。

14.2. 属性的布局

表 2.37. \$EA_INFORMATION (0xD0) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00	2	打包后的扩展属性的大小
0x02	2	具有 NEED EA 集的扩展属性的数量
0x04	4	解压后的扩展属性的大小

15. 属性 - \$EA (0xE0)

15.1 概述

用于在 NTFS 文件系统中实现 HPFS 扩展属性。由于其数据流可能会增长,因此该文件属性可能是非驻留的。

根据 \$AttrDef 中的定义,此属性没有最小大小,但最大大小为 65536 字节。

15.2. 属性的布局

扩展属性是名称、值对的集合。

表 2.38. \$EA (0xE0) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00	4	到下一个扩展属性的偏移量
0x04	1	旗帜
0x05	1	名称长度 (N)
0x06	2	值长度 (V)
0x08	N	姓名
N+0x08	在	价值

反之,到下一个 EA 的偏移量就是这个 EA 的大小。

15.2.1. 旗帜

表 2.39. EA 旗帜

价值	描述
0x80	需要EA

15.3. 注释

15.3.1. 其他信息

此文件属性流的作用和布局是什么?查阅HPFS文档可能会有所帮助。

15.3.2. 问题

EA这个名称真的不是Unicode编码吗?

16. 属性 - \$LOGGED_UTILITY_STREAM (0x100)

16.1 概述

根据 \$AttrDef 中的定义,此属性没有最小大小,但最大大小为 65536 字节。

16.2. 属性的布局

作为属性,它与命名数据属性并无不同。
内容取决于 \$DATA 流的名称。

表 2.40. \$LOGGED_UTILITY_STREAM (0x100) 属性的布局

抵消	尺寸	描述
~	~	标准属性标头
0x00		任何数据

16.3. 注释

16.3.1. 其他信息

所需信息

对此属性的操作会像普通的元数据更改一样记录到日志 (\$LogFile)中。

由加密文件系统 (EFS) 使用。所有加密文件都具有名为 \$EFS 的此属性。

可以是创作者选择的任何内容。
EFS 的使用方式如下:待修复:输入此信息,并不断
验证。(AIA)

第三章 NTFS 文件

1. 概述

NTFS 卷上的所有内容都是文件。文件分为两类:元数据和普通文件。元数据文件包含有关卷的信息,而 Normal 文件包含您的数据。

1.1 卷的布局

下面列出了在 Win2K 卷 (密钥)上找到的文件。

表 3.1. 卷上的文件布局

索引节点	文件名	操作系统	描述
0	\$MFT		主文件表 - 每个文件的索引
1	\$MFTMirr		MFT前4条记录的备份副本。
2	日志文件		事务日志文件
3	成交量		序列号、创建时间、脏标记
4	属性定义		属性定义
5	. (点)		磁盘的根目录
6	\$Bitmap		包含卷的集群映射 (已用与未用)
7	\$Boot		卷的启动记录
8	\$BadClus		列出卷上的不良簇
9	配额	北领地	配额信息
9	安全	2K	卷使用的安全描述符
10	上移		用于排序的大写字符表
11	\$Extend	2K 目录	:\$ObjId,\$Quota,\$Reparse,\$UsnJrnl
12-15	<未使用>		标记为使用中,但实际上是空的
16-23	<未使用>		标记为未使用
任何 \$ObjId		每个文件分配	2000 个唯一 ID
任何 \$Quota		2K	配额信息
任何 \$Reparse		2K	重解析点信息
任何 \$UsnJrnl		2K	加密日志
>24 A_File			普通文件
>24 A_Dir			普通目录
...	...		...

1.2 注释

1.2.1. 未使用的 inode

在新格式化的卷上,inode 0x0B 到 0x0F 被标记为正在使用,但实际上是空的。inode 0x10 到 0x17 被标记为空闲且未使用。只有当音量承受很大压力时,这种情况才会改变。

当 \$MFT 变得非常碎片化时,它将无法放入一个文件记录中,这时就需要一个扩展记录。需要这样做。如果只是在 \$MFT 的末尾分配一条新记录,就会遇到问题。
\$DATA 属性描述新记录的位置,该属性位于新记录中。

因此,新记录从 inode 0x0F 开始分配。\$MFT 始终至少为 16 个文件记录,因此始终存在。当 inode 0x0F 到 0x17 用完后,将使用更高、未重新分配的 inode。

这种影响可能不仅限于 MFT,但还需要更多证据。

1.2.2 其他信息

出于某种原因,\$ObjId、\$Quota、\$Reparse 和 \$UsnJrnl 的 inode 编号没有小于 24 的,就像……其余元数据文件。

此外,每个系统文件的序列号始终等于其 MFT 记录号。从未修改过。

2. NTFS 文件:\$MFT (0)

2.1 概述

在NTFS文件系统中,磁盘上的所有内容都是一个文件。甚至元数据也是以文件集的形式存储的。

主文件表 (MFT) 是卷上每个文件的索引。对于每个文件,MFT 都维护着一组记录由称为属性的记录组成,每个属性存储不同类型的信息。

2.2. \$MFT 属性

表 3.2. \$MFT 属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$MFT
0x80	\$DATA	[未命名]
0xB0	位图	[未命名]

2.3 文件布局

2.3.1. 未命名数据流

每个文件的描述都打包到 FILE 记录中。

如果一条记录不够长(这种情况不常见),则需要 \$ATTRIBUTE_LIST 属性。

前 24 条 FILE 记录保留给系统文件。完整列表请参见“文件”页面。

表 3.3. \$MFT 成立之初的样本记录

索引节点文件名	描述
0	\$MFT主文件表 - 每个文件的索引
1	\$MFTMirrMFT前4条记录的备份副本。
2	日志文件事务日志文件
3	成交量序列号、创建时间、脏标记
...	...

2.4. 注释

2.4.1. MFT 区

为了防止主文件表 (MFT) 碎片化,Windows 会在其周围维护一个缓冲区。不会有新文件被写入其中。
缓冲区中会创建数据,直到其他磁盘空间用完为止。缓冲区大小可配置,并且可以
缓冲区大小可以是磁盘空间的 12.5%、25%、37.5% 或 50%。每次磁盘剩余空间用完时,缓冲区大小都会相应增加。
更糟。

2.4.2 其他信息

MFT是自我参照的。

MFT预留了一些空间用于未来扩展。MFT记录12-15被标记为正在使用中,但是
记录为空。MFT 记录 16 至 23 被标记为未使用,但实际上从未被使用过。

在 Windows 系统中,MFT (主文件表)在系统运行时无法缩小。

3. NTFS 文件 :\$MFTMirr (1)

3.1 概述

这是一个系统文件,它至少复制了 MFT 的前四个 FILE 记录,以便进行恢复。

如果卷的簇大小小于或等于 MFT 记录大小的四倍,即通常情况下
如果簇大小小于或等于 4096 字节,则前四个 MFT 记录存储在
\$MFTMirr。

如果簇大小大于 MFT 记录大小的四倍,则 \$MFTMirr 的大小为一个簇
并且其中存储了集群内可容纳的所有 MFT 记录。

例如,给定一个 MFT 记录大小为 1024 字节,簇大小为 8192 字节,则 \$MFTMirr
长度为 8192 字节,包含 MFT 的前八个 FILE 记录。

3.2. \$MFTMirr 属性

表 3.4. \$MFTMirr 属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$MFTMirr
0x80	\$DATA	[未命名]

3.3 文件布局

3.3.1. 未命名数据流

\$MFT 的前四个 FILE 记录的副本。

表 3.5. \$MFTMirr 的布局

索引节点	文件名	描述
0	\$MFT	主文件表 - 每个文件的索引
1	\$MFTMirr	MFT前4条记录的备份副本。
2	日志文件	事务日志文件
3	成交量	序列号、创建时间、脏标记
4	...	如果存在,则从 MFT 中获取更多 FILE 记录 (参见 \$MFT)

4. NTFS 文件:\$LogFile (2)

4.1 概述

4.2. 日志文件属性

表 3.6. 日志文件属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	日志文件
0x80	\$DATA	[未命名]

4.3 文件布局

4.3.1. 未命名数据流

人们对日志文件的结构知之甚少。

4.4 注释

4.4.1 其他信息

日志区域由一系列 4KB 的日志记录组成。

每个日志记录的结构如下：

偏移量 (长度)内容	
0(4)	神奇数字 “RCRD”
1E(12)	修复

日志记录据称包含一系列大小不一的记录,但其结构尚不明确。文件 2 为 \$LogFile,其中包含事务记录,以确保在系统故障时数据完整性。如第 37 页所述,它由两个重启区域副本和一个“无限”日志区域组成。

当您想向存储单元写入文件时,您不仅需要更新文件本身,还需要更新文件系统中的一些表(例如文件日期)。此时,您需要一个包含两个操作的事务(更新文件本身,更新文件日期)。

如果交易成功,您可以确保文件已写入存储单元,并且文件系统已保持在已定义的状态。

如果事务未能完成(例如,由于断电或系统故障),文件系统将处于未定义状态。要将其恢复到已定义(且合理)的状态(此操作称为回滚),唯一的方法是将事务中已成功完成的操作记录到一个特殊的文件 日志文件中。

系统发生故障后,首次访问磁盘时,系统会读取日志文件,并将所有操作回滚到上一次事务的开头。

- 当系统写入日志文件时,该操作必须是原子性的且立即执行的。
- 您可以快速将卷恢复到正常状态,所需时间与磁盘大小无关,仅取决于失败事务的复杂程度。注意:此操作并非由 Windows NT chkdsk 实用程序执行,而是由系统自动执行。这种正常可靠的操作是 NTFS 的一项特性。
- 如果您的硬件可靠,您可以确保始终可以访问卷中的所有文件,因为它运行稳定。但您无法恢复可能发生的数据丢失。

日志文件组织方式:

前两页(重启页)中有两个重启区域。卸载卷后,它们应该完全相同。

接下来是按页组织的日志记录,每页都以记录头开头,直至日志文件大小。

卷首次格式化时,并非所有页面都包含日志记录,但随着卷的使用时间延长,所有记录都将被使用。

当日志文件填满时,开头的记录会被清除(可能是通过将 oldest_lsn 的值提高来清除),然后从文件开头重新开始写入。实际上,日志文件被视为一个循环实体。

日志文件重启页面标题(重启区域开始):

结构体 {
NTFS_RECORD;
__u64 chkdsk_lsn;

__u32 system_page_size; 系统页的字节大小,必须是

}

关键在于“RSTR”。
此重启页面的磁盘检查日志文件序列号。

仅当魔法值更改为“CHKD”时使用。= 0

大于等于 512 且为 2 的幂。利用此方法计算所需大小。

	将 usa 的值添加到 ntfs.usa_offset 值中。然后验证结果是否小于 restart_offset 的值。= 0x1000 字节日志文件记录的大小,必须大于等于 512 且为 2 的幂。= 0x1000 字节从记录的起始位置到重启记录的偏移量。
__u32 log_page_size;	
__u16 重启偏移量;	
__s16 次要版本;	值必须按 8 字节边界对齐。= 0x30 日志文件次版本号。仅检查主版本号是否为 1。(等于 1 和大于等于 1 的情况相同,小于等于 0 的情况也有效)
__u16 major_ver;	日志文件主版本号 (=1 但 =0 也可接受)
} RESTART_PAGE_HEADER;	

日志文件重启区域记录：

该记录的偏移量是通过将 RESTART_PAGE_HEADER 的偏移量添加到其中找到的 re-start_offset 值来找到的。

结构体 { __u64 current_lsn; __u16 log_clients;	日志文件记录。= 0x700000, 0x700808 重启区域之后的日志客户端记录数。= 1
__u16 client_free_list; 有多少客户端空闲?如果 !=	0xffff,检查 log_clients > client_free_list。= 0xffff
__u16 client_in_use_list; 有多少客户端正在使用中?	如果 != 0xffff,请检查 log_clients > client_in_use_list。= 0
__u16 标志; ??? = 0 __u32 序列号位; ??? = 0x2c 或 0x2d __u16 重启区域长度;重启区域的长度。	如果版本匹配,则需要进行以下检查。否则,跳过这些检查。restart_offset + restart_area_length 必须小于等于 system_page_size。 此外,restart_area_length 必须大于等于 client_array_offset + (log_clients * 0xa0)。= 0xd0 __u16 client_array_offset; 从此记录开头算起的偏移量 如果版本匹配,则偏移量为第一个客户端记录。否则,偏移量为零。 假定为 (sizeof(RESTART_AREA) + 7) 且 ~7,即向上取整到第一个 8 字节边界。无论如何,客户端数组的偏移量必须与 8 字节边界对齐。此外, restart_offset + 客户端数组的偏移量必须小于等于 510。同时,客户端数组的偏移量 + (log_clients * 0xa0) 必须小于等于 SystemPageSize (即 0x30,日志文件的字节大小) 。如果 restart_offset + 文件大小的偏移量大于 510,则表示数据已损坏。这是启动时的第一个检查。
__u64 文件大小;	

如果 restart_area 失败,则意味着上述某些值将因多扇区传输保护而损坏!当然,如果结构体已解除保护,则这些检查将毫无意义。

计算文件大小位,并检查序列号位是否等于 0x43 - 文件大小位= 0x400000 __u32
last_lsn_data_length;??? = 0, 0x40 __u16 record_length;

此记录的字节大小。如果版本匹配,则检查

record_length 的值是 8 的倍数,即 (record_length + 7) & ~7 ==
record_length= 0x30 __u16 log_page_data_offset;??? = 0x40 }
RESTART_AREA;

日志文件客户端记录:

但它却从 0x58 开始,尽管据我所知,它应该从 0x60 开始。这有点蹊跷。/-:

```
结构体 { __u64
    oldest_lsn;
    // 此客户端记录的最早日志文件序列号 = 0xbd16951d

    __u64 client_restart_lsn; ??? = 0x700000, 0x700827, 0x700d07 __u16 prev_client; ??? = 0x808, 0xd07, 0xd5d __u16 next_client; ??? = 0x70
    __u16 seq_number; ??? = 0, 4 大小不确定,Regis 将其称为“卷清除标志”,大小为 1 字节。??? = 空字符串??? 大小不确定

    __u16 客户端名称;
} RESTART_CLIENT;
```

注意:上述客户端记录后面跟着 0xffffffff,可能表示重启区域的结束。

然后是 8 个字节 = 0,接着是一个 __u32 = 8,然后是 Unicode 字符串“NTFS”,最后是零直到页面末尾。这重要吗?

日志页面记录页面标题:

每个日志页面都以该标头开头,后面跟着几个 LOG_RECORD 结构。

```
结构体 {
    NTFS_RECORD; union
    { __u64
        last_lsn; __u32 file_offset; }
    copy; __u32 flags; __u16
    page_count;
    __u16 page_position;
    union { struct {
        // 关键在于“RCRD”。
        __u64 next_record_offset; __u64 last_end_lsn; }
        packed; } header;

    } RECORD_PAGE_HEADER;
```

日志记录的可能标志：

枚举

```
{ LOG_RECORD_MULTI_PAGE = 1,          ? ? ?
  LOG_RECORD_SIZE_PLACEHOLDER = 0xffff, 这与日志记录无关。
```

这样做只是为了让 gcc 知道要将标志位设置为 16 位。

```
} LOG_RECORD_FLAGS;
```

日志记录头：

```
struct { __u64
    this_lsn; __u64 client_previous_lsn;
    __u64 client_undo_next_lsn; __u32 client_data_length;
    struct { __u16 seq_number; __u16 client_index; } client_id;
    __u32 record_type; __u32 transaction_id;

    LOG_RECORD_FLAGS flags; __u16
        reserved_or_alignment[3]; *** 现在位于结
    构体的 ofs 0x30 处。 ***
```

```
__u16 重做操作; __u16 撤销操作; __u16 重
做偏移量; __u16 重做长度; __u16 撤销偏移
量; __u16 撤销长度; __u16 目标属性;
__u16 后续 lcns;
```

此条目之后的 lcn_list 条目数。

```
__u16 record_offset; __u16
attribute_offset; __u32 alignment_or_reserved;
__u32 target_vcn; __u32 alignment_or_reserved1;
struct { 仅当 lcns_to_follow 不为
0 时存在。 __u32 lcn; __u32 alignment_or_reserved; }
lcn_list[0]; } LOG_RECORD;
```

重启区域（理论上）包含指向日志区域的指针，例如写入的第一条和最后一条日志记录以及写入的最后一条检查点记录。如果重启区域损坏，恢复将非常困难，因此需要两份重启区域的副本。

各个日志记录由逻辑序列号 (LSN) 标识。日志区域会循环，但 LSN 不会（至少短期内不会），因此使用 LSN 而不是日志文件中的偏移量来标识日志记录。

任何对元数据的修改（例如更新文件系统打开的时间戳）都会导致日志文件发生变化，进而导致重启区域发生变化。脏位很可能是隐式的而非显式的：如果最后一条日志记录表明没有待处理的事务，则文件系统是干净的。

5. NTFS 文件 :\$Volume (3)

5.1 概述

这是一个包含卷信息的系统文件。

5.2. \$Volume 属性

表 3.7. 成交量属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	成交量
0x50	\$SECURITY_DESCRIPTOR	
0x60	\$VOLUME_NAME	
0x70	成交量信息	
0x80	\$DATA	[未命名]

5.3 文件布局

5.3.1. 未命名数据流

\$DATA 属性的长度为零。

5.4. 注释

5.4.1 其他信息

卷的序列号存储在 \$Boot 中。

这是唯一使用 \$VOLUME_NAME 和 \$VOLUME_INFORMATION 文件的元数据文件属性。

6. NTFS 文件 :\$AttrDef (4)

6.1 概述

这是一个系统文件,其中包含有关卷中所有可用文件属性的信息。

属性结束标记 0xFFFFFFFF

6.2. \$AttrDef 属性

表 3.8. \$AttrDef 属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	属性定义
0x50	\$SECURITY_DESCRIPTOR	
0x80	\$DATA	[未命名]

6.3 文件布局

6.3.1. 未命名数据流

它的布局是一系列记录。每条记录定义一个文件属性,其布局如下:

表 3.9. \$AttrDef 的布局

抵消	尺寸	描述
0x00	128	Unicode 标签
0x80	4	类型
0x84	4	显示规则
0x88	4	排序规则
0x8C	4	旗帜
0x90	8	最小尺寸
0x98	8	最大尺寸

6.3.2. 显示规则

目前该值始终为零。

6.3.3. 排序规则

目前该值始终为零,但可能的值有:

表 3.10. \$AttrDef 排序规则

旗帜	描述
0x00	二进制
0x01	文件名
0x02	Unicode 字符串
0x10	未签名长
0x11	SID
0x12	安全哈希
0x13	多个无符号长整型

6.3.4. 旗帜

我们只观察到三个标志位:0x02、0x40 和 0x80。0x40 和 0x80 似乎从未同时出现过。因此,我们猜测:

表 3.11. \$AttrDef 标志

旗帜	描述
0x02	索引
0x40	居民 (始终)
0x80	非居民 (允许)

6.4 注释

- \$PROPERTY_SET 曾短暂存在于 NTFS v3.0 中。它的目的是支持本地结构存储。
年龄 (NSS)。
- 已过时;\$VOLUME_VERSION 和 \$SYMBOLIC_LINK 出现在 WinNT 中,但未使用。
它们在Win2K或WinXP中均未出现。

6.4.1 其他信息

应该可以向此文件添加用户自定义属性。

\$AttrDef 的值很大,是吗?36K?

是的,在 nt4 中 = 36K 大部分是空白

现在 2560 = 15 个属性 + 1 个空白 (2.5K)

6.5 示例

6.5.1. Windows NT 示例

表 3.12. Windows NT 中的 \$AttrDef 示例

类型名称		旗帜	国际注册网络	最小尺寸	最大尺寸
0x10	标准信息	0x40	拉	0x30	0x30
0x20	属性列表	0x80 N		-	-
0x30	\$FILE_NAME	0x42	和	0x44	0x242
0x40	\$VOLUME_VERSION	0x40	拉	0x8	0x8
0x50	\$SECURITY_DESCRIPTOR	0x80 N		-	-
0x60	\$VOLUME_NAME	0x40	拉	0x2	0x100
0x70	成交量信息	0x40	拉	0xC	0xC
0x80	\$DATA	0x00		-	-
0x90	\$INDEX_ROOT	0x40	拉	-	-
0xA0	索引分配	0x80 N		-	-
0xB0	位图	0x80 N		-	-

类型名称		旗帜	国际注册网络	最小尺寸 最大	尺寸
0xC0	\$SYMBOLIC_LINK	0x80 N		-	-
0xD0	\$EA_INFORMATION	0x40	拉	0x8	0x8
0xE0	每股	0x00		-	0x10000

6.5.2. Windows 2000 和 Windows XP 示例

表 3.13. Windows 2000/XP 中的 \$AttrDef 示例

类型名称		旗帜	国际注册网络	最小尺寸 最大	尺寸
0x10	标准信息	0x40	拉	0x30	0x48
0x20	属性列表	0x80 N		-	-
0x30	\$FILE_NAME	0x42	和	0x44	0x242
0x40	\$OBJECT_ID	0x40	拉	-	0x100
0x50	\$SECURITY_DESCRIPTOR	0x80 N		-	-
0x60	\$VOLUME_NAME	0x40	拉	0x2	0x100
0x70	成交量信息	0x40	拉	0xC	0xC
0x80	\$DATA	0x00		-	-
0x90	\$INDEX_ROOT	0x40	拉	-	-
0xA0	索引分配	0x80 N		-	-
0xB0	位图	0x80 N		-	-
0xC0	\$REPARSE_POINT	0x80 N		-	0x4000
0xD0	\$EA_INFORMATION	0x40	拉	0x8	0x8
0xE0	每股	0x00		-	0x10000
0xF0	\$PROPERTY_SET 0x100	?	?	?	?
\$LOGGED_UTILITY_STREAM		0x80 N		-	0x10000

7. NTFS 文件：.（根目录）（5）

7.1 概述

NTFS 文件系统的根目录,称为.（点）,是一个普通的目录。如果卷有重解析点，那么该目录将有一个名为\$MountMgrDatabase的命名数据流。请参阅目录页面。了解更多信息。

7.2. 点 (.) 属性

表 3.14. 点 (.) 属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	.
0x50	\$SECURITY_DESCRIPTOR	

类型	描述	姓名
0x80	\$DATA	\$MountMgr数据库
0x90	\$INDEX_ROOT	130美元
0xA0	索引分配	130美元
0xB0	位图	130美元

7.3 文件布局

7.3.1. \$MountMgrDatabase 数据流

此数据流仅在卷上存在重解析点时存在。它由重复的数据组成。
群体：

表 3.15. 点 (.) 的布局

抵消	尺寸	描述
0x00	4	入口大小
0x04	4	标志位? (位域?)
0x08	2	偏移量到 UNC 路径
0x0A	2	UNC路径的大小
0x0C	2	数据偏移
0x0E	2	数据大小

7.4. 注释

7.4.1 其他信息

更多信息请参见目录页面。

8. NTFS 文件:\$Bitmap (6)

8.1 概述

此文件列出了正在使用的簇。文件中的每一位代表一个 LCN。

8.2. 位图属性

表 3.16. 位图属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$Bitmap
0x80	\$DATA	[未命名]

8.3 文件布局

8.3.1. 未命名数据流

最低位代表编号最小的 LCN。因此：

表 3.17. \$Bitmap 的布局

少量	液晶网络
00000001 2	0
00000010 2	1
00000100 2	2
...	ETC

8.4 注释

8.4.1. MFT 区

为了防止主文件表 (MFT) 碎片化,Windows 会在其周围维护一个缓冲区。在其他磁盘空间用完之前,不会在这个缓冲区中创建新文件。

缓冲区大小可配置,可以是磁盘空间的 12.5%、25%、37.5% 或 50%。每次磁盘剩余空间用完时,缓冲区大小减半。

8.4.2 其他信息

此文件的大小始终是 8 字节 (64 个簇)的倍数。由于这种向上取整,\$Bitmap 表示的簇数将略多于磁盘实际拥有的簇数。这些位始终设置为 1。

引导扇区的备份副本位于这片无人区,因此该集群被标记为正在使用中。

理论上,在文件体积非常小的情况下,这个属性可以驻留。但实际上,Windows 会崩溃。

9. NTFS 文件 :\$Boot (7)

9.1 概述

这是系统启动所需的系统文件。

此元数据文件指向卷的引导扇区。

它包含有关体积大小、簇和 MFT 的信息。

这是唯一一个无法移动的文件。

9.2. 启动属性

表 3.18. 启动属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$Boot
0x50	\$SECURITY_DESCRIPTOR	
0x80	\$DATA	[未命名]

9.3 文件布局

9.3.1. 未命名数据流

表 3.19. \$Boot 的布局

抵消	尺寸	描述
0x0000	3	跳转到引导加载程序例程
0x0003	8	系统 ID: “NTFS”
0x000B	2	每扇区字节数
0x000D	1	每个集群的部门
0x000E	7	未使用的
0x0015	1	媒体描述符 (a)
0x0016	2	未使用的
0x0018	2	每条轨道上的扇形
0x001A	2	头数
0x001C	8	未使用的
0x0024	4	通常为 80 00 80 00 (b)
0x0028	8	成交量中的扇区数量
0x0030	8	\$MFT 的 VCN 0 的 LCN
0x0038	8	\$MFTMirr 的 VCN 0 的 LCN
0x0040	4	每个 MFT 记录的聚类数 (c)
0x0044	4	每个索引记录的聚类数 (c)
0x0048	8	卷序列号
~	~	~
0x0200		Windows NT 加载器

9.4.

(a)媒体描述符 0xF8 表示硬盘。

(b) 在格式化为 NTFS 的 USB 闪存驱动器上发现了值为 80 00 00 00 的内容。
Windows XP 系统。请注意,这是可移动存储介质,未进行分区,整个驱动器已格式化为 NTFS 文件系统。

(c) 这可以是负值,这意味着 MFT/索引记录的大小小于一个簇。
在这种情况下,MFT/索引记录的大小 (以字节为单位)等于 2^(-1 * 每个 MFT/索引记录的簇数)。
例如,如果每个 MFT 记录的簇数为 0xF6 (十进制为 -10) ,则 MFT 记录大小为 2^(-1 * -10)

= 2^10 = 1024 字节。

9.5. 注释

9.5.1 其他信息

前 40 个字节与 FAT 引导扇区相同,只是未使用的字段被置零。

由于此文件以引导扇区开头,因此它必须从物理簇 0 开始 (这是唯一——个…… NTFS 文件系统无法移动文件)。这强制该文件的数据属性为非驻留数据。因此,引导扇区 (关键数据)的副本可以位于卷上的任何位置。

为了进行崩溃恢复,Windows NT 3.51 会保存一份引导扇区的副本并将其放置在逻辑目录中。放在卷的中间。Windows NT 及更高版本放在卷的末尾。

10. NTFS 文件:\$BadClus (8)

10.1 概述

此元数据文件包含卷上所有坏簇的列表。

该文件内容稀疏,仅有的数据运行指向了不良集群。

文件自然无法读取。

10.2. \$BadClus 属性

表 3.20. \$BadClus 属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$BadClus
0x80	\$DATA	[未命名]
0x80	\$DATA	坏

10.3 布局

10.3.1. 未命名数据流

长度始终为零。

10.3.2. 错误数据流

这是一个与卷大小相同的文件。任何正常的簇都用稀疏 (零)簇表示。任何错误簇指向磁盘上的该簇。

10.4. 注释

10.4.1 其他信息

如果一个星型集群包含至少一个坏扇区,则该集群为坏星型集群。

因为这个系统文件的工作方式与其他任何文件一样,所有坏的簇都会在 \$Bitmap 系统文件中被标记为已使用,因此它们永远不能被任何其他文件使用。

NTFS 支持热修复:不再像 FAT 那样纠结于“中止、重试还是失败?”。如果在系统运行时发现新的坏簇,它会被静默地添加到此文件中。如果该坏簇位于容错卷上,则容错卷驱动程序 fdisk 会重建数据,并将 NTFS 数据存储到另一个空闲簇中。

- 具有空的未命名数据流。
- 具有名为 (\$Bad) 的数据流,卷大小 (稀疏)
- 整个簇体积 (VCN)
- 已分配空间 = 卷大小 (字节)
- 属性大小 = 卷大小 (字节)
- 初始大小 = 0 (或上述值之一,冗余)
- 运行意味着卷的文件大小稀疏
- 初始化为 0 表示完全稀疏

本文件处理的是簇,而不是扇区。簇是 NTFS 文件系统中最小的磁盘空间单位。使用。

11. NTFS 文件:\$Secure (9)

11.1 概述

在 NTFS v1.2 中,每个文件都有一个 \$SECURITY_DESCRIPTOR 属性。每次访问文件时都读取和检查这些属性效率低下,而且大多数属性的值都相同。

NTFS v3.0 引入了一个新的元数据文件 \$Secure。

\$STANDARD_INFORMATION 中的一个新字段 “安全 ID”是 \$Secure 的索引。

存在一个数据流 \$SDS,以及两个索引 \$SII 和 \$SDH。

数据流包含卷上每个 \$SECURITY_DESCRIPTOR 属性的副本,索引交叉引用所有内容。

11.2. 安全属性

表 3.21. 安全属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	安全

类型	描述	姓名
0x80	\$DATA	\$SDS
0x90	\$INDEX_ROOT	\$SDH
0x90	\$INDEX_ROOT	捐赠
0xA0	索引分配	\$SDH
0xA0	索引分配	捐赠
0xB0	位图	\$SDH
0xB0	位图	捐赠

11.3 文件布局

11.3.1. \$SDS 数据流

安全描述符流 (\$SDS) 包含卷上所有安全描述符的列表。

每个条目都填充到 16 字节边界,并带有用于索引的哈希值。

表 3.22. \$Secure:\$SDS 的布局

抵消	尺寸	描述
0x00	4	安全描述符的哈希值
0x04	4	安全 ID
0x08	8	此条目在此文件中的偏移量
0x10	4	此条目的大小
0x04	在	自相对安全描述符
V+0x04	P16	填充

按安全 ID 排序
自相关?== 有 2 * SID
通常是一个大文件,并非全部都会用到。
可能缺少条目 - 测试
开头是一大块 ID,然后是一些乱码,接着是另一块 256KB 的数据。

11.3.2. \$SDH 指数

安全描述符哈希 (\$SDH) 索引

表 3.23. \$Secure:\$SDH 的布局

抵消	尺寸	价值	描述
~	~	~	标准索引标题
0x00	2	0x18	数据偏移
0x02	2	0x14	数据大小
0x04	4	0x00	填充
0x08	2	0x30	索引条目的大小

抵消	尺寸	价值	描述	
0x0A	2	0x08	索引键的大小	
0x0C	2		旗帜	
0x0E	2	0x00	填充	
0x10	4		钥匙	安全描述符的哈希值
0x14	4		钥匙	安全 ID
0x18	4		数据	安全描述符的哈希值
0x1C	4		数据	安全 ID
0x20	8		数据	安全描述符偏移量（以 \$SDS 为单位）
0x28	4		数据	安全描述符大小（以 \$SDS 为单位）
0x2C	P8		数据	填充

最后填充始终为 4 个字节,并且始终显示为 Unicode 字符串 “\0”。

11.3.3. \$SII 指数

证券识别指数 (\$SII)

表 3.24. \$Secure:\$SII 的布局

抵消	尺寸	价值	描述	
~	~	~	标准索引标题	
0x00	2	0x14	数据偏移	
0x02	2	0x14	数据大小	
0x04	4	0x00	填充	
0x08	2	0x28	索引条目的大小	
0x0A	2	0x04	索引键的大小	
0x0C	2		旗帜	
0x0E	2	0x00	填充	
0x10	4		钥匙	安全 ID
0x14	4		数据	安全描述符的哈希值
0x18	4		数据	安全 ID
0x1C	8		数据	安全描述符偏移量（以 \$SDS 为单位）
0x24	4		数据	安全描述符大小（以 \$SDS 为单位）

该文件已按哈希值排序。
安全描述符存储在 \$SDS 数据流中。
令人惊讶的是,偏移量（64 位不是 8 字节对齐的）

11.4. 注释

11.4.1. 问题

- 为什么有些文件仍然具有 \$SECURITY_DESCRIPTOR 属性？
- 安全哈希是如何生成的？

12. NTFS 文件:\$UpCase (10)

12.1 概述

这是一个128KB的文件,里面全是大写字母。

Unicode 字母表中的每个字符在这个文件中都有一个条目。

它用于比较和排序文件名。

12.2. \$UpCase 属性

表 3.25. \$UpCase 属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	上移
0x80	\$DATA	[未命名]

12.3 文件布局

12.3.1. 未命名数据流

表 3.26. \$UpCase 的布局

抵消	特点
~	~
0x82	一个
0x84	B
0x86	C
~	~

12.4. 注释

12.4.1. 其他信息

此文件允许系统独立于代码页比较文件名。

13. NTFS 文件:\$Extend (11)

13.1 概述

Windows 2000 引入了一个新的元数据文件目录。

这是一个包含元数据文件的目录:\$ObjId,\$Quota,\$Reparse 和 \$UsnJrnl。

13.2. 扩展属性

表 3.27. 扩展属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$Extend
0x90	\$INDEX_ROOT	130美元

13.3 文件布局

13.3.1. \$I30 指数

这是一个普通目录。此文件没有数据流。

13.4. 注释

13.4.1. 其他信息

因为这个目录下最多只有四个文件,所以永远不需要任何操作。
\$INDEX_ALLOCATION 和 \$BITMAP。

14. NTFS 文件:\$ObjId (任意)

14.1 概述

此系统文件是卷上所有正在使用的 \$OBJECT_ID 属性的索引。请参阅
更多详情请参见 \$OBJECT_ID 页面。

14.2. \$ObjId 属性

表 3.28. \$ObjId 属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$ObjId
0x90	\$INDEX_ROOT	\$O
0xA0	索引分配	\$O

类型	描述	姓名
0xB0	位图	\$O

14.3 文件布局

14.3.1. \$O 指数

表 3.29. \$ObjId:\$O 的布局

抵消	尺寸	价值	描述	
~	~	~	标准索引标题	
0x00	2	0x20	数据偏移	
0x02	2	0x38	数据大小	
0x04	4	0x00	填充	
0x08	2	0x58	索引条目的大小	
0x0A	2	0x10	索引键的大小	
0x0C	2		旗帜	
0x0E	2	0x00	填充	
0x10	16		钥匙	GUID 对象 ID
0x20	8		数据	MFT 参考
0x28	16		数据	GUID 出生量 ID
0x38	16		数据	GUID 出生对象 ID
0x48	16		数据	GUID 域 ID

14.3.2. 旗帜

表 3.30. \$ObjId 标志

旗帜	描述
0x01	条目包含子节点
0x02	最后一条记录

14.4. 注释

14.4.1. 其他信息

该索引名为\$O。这是对象 ID 的索引。请勿将其与索引混淆。
同名,与元数据文件 \$Quota 所用名称相同。

索引 \$O 按 GUID (0x13) 排序。此排序规则在索引根中指定。

文件的 \$OBJECT_ID 属性包含一个 GUID,该 GUID 可以在此索引中找到。索引的数据提供指向文件的 MFT 引用。

15. NTFS 文件 :\$配额 (NT:9,2K:任意)

15.1 概述

该文件最初出现在 Windows NT 中,但当时并未被使用。在 Windows 2000 及更高版本中,它用于跟踪文件。配额。

配额是按人头和按体积计算的。

15.2. 配额属性

表 3.31. 配额属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	配额
0x90	\$INDEX_ROOT	\$O
0x90	\$INDEX_ROOT	\$Q
0xA0	索引分配	\$O
0xA0	索引分配	\$Q
0xB0	位图	\$O
0xB0	位图	\$Q

15.3 文件布局

15.3.1. \$O 指数

表 3.32. \$Quota:\$O 的布局

抵消	尺寸	价值	描述	
~	~	~	标准索引标题	
0x00	2	0x1C	数据偏移	
0x02	2	0x04	数据大小	
0x04	4	0x00	填充	
0x08	2	0x20	索引条目的大小	
0x0A	2	0x0C	索引键大小 (K)	
0x0C	2		旗帜	
0x0E	2	0x00	填充	
0x10	K		钥匙	SID
K+0x10	4		数据	所有者 ID
K+0x14	P		数据	填充8

旗帜?

15.3.2. \$Q 指数

标题和重复组

表 3.33. 配额布局:\$Q

抵消	尺寸	价值	描述	
~	~	~	标准索引标题	
0x00	2	0x14	数据偏移	
0x02	2		数据大小	
0x04	4	0x00	填充	
0x08	2		索引条目的大小	
0x0A	2	0x04	索引键的大小	
0x0C	4	0x00	填充	
0x10	4		钥匙	所有者 ID
0x14	4	0x02	数据	版本
0x18	4		数据	旗帜
0x1C	8		数据	已用字节数
0x24	8		数据	变更时间
0x2C	8		数据	警告限制
0x34	8		数据	硬限制
0x3C	8		数据	超时
0x44	在		数据	SID
V+0x44	P	0x00	数据	填充8

SID 可能缺失 (配额标志 = 默认限制 => 没有 SID,只有填充)
可能不需要填充物
索引键 - 指向哪个索引的交叉引用?
更改时间 - 日期/时间
超时时间 - 2001年10月4日 (不是+5天)
在最后一个 (空)条目中,0x0C 处的填充值为 0x02

15.3.3. 旗帜

表 3.34. 配额标志

旗帜	描述
0x0001	默认限制
0x0002	已达上限
0x0004	ID已删除
0x0010	已启用跟踪
0x0020	已启用强制执行
0x0040	已请求跟踪

旗帜	描述
0x0080	对数阈值
0x0100	日志限制
0x0200	过时
0x0400	腐败
0x0800	待删除

15.4. 注释

15.4.1. 其他信息

该索引名为\$O。这是所有者 ID 的索引。请勿将其与……的索引混淆。
与元数据文件 \$ObjId 使用的名称相同。

文件所有者 ID 存储在 \$STANDARD_INFORMATION 属性中。所有者 ID 可以在 \$O 中查找,以提供安全 ID (SID);或在 \$Q 中查找,以提供配额信息。

\$Q 索引包含每个现有 user_id 的一个条目。
卷。索引键是拥有此卷的用户/组的 user_id。
配额控制条目,即键是 owner_id。user_id 为文件所有者 (即 owner_id)位于标准数据库中。
信息属性。\$Q 的排序规则是 COLLATION_NTFS_ULONG。

\$O 索引包含每个用户/组的条目,这些用户/组都已……
已为该卷分配配额。索引键保存了 SID。
条目所属的 user_id,即 owner_id。排序规则 \$O 的规则是 COLLATION_NTFS_SID。

\$O 索引条目数据是对应用户的 user_id。
到 SID。
此 user_id 用作 \$Q 的索引,以查找配额控制项。
与 SID 关联的条目。

16.NTFS 文件:\$Reparse (任意)

16.1 概述

Win2K 可以将卷和共享挂载到现有目录之上。这是软件管理的一部分。
一部分来自体积本身。

16.2. 重新解析属性

表 3.35。 \$重新解析属性

类型	描述	姓名
0x10	标准信息	
0x30	\$FILE_NAME	\$Reparse
0x90	\$INDEX_ROOT	\$R

类型	描述	姓名
0xA0	索引分配	\$R
0xB0	位图	\$R

16.3 文件布局

16.3.1. \$R 指数

表 3.36. \$Reparse:\$R 的布局

抵消	尺寸	价值	描述
~	~	~	标准索引标题
0x00	2	0x1C	数据偏移
0x02	2	0x00	数据大小
0x04	4	0x00	填充
0x08	2	0x20	索引条目的大小
0x0A	2	0x0C	索引键的大小
0x0C	2		旗帜
0x0E	2	0x00	填充
0x10	4		关键重解析标签（和标志）
0x14	8		关键 MFT 参考重解析点
0x1C	4	0x00	按键填充（对齐到 8 字节）

0xA000003 标志 - 请参阅 \$REPARSE_POINT
没有数据!

16.4. 注释

16.4.1. 其他信息

需要更多信息。

17. NTFS 文件:\$UsnJrnl（任意）

17.1 概述

日志文件的用户可读版本。

17.2. \$UsnJrnl 属性

表 3.37. \$UsnJrnl 属性

类型	描述	姓名
0x10	标准信息	

类型	描述	姓名
0x30	\$FILE_NAME	\$UsnJrnl
0x80	\$DATA	\$J
0x80	\$DATA	最大

17.3 文件布局

17.3.1. \$J 数据流

重复组

表 3.38. \$UsnJrnl:\$J 的布局

抵消	尺寸	描述
0x00	4	入口大小
0x04	2	主要版本
0x06	2	次要版本
0x08	8	MFT 参考
0x10	8	父母 MFT 参考
0x18	8	此条目的偏移量 (单位:美元)
0x20	8	时间戳
0x28	4	原因
0x2B	4	来源信息
0x30	4	安全ID
0x34	4	文件属性
0x38	2	文件名大小 (以字节为单位)
0x3A	2	文件名偏移量
0x3C	在	文件名
V+0x3C	P	填充 (对齐到 8 字节)

17.4.

17.4.1. \$Max 数据流

表 3.39. \$UsnJrnl:\$Max 的布局

抵消	尺寸	描述
0x00	8	最大尺寸
0x08	8	分配差额
0x10	8	美国海军 ID (a)
0x18	8	最低有效美国海军编号

(a)在 USN Journal 2.0 版本中,微软使用 FILETIME 64 位值对 USN 进行随机化处理。ID。但是,未来的版本可能会使用其他方式生成 ID,因此不能想当然地认为 ID 始终如此。即期刊创建时间。

17.5. 注释

版本号

当前版本号为 2.0 (主版本号 = 2,次版本号 = 0) 。

原因标志

表 3.40. \$UsnJrnl 原因标志

旗帜	描述
0x01	该文件的一个或多个命名数据流中的数据被覆盖。
0x02	文件或目录已添加。0x04 文件或目录已截断。
0x10	该文件的一个或多个命名数据流中的数据被覆盖。
0x20	为该文件添加了一个或多个命名数据流。
0x40	该文件的一个或多个命名数据流被截断。
0x100	该文件或目录是首次创建。
0x200	文件或目录已被删除。
0x400	用户更改了文件或目录的扩展属性。这些 NTFS at 属性无法被 Windows 应用程序访问。
0x800	文件或目录的访问权限发生了更改。
0x1000	文件或目录已重命名,此结构中的文件名是之前的文件名。 姓名。
0x2000	文件或目录已重命名,此结构中的文件名是新名称。
0x4000	用户更改了 FILE_ATTRIBUTE_NOT_CONTENT_INDEXED 属性。 也就是说,用户将文件或目录从可进行内容索引的文件或目录更改为不能,或者反之亦然。
0x8000	用户已更改一个或多个文件或目录属性,或者更改过一次或多次 邮票。
0x10000	向文件或目录添加或删除了 NTFS 硬链接。
0x20000	文件或目录的压缩状态从“已压缩”更改为“已压缩”。
0x40000	文件或目录已被加密或解密。
0x80000	文件或目录的对象标识符已更改。
0x100000	文件或目录中包含的重解析点已更改,或者重解析点已更改 文件或目录被添加或删除。
0x200000	已将命名流添加到文件或从文件中删除,或者已将命名流添加到文件或从文件中删除。 已更名。
0x80000000	文件或目录已关闭。

来源信息标志

表 3.41. \$UsnJrnl 源信息标志

NTFS 文件

旗帜	描述
0x01	该操作提供有关文件或目录所做更改的信息。 操作系统。一个典型的应用场景是远程存储系统移动数据。 从外部存储到本地存储。远程存储是一种分层存储管理方式。 软件。这种做法通常至少会增加一些成本。在 USN_REASON_DATA_OVERWRITE (0x01) 标志添加到 USN 记录。
0x02	该操作会将私有数据流添加到文件或目录中。例如： 病毒检测器正在添加校验和信息。当病毒检测器修改该项目时， 系统生成 USN 记录。USN_SOURCE_AUXILIARY_DATA (0x02) 表示修改并未更改应用程序数据。
0x04	该操作创建或更新复制文件的内容。例如，文件 复制服务在复制目录中创建或更新文件时会设置此标志。

第四章 NTFS概念

1. 概述

当词汇表中的几行文字不足以解释清楚的时候。

1.1 索引

表 4.1. NTFS 概念

概念	描述
属性标头	标准属性标头
属性 ID	MFT 文件记录中使用的属性 ID
B*Tree	平衡树数据结构,用于保存 NTFS 目录树。
集群	LCN、VCN、尺寸
整理	排序和搜索
压缩	文件和目录级压缩
数据运行	数据运行
目录	NTFS 文件系统上的典型目录。
文件	NTFS 文件系统上的典型文件。
文件记录	MFT 文件记录
文件名命名空间	允许的文件名
文件引用	文件引用
修复	行业修复
索引标题	标准索引标题
INDEX记录	目录索引
链接	硬链接和符号链接
重启	日志文件重启区域
SID	内置安全标识符
疏	稀疏文件

2. 概念 - 属性标头

2.1 概述

每个 MFT 记录中的每个属性都有一个标准头部。头部存储有关属性的类型、大小、名称（可选）以及它是否驻留的信息。

属性的大小取决于两件事:它是否有名称?它是否驻留?为了简化说明
表格中将完整显示所有四种可能性（其中一些值已填写）。

2.2. 标准属性标头

2.2.1. 居民,无名氏

表 4.2. 居民未命名属性头的布局

抵消	尺寸	价值	描述
0x00	4		属性类型 (例如 0x10,0x60)
0x04	4		长度 (包括此标题)
0x08	1	0x00	非居民旗帜
0x09	1	0x00	名称长度
0x0A	2	0x00	名称偏移量
0x0C	2	0x00	旗帜
0x0E	2		属性 ID (a)
0x10	4	L	属性长度
0x14	2	0x18	属性偏移量
0x16	1		索引标志
0x17	1	0x00	填充
0x18	L		属性

(a)每个属性都有一个唯一的标识符

2.2.2. 居民,指定

表 4.3. 居民命名属性标题的布局

抵消	尺寸	价值	描述
0x00	4		属性类型 (例如 0x90,0xB0)
0x04	4		长度 (包括此标题)
0x08	1	0x00	非居民旗帜
0x09	1	N	名称长度
0x0A	2	0x18	名称偏移量
0x0C	2	0x00	旗帜
0x0E	2		属性 ID (a)
0x10	4	L	属性长度
0x14	2	2N+0x18	属性偏移量 (b)
0x16	1		索引标志
0x17	1	0x00	填充
0x18	2N	Unicode	属性名称
2N+0x18	L		属性 (c)

(a)居民属性不能压缩。

(b)每个属性都有一个唯一的标识符。

(c)向上取整到 4 字节的倍数。

2.2.3. 非居民,无姓名

表 4.4. 非居民未命名属性头的布局

抵消	尺寸	价值	描述
0x00	4		属性类型 (例如 0x20,0x80)
0x04	4		长度 (包括此标题)
0x08	1	0x01	非居民旗帜
0x09	1	0x00	名称长度
0x0A	2	0x00	名称偏移量
0x0C	2		旗帜
0x0E	2		属性 ID (a)
0x10	8		启动 VCN
0x18	8		最后一个VCN
0x20	2	0x40	数据运行偏移量
0x22	2		压缩单元尺寸 (b)
0x24	4	0x00	填充
0x28	8		属性 (c) 的分配大小
0x30	8		属性的实际大小
0x38	8		流的初始数据大小 (d)
0x40	...		数据运行

(a)每个属性都有一个唯一的标识符

(b)压缩单元尺寸 = 2^x 簇。0 表示未压缩

(c) 这是属性大小向上取整到聚类大小的结果

(d)何时这不等于分配的大小？

2.2.4. 非居民,指定

表 4.5. 非居民命名属性头的布局

抵消	尺寸	价值	描述
0x00	4		属性类型 (例如 0x80,0xA0)
0x04	4		长度 (包括此标题)
0x08	1	0x01	非居民旗帜
0x09	1	N	名称长度
0x0A	2	0x40	名称偏移量
0x0C	2		旗帜
0x0E	2		属性 ID (a)
0x10	8		启动 VCN

抵消	尺寸	价值	描述
0x18	8		最后一个VCN
0x20	2	2N+0x40	数据运行偏移量 (b)
0x22	2		压缩单元尺寸 (c)
0x24	4	0x00	填充
0x28	8		属性的分配大小 (d)
0x30	8		属性的实际大小
0x38	8		流的初始数据大小 (e)
0x40	2N	Unicode	属性名称
2N+0x40	...		数据运行 (b)

(a)每个属性都有一个唯一的标识符

(b) 向上取整到 4 字节的倍数

(c)压缩单元尺寸 = 2^x 簇。0 表示未压缩

(d) 这是属性大小向上取整到聚类大小的结果

(e)何时这不等于分配的大小？

2.2.5. 旗帜

表 4.6. 属性标志

旗帜	描述
0x0001	压缩
0x4000	加密
0x8000	疏

2.3 注释

2.3.1 其他信息

只有数据属性才能被压缩或稀疏化,并且只有当它是非驻留数据时才能进行压缩或稀疏化。

虽然压缩标志存储在头部,但它并不影响头部的大小。

名称不以空字符结尾

待修复
0x40 __s64 压缩大小;
压缩后属性值的字节大小。
仅在压缩时存在。始终是簇的倍数。
大小。表示磁盘上实际使用的磁盘空间量。

待修复:索引标志仅出现在居民属性中。这是否意味着您只能对居民进行索引？

识别属性？

3. 概念 - 属性 ID

3.1 概述

每个文件记录中的每个属性都有一个属性 ID。该 ID 在文件记录中是唯一的,用于维护数据完整性。

链接到文件记录,文件记录的每个字段都有一个id,当id为零时会重复使用。

下一个属性 ID

将分配给添加到此 MFT 记录的下一个属性的属性 ID。

注意:每次使用时递增。

注意:每次重用 MFT 记录时,此 ID 都会设置为零。

注意:第一个实例编号始终为 0。

4. 概念 - B*树

4.1 概述

B+树

固定顺序高度,在添加/删除按键时保持平衡,对指针的干扰最小,仅向下。

4.2 基本术语

· 钥匙

一个承载数据的物体

· 叶子

一把没有孩子的钥匙

· 节点

一系列钥匙

· 命令

阶数为 n 的节点最多有 n-1 个键

· 树

有序数据结构

- 根节点

没有父节点的节点

- 中位数

节点中第 $\text{ceil}((n-1)/2)$ 个键

- 兄弟姐妹

同一个节点中存在两个键,或者两个节点具有相同的父节点。

- 深度

树状图中的层数。祖父母、父母、子女 = 3

- b树

一棵平衡的树

- b+树

节点至少半满的平衡树

- b*树

节点至少有 $2/3$ 被填满的平衡树

4.3. NTFS 树

索引根索引分配虚拟
键数据位于非叶子键中磁盘指针
仅指向下方

我们目前所掌握的情况

...

概述

...

添加规则

找到第一个比新键大的键（这必然是一个叶子节点）。

将新键插入到此键之前（在同一节点中）。
当节点已满时

将当前节点拆分为两个
将中位键提升到父级
现在考虑一下父母

结尾

删除规则

删除密钥
如果密钥有孩子
 找到后继节点并将其移动到此节点
 现在考虑后继节点的旧节点
结尾
虽然节点尚未完全饱和
 如果兄弟姐妹有足够的钥匙,就偷一把。

别的
 与其中一个兄弟姐妹合并
结尾
结尾

4.4 讨论

来自 IRC 频道 #ntfs 的讨论记录。

flatcap:嗨 _Oracle_ _Oracle_ 嗨 flatcap:
有什么可以帮你的吗?

Oracle:我在想NTFS的B+树是怎么回事。Oracle:它们看起来有点怪,或者至少 跟我预想的不一樣。) flatcap:它们确实看起来很奇怪,但它们非常适合文件系统。Oracle:不,我是指它们在磁盘上的表示。Oracle:它们有某种虚拟节点吗?flatcap:NTFS中的树不是真正的B+树。flatcap:一个虚拟键。Oracle:这正是我希望听到的!flatcap: (今天早上思维有点混乱,请见谅:-)

Oracle:没问题;-) flatcap:树由节点组成,节点包含键。
flatcap:在实际 (理想情况下)的 B+ 树中,键只是分隔符,而 d 是分隔符。Oracle:对,_Oracle_:顺便问一下,NTFS 下的节点有多大?我的意思是,一个节点能容纳多少个键?flatcap:INDX 记录是 4k,可以容纳几十个文件名。flatcap:但是……这取决于文件名的长度。Oracle:我以为节点中的键数是 B+ 树的固定属性。flatcap:呵呵,通常是这样。
flatcap:NTFS 的键实际上包含数据,以及指向其键的指针。Oracle:所以我注意到了。AntonA:应该补充一点,也见过 2k 大小的 INDX 记录。Oracle:真的吗?

Oracle:什么操作系统?
AntonA:NT4 flatcap:因为子键
比键多一个,所以必须有一个虚拟键 (_Oracle_:有意思……)

AntonA:我的某些目录 (例如 c:\winnt 和 c:\program files)中有 2k 个元素。_Oracle_:所以虚拟键总是 “最大”的吗?flatcap:是的。_Oracle_:我明白了……

Oracle:所以如果非叶子节点包含自身的数据,那岂不是……flatcap:我刚写了一个测试程序来帮助我理解树,Oracle:我很想看看。flatcap:我看了很多网页,我觉得最接近的术语是 ab*tree。Oracle:它和B树有什么区别?flatcap:B树至少保留1/2个完整节点 (根节点除外) 。flatcap:ab*tree稍微改变了规则,保留2/3个完整节点。Oracle:所以它只是改变了将两个节点合并为一个节点之类的规则?flatcap:正是如此。

Oracle:嗯……

Oracle:让我想想 :) flatcap:在真正的 B+ 树中,数据键 (叶子节点)也应该有指向 flatcap 的指针:我很快会把我所知道的关于 NTFS 树的所有内容都写出来 Oracle:让我看看我理解得对不对……

Oracle:索引根指向根 INDX 记录。flatcap:你可以在这里看到我的测试程序: <http://linux-ntfs.bkbits.net:8080/tng-su>。Oracle:每个 INDX 记录都包含指向文件本身的指针。flatcap:是的。Oracle:我看到了。flatcap:索引根位于 MFT 记录中。Oracle:是的,我发现了 :) flatcap:

其余的 (索引分配)都是非资源性的。Oracle:单个 INDX 记录中的键的数量完全灵活吗?

AntonA:是的。flatcap:是的,但

是存在最小值。AntonA:最小值?flatcap:是的,那是树算法的一部分。AntonA:最小值肯定是一个不包含数据的终止符条目吧?

Oracle:最小值是多少?flatcap:b+树的最小值是1/2满, b*树的最小值是2/3满。flatcap:只有根节点可以包含更少的元素。_Oracle_:哦。

Oracle:是啊 AntonA:还有最后

一个节点…… flatcap:键值会移动以保持这个值不变 flatcap:即使是最后

一个节点也会有“正确的数字” AntonA:那意味着在一个非常大的目录中,删除一个文件可能会…… flatcap:不,你可能会这么想,但平衡操作不会影响很多其他文件 flatcap:如果树的深度是4 (相当于NTFS的 10^5 个文件),你只会…… flatcap:等我有空的时候 (可能明天)我会画很多图

Oracle:那应该很有意思!flatcap:Oracle,你在我们的开发者邮件列表里吗?Oracle:什么邮件列表? (呃……没有。)

AntonA:我脑海中浮现的主要问题是,Windows NTFS Flatcap 会怎么样?呵呵,我都不敢想 :-)

Oracle:我肯定不想去那里。flatcap:chkdsk 可能会尝试重新平衡它,你可能会发现 ntf。Oracle:我该如何加入邮件列表?flatcap: <http://lists.sourceforge.net/lists/listinfo/linux-ntfs-dev> AntonA:嗯,如果 Windows 能够顺利地处理这些碎片,那就容易多了。flatcap:我会给邮件列表发邮件并在那里回答问题。AntonA:如果 Windows 能够顺

利地处理这些碎片,我……flatcap:是的,有可能,但我认为我现在掌握的知识足以构建

一个类似的……flatcap: (我只是想要一个大项目,这样我就可以顺利上手,而不会遇到什么问题。)AntonA:好的。Oracle:如果你有时间,我还有几个问题。AntonA:正如我之前所说,我不会碰任何目录。flatcap:当然。Oracle:不过,小一些的目录可以。

4.5 参考文献

以下是在我编写 B 树代码时发现的一些很有帮助的网站。

<http://tide.it.bond.edu.au/inft320/003/lectures/physical.htm>
<http://cis.stvincent.edu/carlsond/swdesign/btree/btree.html>
<http://www.fit.qut.edu.au/~maire/baobab/baobab.html>
<http://www.fit.qut.edu.au/~maire/baobab/lecture/index.htm>

5. 概念 集群

5.1 概述

在 NTFS 文件系统中,簇是磁盘使用的基本单位。组成一个簇的扇区数始终是 2 的幂,并且在卷格式化时就已确定。这个数字称为簇因子,通常以字节为单位,例如 8KB、2KB。NTFS 通过逻辑簇号来寻址所有数据。

5.1.1. 逻辑簇号 (LCN)

卷中的每个簇都被赋予一个顺序编号,即逻辑簇号 (LCN)。LCN 0 (零)指的是卷中的第一个簇 (引导扇区)。

要将 LCN 转换为卷中的物理偏移量,请将 LCN 乘以簇大小。

5.1.2. 虚拟集群编号 (VCN)

每个非驻留流的集群都被赋予一个顺序编号。这就是它的虚拟集群编号。

VCN 0 (零)指的是流的第一个簇。

为了在磁盘中找到数据流,需要将VCN转换为LCN。这可以通过数据运行来实现。

5.1.3. 数据运行

每个连续的 LCN 块都会被赋予一个数据运行,其中包含一个 VCN、一个 LCN 和一个长度。

当 NTFS 需要查找磁盘上的对象时,它会在数据运行中查找 VCN 以获取 LCN。

5.2 注释

5.2.1. 其他信息

格式化卷时可以选择簇大小。

卷的簇大小存储在 \$Boot 中。此外,MFT 文件记录和索引记录的大小 (以簇为单位)也定义在该位置。

通过使用簇号,NTFS 可以寻址比使用扇区号更大的磁盘。

下面列出了允许的集群大小和默认集群大小。

Windows NT

512字节、1KB、2KB或4KB

Windows 2000、Windows XP

512字节、1KB、2KB、4KB、8KB、16KB、32KB或64KB

表 4.7. 默认集群大小

体积	默认集群大小
<512MB	行业规模
小于1GB	1KB
小于2GB	2KB
2GB以上	4KB

5.2.2. 问题

为什么NTFS使用虚拟簇编号？

6. 概念 - 整理

6.1 概述

能够在 NTFS 文件系统中搜索和排序对象。

表 4.8. 排序类型

价值	姓名	比较这些值：
0x00	二进制	二进制,其中第一个字节是最重要的
0x01	文件名	Unicode字符串
0x02	Unicode	Unicode 字符串,但大小写字母应排在前面。
0x10	头	无符号长整型 (32 位,小端序)
0x11	SID	安全标识符
0x12	安全哈希	首先按安全性进行比较 然后通过安全标识符进行哈希运算
0x13	乌隆斯	一组无符号长整型 (32 位,小端序)

6.2 用法

以下是一些使用各种排序规则的示例。

表 4.9. 标准索引的默认排序规则类型

姓名	使用者
头	文件 \$SECure 中的 \$SII
SID	文件 \$Extend/\$Quota 中的 \$O
安全哈希	文件 \$Secure 中的 \$SDH
乌隆斯	文件 \$Extend/\$ObjId 中的 \$O

6.3 注释

6.3.1. 问题

使用 ULONGS 进行比较时,最大长度在哪里指定?或者,两个对象的 ULONGS 是否永远不可能相同?

0x13 ULONGS 指的是 GUID 测试

7. 概念 压缩

7.1 概述

以下是该机制的简要概述:数据。这些数据使用改进的LZ77算法进行压缩。
基本思想是,对于之前已经出现过的子字符串,通过引用而不是重复提及来压缩它们。例如,考虑以下纯文本

```
#include <ntfs.h>\n#include
<stdio.h>\n
```

这被压缩为 #include <ntfs.h>\n (-18,10)stdio(-17,4)

因此,该算法识别出从当前位置向前移动 18 个字节时,已经出现过文本 “#include <” 。然后,stdio 是新的,但 “.h>\n”之前已经出现过。

有趣的细节就在问题中:如何对 (-18, 10) 这对数字进行编码,以及如何将其与纯文本字符串混合。首先要理解的是,这样的数字对需要占用两个字节。由于反向引用需要两个字节,因此反向引用一个或两个字节的子串是没有意义的。这意味着最短的子串长度为 3。也就是说,长度为 0、1 和 2 的值是不可能的。

因此,您可以在编码前将长度减去 3。此外,引用始终是反向的,且永远不会为 0。所以您可以将它们存储为正数,然后减 1。第一个反向引用存储为 (17,7),第二个存储为 (16,1)。

假设一个数据块的大小为 4096,那么您可能需要 12 位来编码反向引用。这意味着你只剩下 4 位来编码长度,最大长度为 19。这并不理想,因为它将压缩比限制在 1:19。另一方面,如果当前偏移量为 123,则反向引用为 -512 是不可能的。一些聪明的微软工程师决定动态地为反向引用分配更多位数,而为长度分配更少位数。具体的分配方式可以用表格或其他方式表示。

```
for(i=clear_pos-1,lmask=0xFFF,dshift=12;i>=0x10;i>=1){ lmask >>= 1; /* 长度位掩码 */ dshift    /* 偏移量
      宽度 */
}
```

现在我们可以将 (偏移量, 长度) 对编码为两个字节,但我们仍然需要知道一个标记是反向引用还是纯文本。每个标记对应一个比特。八个标记被分组在一起,并在前面加上标签字节。因此,该组

>\n(18,10)stdio

将被编码为

00000100 > \n 0A 90 标准输出

(1 位表示反向引用)。极端情况下,全部为空格的块 () 会被压缩为

00000010 FC 0F

或者 (-1,4095)。之所以有效,是因为你总是读取刚刚存储的数据。由于一个压缩单元由 16 个簇组成,因此它通常包含多个这样的块。如果你想访问第二个块,解压缩第一个块会浪费时间。因此,每个块前面都有一个 2 字节的长度。低 12 位是长度,高 4 位用途未知。

待修复:属性头中的压缩单元大小为 2^4 。
该压缩方法基于对 X 个簇的数据块进行独立压缩,其中 X 由非驻留属性记录头中的 compression_unit 值确定 (更准确地说: $X = 2^{\text{compression_unit}}$ 个簇)。在 Windows NT/2k 系统中,X 始终为 16 个簇 (compression_unit = 4)。

- 1)该数据块中的数据全部为零 (稀疏数据块) :
这以稀疏块的形式存储在运行列表中,即运行列表条目的长度为 X,lcn 为 -1。映射对数组实际上使用长度为 0 的 delta_lcn 值,即 delta_lcn 根本不存在,然后驱动程序将其解释为 lcn = -1。

注意:即使是未压缩的文件在 NTFS 3.0 卷上也可能是稀疏的,那么上述原则同样适用,只是长度不受任何特定值的限制。

- 2)该数据块中的数据未压缩:
当压缩未能有效减小块在簇中的大小时,就会发生这种情况。也就是说,如果压缩效果甚微,压缩后的数据仍然占用 X 个簇,那么未压缩的数据就会存储在该块中。

这种情况可以通过运行列表条目的长度为 X 且 lcn >= 0 来识别。映射对数组会像往常一样存储此条目,运行长度为 X,并且具有一些特定的 delta_lcn,即必须存在 delta_lcn。

- 3) 数据块中的数据已压缩:常见情况。这种情况的识别依据是运行列表项的长度 $L < X$ 且 lcn >= 0。映射对数组会按常规方式存储此数据,运行长度为 X,并具有特定的 delta_lcn 值,即必须存在 delta_lcn 值。此运行列表项之后紧跟一个稀疏项,其长度为 $X - L$,lcn = -1。后一个稀疏项用于补足 vcn 计数,使其达到完整的压缩块大小 X。

事实上,情况更加复杂,因为相同类型的相邻条目可以合并。这意味着必须跟踪已处理的簇的数量,并以每次处理 X 个簇为一个块的方式进行操作。例如:如果长度 $L > X$,则表示此特定运行列表条目包含一个长度为 X 的块,以及一个或多个长度为 L 的块的一部分。

L - X。另一个例子:如果长度 L < X,这并不一定意味着该数据块已被压缩,因为数据块内部的 lcn 可能发生变化,因此后续的运行列表条目描述了可能已被压缩的数据块的延续。如果后续的运行列表条目描述了至少 X - L 个稀疏簇,从而构成了如上文第 3 点所述的压缩块长度,则该数据块将被压缩。(当然,可能存在多个长度较小的运行列表条目,因此稀疏条目不会紧随第一个长度 < X 的数据条目之后。)

注意:在压缩属性值的末尾,很可能数据量不足以构成一个压缩块,因此这些数据根本不会尝试压缩,而是直接存储。

如果你仔细观察这个算法,你会发现它并非总能减小数据大小。如果没有反向引用,每个字节的纯文本都将保持不变。然而,每隔 8 个字节会插入一个标签位,该标签位的值将变为零。因此,在最坏的情况下,一个数据块的大小可能会增长到 4610 字节 (包括数据块的长度)。如果数据块的大小增长,它将以未压缩的形式存储。长度正好为 4095 字节时,就用这个值来表示这种情况。下一个数据块仍然有可能被很好地压缩,从而减小整个数据块的大小。如果不能压缩,则整个数据块将以未压缩的形式存储,这在运行列表中会有相应的指示。

每个数据块前面都有一个 2 字节的长度标识。低 12 位是长度,高 4 位用途未知。

位 0x8000 是一个标志位,用于指定数据块是否已压缩。压缩代码会将该位的值 0xB000 (如果已压缩)与数据块进行逻辑或运算,而解压缩代码只检查位 0x8000。

此外,长度实际上存储在低 12 位中的 (n-3)。实际上,如果不计算用于存储长度本身的两个字节,则长度为 (n-1)。因此,对于未压缩的数据块,长度存储为 0xFFFF,这意味着长度为 4096 加上另外两个用于存储长度本身的字节。

长度为 0x1000 的数据块压缩到长度为 0x500 时需要 0x502 字节,并且其公布长度为 0x4FF。

我不知道一个完全不压缩的 16 簇文件是否需要 17 个簇来存储,才能容纳每个块额外的 2 个字节。

我相信只需要16个集群。未压缩的事实会在运行列表中体现出来。
例如,压缩后的文件看起来会像这样

(1000 A) (0 6) // (rel.VCN 长度)

而不可压缩的文件看起来会是这样的

(1000 10)

或者

(1000 A) (1040 6)

换句话说,如果在这 16 个集群中没有任何 VCN==0 的运行,则该数据块完全未压缩。
而且很简单。考虑到压缩算法,创建这样的文件相当容易:

```
s=
for i in range(0,16): #如有必要,调整聚类数以大于 512

s=s+chr(i)+chr(j)
打开( 不可压缩的 , w ).写入(s)
```

8. 概念 - 数据运行

8.1 概述

非驻留属性存储在称为“运行”的簇区间中。每个运行由其起始簇及其长度表示。运行的起始簇被编码为相对于下一个运行起始簇的偏移量。

上一次运行。

普通文件、压缩文件和稀疏文件都是由运行定义的。

例子从简单开始,然后很快变得复杂。

这是一个写入非驻留文件属性的内容部分的表格,它允许访问它的溪流。

注意:假设簇大小为 1KB,并采用小端磁盘存储。

8.2 布局

运行列表是一个元素序列:每个元素存储到前一个运行列表起始 LCN 的偏移量。
元素和运行簇的长度。

为了节省空间,偏移量和长度是可变大小的字段(可能最多 8 个字节),并且一个元素是以这种精简格式写成:

表 4.10. 数据运行布局

以半字节为单位的偏移量 开始 元素	尺寸	描述
0	1	F=偏移字段的大小
1	1	L=长度字段的大小
2	2升	跑步长度
2+2*L	2*F	相对于前一个元素的起始 LCN 的偏移量

相对于前一个元素的起始 LCN 的偏移量

这是一个有符号值。对于第一个元素,偏移量应相对于 LCN 0 (起始位置)而言。
体积。

运行列表的布局必须考虑数据压缩:包含以下信息的VCN集合

压缩文件属性流被分割成 16 个簇的压缩单元（也称为块）.VCN 0 到 15 构成第一个压缩单元,VCN 16 到 31 构成第二个压缩单元,依此类推……对于每个压缩单元，

- 压缩的初始阶段非常简单,并且与用于压缩文件的压缩引擎无关:如果一个压缩单元的所有 16 个簇都为零,则该压缩单元被称为稀疏单元,并且不会被实际存储。取而代之的是,在运行列表中放置一个没有偏移量字段（F=0,偏移量也被假定为 0）且长度为 16 个簇的元素。
- 否则,压缩的 beta 阶段由用于压缩文件的压缩引擎在属性处完成:如果单元的压缩是可能的,则物理存储 N (< 16) 个簇,并将长度为 N 的元素放入运行列表中,然后放入另一个没有偏移字段 (F=0,偏移量也假定为 0) 且长度为 16 - N 的元素。
- 否则,该单元不会被压缩,物理上存储 16 个簇,并将长度为 16 的元素放入运行列表中。

实际上,这要复杂一些,因为有些元素是可以收集的。但我们来看一个例子……
...

8.2.1. ...示例

我们需要解码以下运行列表：

运行列表:21 14
00 01 11 10 18 11 05 15 01 27 11 20 05

解码 0x14 at
0x100 21 0x100, 0x14 0x10 at + 0x18 11 0x18, 0x10 0x05 at + 0x15 11 0x15,
0x05 0x27 at + none 01 0x27, none 0x20 at + 0x05 11 0x05, 0x20

绝对 LCN:0x14（地址
0x100）、0x10（地址 0x118）、0x05
（地址 0x12D）、0x27（地址无）、
0x20（地址 0x132）

重新集结
0x10 在 0x100

0x04 位于 0x110,0x0C 位于 0x118

0x04 位于 0x118,0x05 位于 0x12D,
0x07 位于无

0x10 位于无

0x10 位于无

0x10 位于 0x132

0x10 位于 0x142

压缩单元起始于 VCN 0x0 0x10,簇位于 LCN 0x100

单元未压缩

压缩单元起始于 VCN 0x10,簇 0x4 起始于 LCN 0x110,簇 0xC 起始于 LCN 0x118

单元未压缩

压缩单元起始于 VCN 0x20,包含 0x4 个簇（位于 LCN 0x124）,0x5 个簇（位于 LCN 0x12D）,以及 0x7 个未使用的簇:压缩单元

压缩单元起始于 VCN 0x30 0x10,簇已清零:稀疏单元

压缩单元起始于 VCN 0x40 0x10,簇已清零:稀疏单元

压缩单元起始于 VCN 0x50
LCN 0x132 处有 0x10 个簇

单元未压缩

压缩单元起始于 VCN 0x60
LCN 0x142 处有 0x10 个簇

单元未压缩

~~~~~

file.txt 文件大小为 31KB（磁盘簇大小为 1KB）

它存储在集群 10-26,45-49 和 100-108 中。

LCN 10 处有 17 个聚类,LCN 45 处有 5 个聚类,LCN 100 处有 9 个聚类。

接下来,将偏移量设为相对偏移量

LCN 10 处有 17 个聚类,LCN 45 处有 5 个聚类,LCN 100 处有 9 个聚类。

编码为

11

以 16 个簇为单位进行工作,相对偏移量（包括负值）压缩稀疏可变长度结构存储为:节省空间意味着无论 MFT 将数据放在哪里,最好都不要将其分散得太远。

负数表示偏移量为 +129 时需要使用两个字节,因此 -10 = 0xF6,0x80 = -128

0xFF7F = -129

21 14 00 01 11 10 18 11 05 15 01 27 11 20 05

### 8.3 数据运行

长度和起始簇是可变大小的字段。游程的第一个字节指示这两个字段的大小。偏移量的大小存储在高半字节中,长度的大小存储在低半字节中。

对于压缩或稀疏运行,偏移量为 0,偏移量的大小也为 0。每个压缩单元的起始簇数为 16 的倍数。如果可以压缩,则单元的 VCN 处将包含一个或多个数据运行,后跟一个空运行。如果数据运行的簇数超过 16 个,则该单元不可压缩。如果完全没有数据运行(只有一段很长的空运行),则该单元全部为零。

示例:21 20 ED 05 22 48 07 48 22 21 28 C8 DB  
第一次运行:20 个簇,从 5ED 开始 (5ED 到 60D)第二次运行:748 个簇,从 5ED+2248 开始 (2835 到 2F7D)  
第三次运行:28 个簇,从 2835+DBC8 开始 (3FD 到 425)

请注意,偏移量被解释为有符号值。

取一个大小为 0x80 个簇的文件(磁盘上的任意位置)。这用 VCN (虚拟簇号)0x00 到 0x7F 表示。这些 VCN 映射到运行(或扩展)中的 LCN (逻辑簇号),例如 21 80 30 60 00。

这些序列长度可变,以零结尾。第一个字节的低半字节决定下一个数字(1 字节)的长度,即 80。高半字节决定再下一个数字(2 字节)的长度,即 6030。

结果:80 个集群,从集群 6030 开始。

“大小”存储在一个字节中。长度是无符号的。偏移量是有符号的,并且相对于前一个偏移量。

11 30 60 - 21 10 00 01 - 11 20 E0 - 00

第 1 行长度 30 偏移量 60 (相对于 0 的第一次运行)  
运行 2 长度 10 偏移量 100 + 60  
运行 3 长度 20 偏移量 160 - 20 (EO == -20)  
==  
80

文件由一组虚拟上下文 (VCN)表示。稀疏文件,简单来说,就是缺少某些虚拟上下文,例如:

21 09 F5 47 9 个来自 47F5 的聚类 01 07 7 个来自未知位置的聚类 (0) 11 07  
09 7 个来自 47F5 + 9 的聚类  
  
====  
0x17  
  
123456789ABCDEF1234... VCN RRRRRRRRRZZZZZZRRRR...  
实数/零

压缩文件首先被分割成 16 个 (0x10)簇的数据块。想象一下:

VCN0123...

XXXXXXXXXXXXX=数据 O=空间

这里的数据被压缩成十个簇（如果十六个簇中有一个都保存不了,我们就放弃）。以上代码如下：

21 0A 10 F6 10 个压缩数据簇位于 F610 01 06 6 个空簇,将此块向上取整至 16

这 6 个额外的簇实际上并不占用任何磁盘空间,VCN 被打包成 16 个一组。{ 如果一个数据块无法压缩,它将表示为：

21 10 10 F6 16 个压缩数据簇位于 F610

待修正:实际上,情况更为复杂,因为相同类型的相邻条目可以合并。这意味着必须跟踪已处理的簇的数量,并以每次处理 X 个簇为一个块的方式进行操作。例如:如果长度 L > X,则表示此特定运行列表条目包含一个长度为 X 的块以及一个或多个长度为 L - X 的块的一部分。另一个例子:如果长度 L > X,则并不意味着该块已被压缩,因为块内部的 lcn 可能发生变化,因此下一个运行列表条目描述了可能已压缩块的延续。如果下一个运行列表条目描述了至少 X - L 个稀疏簇,从而构成了如上文第 3 点所述的压缩块长度,则该块将被压缩。(当然,可能存在多个长度较小的运行列表条目,因此稀疏条目不会紧随第一个长度小于 X 的数据条目之后。)

注意:在压缩属性值的末尾,很可能数据量不足以构成一个压缩块,因此这些数据根本不会尝试压缩,而是直接存储。

压缩运行和稀疏运行可以混合使用。这一切都是为了节省空间。

## 8.4 示例

### 8.4.1. 示例 1 - 普通、未碎片化的文件

数据运行次数:21 18 34 56 00

重新分组:21 18 34 56 - 00

表 4.11. 已解析数据运行:示例 1 - 普通、未碎片化文件

| 不是<br>米 | 团体       | 标题        |               | 数据                |  |
|---------|----------|-----------|---------------|-------------------|--|
|         |          | 长度大小、偏移大小 | 长度偏移1 字节 0x18 | 1 0x5634 (2 字节) 字 |  |
| 1 21    | 18 34 56 | 节)        | 2 字节          |                   |  |

|         |    |          |      |    |  |
|---------|----|----------|------|----|--|
| 不是<br>米 | 团体 | 标题       |      | 数据 |  |
|         |    | 长度尺寸偏移尺寸 | 长度偏移 |    |  |
| 200     |    | 结尾       |      |    |  |

概括：

· 0x18 个簇 @ LCN 0x5634

因此,Data1 是一个未碎片化的文件,大小为 0x18 个簇,从 LCN 0x5634 开始。

8.4.2. 示例 2 - 普通碎片文件

数据运行:31 38 73 25 34 32 14 01 E5 11 02 31 42 AA 00 03 00

重新分组:31 38 73 25 34 - 32 14 01 E5 11 02 - 31 42 AA 00 03 - 00

表 4.12. 解析后的数据运行 :示例 2 - 普通碎片文件

|         |                     |          |        |                              |                        |
|---------|---------------------|----------|--------|------------------------------|------------------------|
| 不是<br>米 | 团体                  | 标题       |        | 数据                           |                        |
|         |                     | 长度尺寸偏移尺寸 | 长度偏移   |                              |                        |
| 1 31    | 38 73 25 34         | 1 字节     | 3 字节 3 | 0x38                         | 0x342573 (3 字节)        |
| 2 32    | 14 01 E5 11 02 2 字节 |          | 字节     | 0x114 0x363758 (相对于 0x211E5) | 0x342573)              |
| 3 31    | 42 AA 00 03         | 1 字节     | 3 字节   | 0x42                         | 0x393802 (相对于 0x300AA) |
| 4 00    |                     | 结尾       |        |                              |                        |

概括：

· 0x38 个簇 @ LCN 0x342573

· 0x114 个簇 @ LCN 0x363758

· 0x42 个簇 @ LCN 0x393802

因此,Data2 是一个碎片文件,大小为 0x18E 个簇,数据块位于 LCN 0x342573 处。0x363758 和 0x393802。

8.4.3. 示例 3 - 普通文件、乱码文件

数据运行:11 30 60 21 10 00 01 11 20 E0 00

重新分组:11 30 60 - 21 10 00 01 - 11 20 E0 - 00

表 4.13. 解析后的数据运行 :示例 3 - 正常文件、乱码文件

| 不是米  | 团体       | 标题           |        | 数据   |                           |
|------|----------|--------------|--------|------|---------------------------|
|      |          | 长度尺寸偏移尺寸     | 长度偏移   |      |                           |
| 1 11 | 30 60    | 1 字节 0x30 (1 | 1 字节   | 字节)  | 0x60 (1 字节)               |
| 2 21 | 10 00 01 | 1 字节         | 2 字节 1 | 0x10 | 0x160 (相对于 0x60 为 0x100)  |
| 3 11 | 20 E0    | 1 字节         | 字节     | 0x20 | 0x140 (相对于 0x160 为 -0x20) |
| 4 00 |          | 结尾           |        |      |                           |

概括：

- 0x30 个簇 @ LCN 0x60
- 0x10 个簇 @ LCN 0x160
- 0x20 个簇 @ LCN 0x140

因此,Data3 是一个严重碎片化的文件,大小为 0x60 个簇,数据块位于 LCN 0x60 和 0x160 处。以及 0x140。此外,第三块数据在物理上位于第一块和第二块数据之间。块。(第三次运行的偏移量为负值,使其位于前一次运行之前)。

### 8.4.4. 示例 4 - 稀疏、未碎片化的文件

数据运行:11 30 20 01 60 11 10 30 00

重新分组:11 30 20 - 01 60 - 11 10 30 - 00

表 4.14. 解析后的数据运行 :示例 4 - 稀疏、未碎片化文件

| 不是米  | 团体    | 标题           |        | 数据       |                      |
|------|-------|--------------|--------|----------|----------------------|
|      |       | 长度尺寸偏移尺寸     | 长度偏移   |          |                      |
| 1 11 | 30 20 | 1 字节 0x30 (1 | 1 字节   | 字节)      | 0x20 (1 字节)          |
| 2 01 | 60    | 1 字节         | 0 字节 1 | 0x60 不适用 |                      |
| 3 11 | 10 30 | 1 字节         | 字节     | 0x10     | 0x50 (0x30 相对于 0x20) |
| 4 00 |       | 结尾           |        |          |                      |

概括：

- 0x30 个簇 @ LCN 0x20
- 0x60 簇 (稀疏)
- 0x10 个簇 @ LCN 0x50

因此,Data4 是一个稀疏的、未碎片化的文件,大小为 0xA0 个簇,数据块位于 LCN 0x20 处。以及 0x50。

该文件中间有一段大小为 0x60 个簇的稀疏部分。它不占用磁盘空间,但它代表……

0x60 VCN 表示不满。

8.4.5. 示例 5 - 压缩的、未碎片化的文件

数据运行时间:11 08 40 01 08 11 10 08 11 0C 10 01 04 00

重新分组:11 08 40 - 01 08 - 11 10 08 - 11 0C 10 - 01 04 - 00

表 4.15. 解析后的数据运行:示例 5 - 压缩的、未碎片化的文件

| 不是<br>米 | 团体       | 标题           |      | 数据        |                        |
|---------|----------|--------------|------|-----------|------------------------|
|         |          | 长度尺寸偏移尺寸     | 长度偏移 |           |                        |
| 1       | 11 08 40 | 1 字节 0x08 (1 | 1 字节 | 字节)       | 0x40 (1 字节)            |
| 2       | 18年2月    | 1 字节         | 0 字节 | 0x08 不适用  |                        |
| 3       | 11 10 08 | 1 字节         | 1 字节 | 0x10      | 0x48 (相对于 0x40 为 0x08) |
| 4       | 11 0C 10 | 1 字节         | 1 字节 | 0x0C 0x58 | (0x10 相对于 0x48)        |
| 5       | 01 04    | 1 字节         | 0 字节 | 0x04 不适用  |                        |
| 6       | 00       | 结尾           |      |           |                        |

概括:

- 0x08 簇 @ LCN 0x40
- 0x08 聚类 (稀疏)
- 0x10 个簇 @ LCN 0x48
- 0x0C 簇 @ LCN 0x58
- 0x04 聚类 (稀疏)

因此,Data5 是一个长度为 0x30 的压缩、未碎片化文件,数据块的 LCN 为 0x40、0x48 和 0x58。

存储在磁盘上的数据是连续的。稀疏运行会将压缩单元填充到 16 个数据块中。簇 (0x10)。

8.4.6. 示例 6 - 压缩、稀疏、碎片化文件

脑损伤档案

9. 概念 - 目录

9.1 概述

在 NTFS 文件系统中,卷上的每个对象都是一个文件,甚至包括目录。目录是文件名的索引。

9.2. 属性

表 4.16. 目录记录属性

| 类型   | 描述                    | 姓名    |
|------|-----------------------|-------|
| 类型   | 描述                    | 姓名    |
| 0x10 | 标准信息                  |       |
| 0x30 | \$FILE_NAME           | 目录名   |
| 0x50 | \$SECURITY_DESCRIPTOR |       |
| 0x90 | \$INDEX_ROOT          | 130美元 |
| 0xA0 | 索引分配                  | 130美元 |
| 0xB0 | 位图                    | 130美元 |

9.2.1.

9.2.1.1. 索引条目

索引是索引条目的列表。每个条目包含文件名、标准信息和指向安全信息的指针。正确的起始位置是索引条目。

9.2.1.2. 索引根

该属性始终驻留,包含多个索引条目,是索引树的根节点。

9.2.1.3. 索引分配

一组运行信息,用于告诉系统其他索引的位置。(介词!)

9.2.1.4. 索引位图

正在使用哪些聚类(索引)?

目录甚至可以拥有命名的数据流。

9.3 定义

从人的角度来看,目录是一种特殊的文件类型,它可以包含其他文件。文件文件夹,以嵌套方式用于在卷上创建逻辑文件层次结构。

9.4. 属性

从 NTFS 的角度来看,目录是文件名的索引,或者更准确地说,是索引序列。包含文件名属性的条目。为每个文件的每个文件名属性创建一个索引条目。文件夹内包含此类索引条目。可以按字母顺序进行比较。它们的文件名属性是大写的(多亏了 \$UpCase)。

目录没有数据属性。但是,作为索引,它却有三个其他文件属性:索引根、索引分配和位图。索引以如下方式存储在 B+ 树的节点中:

树的每个节点包含一个或多个索引条目。在节点内,索引条目按以下方式排序:  
递增顺序

每个索引条目都可以指向另一个（子）节点,该节点仅包含较低级别的索引条目。

- 根节点位于索引根属性的流中,其他（子）节点是索引缓冲区。

9.5. 利息

当应用程序读取目录时,NTFS 返回一个已排序的文件名列表。

B+树结构（HPFS 中也使用这种结构）如果构建得当,效率会高得多。  
与线性结构相比,在包含大量文件的文件夹中执行文件名查找更加灵活。

虽然在索引条目中重复索引属性流可能会消耗一些时间,但  
值得推荐的是,您可以浏览索引而无需实际打开所有索引文件（FAT 和 HPFS）。  
也这样做）。

在一个目录中,三个文件属性:索引根、索引分配和位图,分别命名为“\$I30”和 “  
目录只是文件属性的索引,其类型为 30。但 NTFS 一直被视为数据库。  
文件系统,它实际上可以基于任何始终存在的文件属性创建索引。例如,你  
可以创建一个名为“作者姓名”的新文件属性,并根据该标准对文件进行排序。

10. 概念文件

10.1 概述

它由名称和数据等属性组成。

10.2. 属性

表 4.17. 文件记录属性

| 类型   | 描述                    | 姓名    |
|------|-----------------------|-------|
| 0x10 | 标准信息                  |       |
| 0x30 | \$FILE_NAME           | 文件名   |
| 0x50 | \$SECURITY_DESCRIPTOR |       |
| 0x80 | \$DATA                | [未命名] |

10.2.1. 标准信息

这包含了DOS风格的文件权限,例如只读和归档权限。它还包含了四种不同的修改时间类型。

- 文件创建时间
- 最后修改时间
- 文件记录的最后修改时间
- 最后访问时间

### 10.2.2. 文件名

文件名也作为属性存储。一个文件可以有多个文件名。这相当于 Windows 系统中将文件硬链接在一起的功能。

### 10.2.3. 安全描述符

此文件存储了Windows的所有权限,包括访问控制列表 (ACL)、访问控制规则 (ACE)和审核信息。

可能在 Win2K 上不存在 (标准信息,\$secure)

### 10.2.4. 数据

最后,这才是文件的实际数据。它也存储在一个属性中。

未命名数据流是必需的 (如果丢失,chkdsk 会将其恢复)。  
命名数据流是可选的 (数量有限吗?)

## 10.3. 命名数据流

使用 “jim.txt:stream”访问

表 4.18. 虚构的命名数据流

| 类型   | 描述     | 姓名 |
|------|--------|----|
| 0x80 | \$DATA | 图标 |
| 0x80 | \$DATA | 作者 |

## 10.4. 概要信息

Windows 2000 引入了文件摘要信息的概念。这些信息以集合的形式存储。  
四个已命名的数据流。

### 描述

- 标题
- 主题
- 类别
- 关键词 (多行)

- 注释（多行）

起源

- 来源
- 作者
- 修订号

表 4.19. 命名数据流的汇总信息

| 类型   | 描述     | 姓名                                     |
|------|--------|----------------------------------------|
| 0x80 | \$DATA | {4c8cc155-6c1e-11d1-8e41-00c04fb9386d} |
| 0x80 | \$DATA | ^文档摘要信息                                |
| 0x80 | \$DATA | ^ESebiesnrMkudrfcolaamtykdDa           |
| 0x80 | \$DATA | ^摘要信息                                  |

注意:其中三个名称以 CTRL-E (0x05) 开头。这可能是为了阻止人们阅读。  
直接传输数据。

第一个流{4c..始终为空。这可能只是一个标记。

表 4.20. 命名数据流的摘要信息内容

| 数据流                          | 摘要字段 | 数据类型    | 代码   |
|------------------------------|------|---------|------|
| ^文档摘要信息                      | 未知1  | 数字?     | 0x00 |
|                              | 未知2  | 数字      | 0x01 |
|                              | 类别   | ASCII   | 0x02 |
| ^ESebiesnrMkudrfcolaamtykdDa | 未知3  | 数字?     | 0x00 |
|                              | 未知4  | 数字?     | 0x01 |
|                              | 来源   | Unicode | 0x04 |
| ^摘要信息                        | 未知5  | 数字?     | 0x00 |
|                              | 未知6  | 数字?     | 0x01 |
|                              | 标题   | ASCII   | 0x02 |
|                              | 主题   | ASCII   | 0x03 |
|                              | 作者   | ASCII   | 0x04 |
|                              | 关键词  | ASCII   | 0x05 |
|                              | 评论   | ASCII   | 0x06 |
|                              | 修订号  | ASCII   | 0x09 |

11. 概念 文件记录

# 11.1 概述

MFT 是一组文件记录。卷中的每个文件都由一个或多个文件记录完整描述。这些文件记录。文件记录相当于 Unix 术语中的 inode。第一个文件记录描述给定文件的记录称为基本文件记录 ,其他记录称为扩展文件记录。

文件记录由头部、若干可变长度属性和一个结束标记（简称结束标记）组成。0xFFFFFFFF）。

链接到表格和注释

另请参阅 :属性、标准属性头、\$MFT、\$Boot、文件、修复、属性 ID、目录

# 11.2 布局

文件记录

标题

属性

属性

...

结束标记 (0xFFFFFFFF)

表 4.21. 文件记录的布局

| 偏移尺寸   |      | 你  | 描述                 |
|--------|------|----|--------------------|
| 0x00   | 4    |    | 魔数 “文件”            |
| 0x04   | 2    |    | 更新序列的偏移量           |
| 0x06   | 2    |    | 更新序列号和数组的大小（以字为单位） |
| 0x08   | 8    |    | 日志文件序列号 (LSN)      |
| 0x10   | 2    |    | 序列号                |
| 0x12   | 2    |    | 硬链接计数              |
| 0x14   | 2    |    | 到第一个属性的偏移量         |
| 0x16   | 2    |    | 旗帜                 |
| 0x18   | 4    |    | 文件记录的实际大小          |
| 0x1C 4 |      |    | 已分配的 FILE 记录大小     |
| 0x20   | 8    |    | 指向基本 FILE 记录的文件引用  |
| 0x28   | 2    |    | 下一个属性 ID           |
| 0x2A 2 |      | XP | 对齐到 4 字节边界         |
| 0x2C 4 |      | XP | 此MFT记录编号           |
|        | 2    |    | 更新序列号 (a)          |
|        | 2S-2 |    | 更新序列数组 (a)         |

(a)这两个字段的偏移量取决于您的操作系统。

日志文件序列号 (LSN)

每次修改记录时,此值都会改变。

序列号

此 MFT 记录被重复使用的次数。

注意:文件删除时会进行递增(跳过零)。

注意:如果此值设置为零,则其值将保持为零。

硬链接计数

硬链接数量,即引用此记录的目录条目数量。

注意:仅用于mft基础记录。

旗帜

表 4.22. 文件记录标志

| 旗帜   | 描述      |
|------|---------|
| 0x01 | 记录正在使用中 |
| 0x02 | 记录是一个目录 |
| 0x04 | 不知道     |
| 0x08 | 不知道     |

实际/分配面积

已分配大小是指记录在磁盘上占用的空间大小。它应该是簇大小的倍数,并且可能等于 MFT 文件记录的大小。实际大小是指记录实际使用的字节数。

注意:实际大小将填充至 8 字节边界。

基础 MFT 记录

对于基本 MFT 记录,此值为零。当此值不为零时,它是一个 MFT 引用,指向此记录所属的基本 MFT 记录。基本记录包含有关扩展记录的信息。此信息存储在 ATTRIBUTE\_LIST 属性中。

下一个属性 ID

将分配给添加到此 MFT 记录的下一个属性的属性 ID。

注意:每次使用时递增。

注意:每次重用 MFT 记录时,此 ID 都会设置为零。

注意:第一个实例编号始终为 0。

主文件表记录由表头和属性列表组成。其大小为 400 字节 (=1K),或簇大小(以较大者为准)。表头包含以下字段:

## 11.3. 注释

属性列表长度可变,以 FFFFFFFF 结尾。对于 1K 个 MFT 记录,属性列表从偏移量 0x30 开始。

序列号是一个循环计数器 (跳过 0),用于描述被引用的 MFT 记录被 (重新)使用的次数。该序列号必须与被引用的 MFT 记录的序列号匹配,否则该引用将被视为过期并被移除 (待修复:仅 ntfsck 或驱动程序本身?)。

如果序列号为零,则假定不应执行序列号一致性检查。

待修复:MFT 区定义为卷的前 12%。预留此空间是为了确保 MFT 可以连续增长,从而避免碎片化。卷的可用空间包括 MFT 区的空余部分。当卷的可用空间 (剩余 88%)被使用后,MFT 区将缩小一半,从而为更多文件/数据腾出空间。每当除 MFT 区外没有其他可用空间时,此过程都会重复进行,直到完全没有可用空间为止。

mft 记录头位于 mft 中每个记录的开头。

接下来是一系列可变长度的属性记录,该记录以类型为 \$END 的属性结束,这是一个截断属性,因为它只包含属性类型代码 \$END,而属性结构中的其他成员均不存在。

当 (重新)使用 MFT 记录时,我们将更新序列数组放置在此偏移量处,即在属性开始之前。这样做也是合理的,否则更新序列数组本身可能会包含扇区的最后两个字节,这意味着多扇区传输保护将无法工作。因为您不能通过覆盖数据来保护数据,因为覆盖后就无法恢复了……

读取数据时,我们显然使用的是 ntfs 记录头中的数据。

属性序列部分

这是一个长度可变的文件属性序列。在每个 FILE 记录中,该序列按属性类型升序排列。序列以 FF FF FF FF 结尾。

大小在 \$Boot 中定义。

FILE 记录的大小为 1 KB,或者大于簇大小 (就 Helen 而言,其最大大小为 4 KB,但 Windows NT 4 的限制为 64 KB)。它分为两部分:

当文件的全部信息无法全部放入基本 FILE 记录中时 (例如,文件属性序列因文件数量众多而增长,或者由于文件流非常碎片化而导致数据属性的运行列表过长),会使用扩展 FILE 记录。基本 FILE 记录仅用于引用其所描述的文件。由于属性列表文件属性的类型足够小,我们可以确定该文件属性将存在于基本 FILE 记录中。并且,该文件属性提供了对描述该文件的所有扩展 FILE 记录的引用。

当文件被删除时,NTFS 不能简单地删除 MFT 中删除关联的 FILE 记录,否则 FILE 记录号将不会随时间保持不变,所有文件引用都必须更新!

相反,FILE 记录的“使用中”标志指示该记录何时不再使用。创建新文件时,可以重复使用未使用的 FILE 记录,但其序列号会加一。这种机制

ism 允许 NTFS 检查文件引用是否指向已删除的文件。

序列号 = inode,范围为 0x00 < i < 0x10 (inode 0 (MFT)的序列号为 1)

另请参阅属性 ID 页面和文件引用页面

标志位 1 已使用,2 方向,4 ???,8??? (4+8 已使用)

## 12. 概念 - 文件参考

### 12.1 概述

MFT 中 FILE 记录的唯一标识符。

### 12.2 布局

表 4.23. 文件引用的布局

| 抵消   | 尺寸 | 描述     |
|------|----|--------|
| 0x00 | 6  | 文件记录编号 |
| 0x06 | 2  | 序列号    |

### 12.3. 注释

#### 12.3.1. 序列号

如果文件系统一致,则此数字必须与 FILE 记录号引用的 FILE 记录的序列号匹配。

MFT引用 (也称为文件引用或文件记录段引用)是当结构需要引用 mft 中的记录时使用。

参考信息由指向 MFT 的 48 位索引和 16 位序列组成。用于检测过期引用的数字。

序列号何时递增?

## 13. 概念 - 文件命名空间

### 13.1 概述

旧版本的FAT文件系统对文件名有严格的限制。许多字符都被禁止使用,而且长度最初限制为 11 个字符 (命名空间较小)。较新版本的 FAT 文件系统允许更长的命名空间。NTFS 文件系统几乎没有任何限制,支持更长的文件名和更长的字符集。

文件名会带有一个标志,用于指示该名称所属的命名空间。为了支持旧版应用程序,NTFS 会为任何 DOS 不兼容的文件分配一个简短的 DOS 友好名称。

## 13.2. 可能的命名空间

### 0:POSIX

这是最大的命名空间。它区分大小写,允许使用除 NULL (0) 和正斜杠 / 之外的所有 Unicode 字符。名称的最大长度为 255 个字符。  
注意:某些字符,例如 `<`,是不允许的。  
冒号 “:” 在 NTFS 文件系统中是有效的,但 Windows 不允许使用。

### 1:Win32

Win32 是 POSIX 命名空间的一个子集,并且不区分大小写。它使用所有 Unicode 字符,包括  
号 . 或空格 、 结尾,但以下情况除外: < 、 > 、 ? 、 \ 和 | 。注意:名称不能以点

### 2:的

DOS 是 Win32 命名空间的一个子集,仅允许使用大于空格 的 8 位大写字符,并且不包括:  
/ : ; < = > ? \ 。注意:名称必须符合以下模式:1 到 8 个字符,然后是 . ,再是 1 到 3 个字符。

### 3:Win32 和 DOS

此命名空间意味着 Win32 和 DOS 文件名相同,因此已保存在此单个文件名记录中。

要将 POSIX 或 Win32 文件名转换为 DOS 兼容的文件名,请按照以下步骤操作:

1. 删除所有 Unicode 字符
2. 删除除最后一个 (如果不是第一个字符)之外的所有点号。
3. 所有字母大写
4. 删除禁用字符
5. 将可能的 “.” 之前的所有内容截断为 6 个字符,并添加字符串 “~1”。
6. 将可能的 “.” 之后的所有内容截断为 3 个字符。
7. 如果名称已存在,则将字符串 “~1” 递增
8. 注意:步骤 7 表示,虽然生成的 DOS 名称是唯一的,但仅凭 Win32 名称无法推断出它。

## 14. 概念 - 修复

### 14.1 概述

NTFS 使用的最小磁盘空间单位是簇。簇的大小可以从 1 个扇区到 128 个扇区不等,通常为 8 个扇区 (4KB)。当然,这取决于 \$Boot 文件中声明的扇区和簇的大小。

由于单个扇区可能发生故障,因此 NTFS 必须能够检测集群中的错误。为此,

目的是为了扇区具有修复程序,这些修复程序保存在更新序列数组中。

许多重要的元数据记录使用修复程序来保护数据完整性。

- 将记录归档到 \$MFT 中
- 目录和其他索引中的 INDX 记录
- \$LogFile 中的 RCRD 记录
- \$LogFile 中的 RSTR 记录

14.2. 它的作用是什么？

每条记录的头部都包含一个更新序列号和一个缓冲区。最后两条将记录的每个扇区的字节复制到缓冲区,并写入更新序列号。代替它们。

读取记录时,从记录头读取更新序列号,并将其与……进行比较。每个扇区的最后两个字节。如果成功,则将缓冲区中的字节复制回其原始位置。地点。

14.3. 示例

以下是修复之前的示例,簇大小为 2KB（4 个扇区）。

表 4.24. 修复示例:之前

| 抵消     | 数据    |    |    |    |    |    |    |    | 描述     |
|--------|-------|----|----|----|----|----|----|----|--------|
| 0x0000 | ...   |    |    |    |    |    |    |    | 标题     |
| 0x0028 | CD AB |    |    |    |    |    |    |    | 更新序列号  |
| 0x002A | 00    | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 更新序列数组 |
| ...    | ...   |    |    |    |    |    |    |    |        |
| 0x01F8 | 11    | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 第一区结束  |
| ...    | ...   |    |    |    |    |    |    |    |        |
| 0x03F8 | 21    | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 第二区结束  |
| ...    | ...   |    |    |    |    |    |    |    |        |
| 0x05F8 | 31    | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 第三区结束  |
| ...    | ...   |    |    |    |    |    |    |    |        |
|        |       |    |    |    |    |    |    |    |        |

|        |    |    |    |    |    |    |    |          |
|--------|----|----|----|----|----|----|----|----------|
| 0x07F8 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 第四区结束 |
|--------|----|----|----|----|----|----|----|----------|

此处更新序列号为 0xABCD,更新序列数组仍为空。

表 4.25. 修复示例 :之后

| 抵消     | 数据    |    |    |    |    |          |      |    | 描述     |
|--------|-------|----|----|----|----|----------|------|----|--------|
| 0x0000 | ...   |    |    |    |    |          |      |    | 标题     |
| 0x0028 | CD AB |    |    |    |    |          |      |    | 更新序列号  |
| 0x002A | 17    | 18 | 27 | 28 | 37 | 38       | 47   | 48 | 更新序列数组 |
| ...    | ...   |    |    |    |    |          |      |    |        |
| 0x01F8 | 11    | 12 | 13 | 14 | 15 | 16 CD AB | 第一扇区 | 结束 |        |
| ...    | ...   |    |    |    |    |          |      |    |        |
| 0x03F8 | 21    | 22 | 23 | 24 | 25 | 26 CD AB | 扇区 2 | 结束 |        |
| ...    | ...   |    |    |    |    |          |      |    |        |
| 0x05F8 | 31    | 32 | 33 | 34 | 35 | 36 CD AB | 扇区 3 | 结束 |        |
| ...    | ...   |    |    |    |    |          |      |    |        |
| 0x07F8 | 41    | 42 | 43 | 44 | 45 | 46 CD AB | 第四扇区 | 结束 |        |

每个扇区的最后两个字节已被复制到更新序列数组中,并且更新序列号已被写入每个扇区的最后两个字节。

14.4 细节

14.4.1. 写作

在写入受修复保护的记录之前：

- 1. 将更新序列号加 1（必须跳过 0x0000）
- 2. 对于每个扇区,将最后两个字节复制到更新序列数组中。
- 3. 将新的更新序列号写入每个扇区的末尾。
- 4. 将记录写入磁盘

## 14.4.2 阅读

读取受修复保护的记录时：

1. 从磁盘读取记录
2. 检查魔数是否正确
3. 读取更新序列号
4. 将其与每个扇区的最后两个字节进行比较。
5. 将更新序列数组的内容复制到正确的位置
6. 如果读取过程中任何一项检查失败,则可能意味着存在 :坏扇区、磁盘损坏或故障。  
在驾驶员侧。

# 15. 概念 - 索引标题

## 15.1 概述

每个索引记录都有一个标准标头和一组包含索引键和索引数据的数据块。

索引记录的大小在 \$Boot 中定义,并且似乎始终为 4KB。

## 15.2 布局

### 15.2.1. 标准索引标题

表 4.26. 标准索引标题的布局

| 抵消   | 尺寸   | 描述                   |
|------|------|----------------------|
| 0x00 | 4    | 神奇数字 “INDX”          |
| 0x04 | 2    | 更新序列的偏移量             |
| 0x06 | 2    | 更新序列号和数组的大小 (以字为单位)  |
| 0x08 | 8    | 日志文件序列号              |
| 0x10 | 8    | 索引分配中此 INDX 缓冲区的 VCN |
| 0x18 | 4    | 索引条目的偏移量 (a)         |
| 0x1C | 4    | 索引条目的大小 (a)          |
| 0x20 | 4    | 索引条目的分配大小 (a)        |
| 0x24 | 1    | 1 如果不是叶节点 (b)        |
| 0x25 | 3    | 填充 (始终为零)            |
| 0x28 | 2    | 更新序列                 |
| 0x2A | 2S-2 | 更新序列数组               |

(a)这些值是相对于 0x18 的。

(b)有子女

### 15.3. 注释

#### 15.3.1. 常用索引列表

表 4.27. 常用索引列表

| 姓名    | 索引     | 描述        |
|-------|--------|-----------|
| 130美元 | 文件名    | 目录使用      |
| \$SDH | 安全描述符  | 安全        |
| 捐赠    | 安全 ID  | 安全        |
| \$O   | 对象 ID  | \$ObjId   |
| \$O   | 所有者 ID | 配额        |
| \$Q   | 配额     | 配额        |
| \$R   | 重解析点   | \$Reparse |

#### 15.3.2. 其他信息

索引记录中不包含描述索引存储内容的任何信息（此信息已保留）。  
在索引根中）。

## 16. 概念 - 索引记录

### 16.1 概述

这仅适用于文件索引（\$I30）

indx help describe as    index = key + data

给定一条INDX记录,很难确定其含义。  
已被索引（该信息位于索引根目录）

### 16.2 定义

这是 B+ 树的一个子节点,它实现了一个索引（例如目录）。它存储在流中。  
索引分配属性,与它所属的索引相关联。

### 16.3 布局

INDX 缓冲区至少为 2 KB,或者大于簇大小（这似乎是每个索引的参数）。它分为两部分：

#### 16.3.1. 头部部分

这不仅仅是标题……

表 4.28. 索引记录头的布局

| 抵消        | 尺寸 | 描述              |
|-----------|----|-----------------|
| ~         | ~  | 标准索引标题          |
| 0x00      | 8  | 文件 MFT 参考       |
| 0x08      | 2  | 此索引条目的大小        |
| 0x0A      | 2  | 文件名偏移量          |
| 0x0C      | 2  | 索引标志            |
| 0x0E      | 2  | 填充（对齐到 8 字节）    |
| 0x10      | 8  | 父级 MFT 文件引用     |
| 0x18      | 8  | 文件创建时间          |
| 0x20      | 8  | 最后修改时间          |
| 0x28      | 8  | 文件记录的最后修改时间     |
| 0x30      | 8  | 最后访问时间          |
| 0x38      | 8  | 文件分配大小          |
| 0x40      | 8  | 文件实际大小          |
| 0x48      | 8  | 文件标志            |
| 0x50      | 1  | 文件名长度（F）        |
| 0x51      | 1  | 文件名命名空间         |
| 0x52      | 2F | 文件名             |
| 2F+0x52   | P  | 填充（对齐到 8 字节）    |
| P+2F+0x52 | 8  | 带有子节点的索引缓冲区的VCN |

注意:文件名不能以空字符结尾。  
标志位肯定不可能只有 8 个字节长。  
旗帜表格  
ib 的 VCN 仅在 flags&1 时存在  
最后一条记录的大小为 0x10（刚好够大）。  
对于标志（以及 mft 参考值为零）

### 16.3.2. 索引条目的序列部分

这是一系列索引条目,类似于索引根属性中找到的条目。

索引条目具有以下结构:

索引条目标志 (16 位)。

INDEX\_ENTRY\_NODE = cpu\_to\_le16(1), 此条目包含一个子节点,  
即对索引的引用  
以虚拟形式存在的块  
集群编号 (见下文)。

INDEX\_ENTRY\_END = cpu\_to\_le16(2),这表示最后一个条目  
索引块。索引项  
它不代表一个文件,但它  
可以指向子节点。

这是一个索引条目。每个 INDEX\_HEADER 后面都跟着一系列这样的条目。  
结构。它们共同构成一个完整的索引。索引遵循以下任一规则。  
索引根属性或索引分配属性。

注意:在 NTFS 3.0 之前,只有文件名属性会被索引。

如果条目是最后一个,则大多数条目无效(且不存在)。此条目不代表一个文件。  
仅用于子节点。只有当条目包含子节点时,才会存在指向子节点缓冲区的指针。

## 17. 概念 - 链接

### 17.1 概述

### 17.2 利息

NTFS 不管理 POSIX 符号链接。然而,这个文件属性让我们认为 NTFS 将会……  
在 Windows NT 5.0 中管理符号链接(或用微软术语来说是重新解析点),就像所有现代 Unix 文件系统(例如 Linux 文件系统 Ext2)所做的那样。

### 17.3. 问题

此文件属性流的作用和布局是什么?

NTFS 将 POSIX 风格的硬链接表示为具有多个文件名的文件。  
NTFS 表示具有多个文件名的硬链接。  
这与一个文件在不同命名空间中存在文件名的情况不同。  
从硬链接文件中删除名称,只会删除该名称。

## 18. 概念 重启

### 18.1 概述

重启区域的每个副本大小为 4KB,结构如下:

| 偏移量 (长度) | 描述                 |
|----------|--------------------|
| 0 (4)    | 神奇数字 “RSTR”        |
| 1E(12)   | 修复                 |
| 30(4)    | LSNa               |
| 58(4)    | LSNb               |
| 60(4)    | LSNc (==LSNa? )    |
| 6C(1)    | 音量清除标志             |
| 78(8)    | Unicode 字符串 “NTFS” |

各种LSN的用途尚不明确。偏移量3C附近的数据似乎与清晰/音量也有问题。

## 19. 概念 - SID

# 19.1 概述

NT 系统预留了几个 SID。

返回第二页

秒

S-1-5-21-646518322-1873620750-619646970-1110  
S 代表安全 ID  
1 修订级别  
5 标识符授权 (48 位)5 = 登录 ID  
21 子权限 (21 = nt 非唯一)  
646518322 在  
1873620750 SA 域 ID  
619646970 在  
1110 用户 ID

表 4.29. 常见且广为人知的 SID

| SID          | 描述        |
|--------------|-----------|
| S-1-5-32-544 | 本地管理员。    |
| S-1-1-0      | 全世界 (所有人) |
| S-1-5-21     | NT 非唯一 ID |

标识符授权机构

表 4.30. 标识符授权机构

| 标识符授权   | 缩写    |
|---------|-------|
| 空 SID   | S-1-0 |
| 世界SID   | S-1-1 |
| 本地 SID  | S-1-2 |
| 创建者 SID | S-1-3 |
| 并非独一无二  | S-1-4 |
| NT SID  | S-1-5 |

相对标识符 (RID)

这些相对标识符 (RID) 与上述标识符一起使用。  
由有关部门制定普遍认可的SID。

注:相对标识符 (RID) 指的是 SID 的一部分，  
识别与颁发 SID 的机构相关的用户或组。  
例如,通用的、众所周知的 SID 创建者所有者 ID (S-1-3-0) 是  
由标识符授权 SECURITY\_CREATOR\_SID\_AUTHORITY (3) 和

相对标识符 SECURITY\_CREATOR\_OWNER\_RID (0)。

相对标识符

表 4.31. 相对标识符

| 相对标识符     | 代码 | SID        |
|-----------|----|------------|
| 无效的       | 0  | S-1-0-0    |
| 世界        | 0  | S-1-1-0    |
| 当地的       | 0  | S-1-2-0    |
| 创建者/所有者   | 0  | S-1-3-0    |
| 创作者集团     | 1  | S-1-3-1    |
| 创建者所有者服务器 | 2  | S-1-3-2    |
| 创建者组服务器   | 3  | S-1-3-3    |
| 拨号上网      | 1  | S-1-5-1    |
| 网络        | 2  | S-1-5-2    |
| 批         | 3  | S-1-5-3    |
| 交互的       | 4  | S-1-5-4    |
| 登录 ID     | 5  | S-1-5-5-XY |
| 服务        | 6  | S-1-5-6    |
| 匿名登录      | 7  | S-1-5-7    |
| 代理人       | 8  | S-1-5-8    |
| 企业控制器     | 9  | S-1-5-9    |
| 服务器登录     | 9  | S-1-5-9    |
| 校长自述      | 10 | S-1-5-10   |
| 已认证用户     | 11 | S-1-5-11   |
| 受限代码      | 12 | S-1-5-12   |
| 终端服务器     | 13 | S-1-5-13   |
| 本地系统      | 18 | S-1-5-18   |
| NT 非唯一    | 21 | S-1-5-21   |
| 内置域       | 32 | S-1-5-32   |

众所周知的域相对子权威值（RID）。

域用户

表 4.32. 域用户

| 域用户         | 代码  |
|-------------|-----|
| 行政          | 500 |
| 客人          | 501 |
| Kerberos 目标 | 502 |

## 域组

表 4.33. 域组

| 域组    | 代码  |
|-------|-----|
| 管理员   | 512 |
| 用户    | 513 |
| 宾客    | 514 |
| 计算机   | 515 |
| 控制器   | 516 |
| 证书管理员 | 517 |
| 架构管理员 | 518 |
| 企业管理员 | 519 |
| 政策管理员 | 520 |

## 域名别名

表 4.34. 域名别名

| 域名别名         | 代码  |
|--------------|-----|
| 管理员          | 544 |
| 用户           | 545 |
| 宾客           | 546 |
| 高级用户         | 547 |
| 账户运营         | 548 |
| 系统运维         | 549 |
| 打印操作         | 550 |
| 备份操作         | 551 |
| 复制器          | 552 |
| RAS服务器       | 553 |
| 2000 年前计算机访问 | 554 |

## 通用知名SID

表 4.35. 通用知名 SID

| SID     | 缩写      |
|---------|---------|
| 无效的     | S-1-0-0 |
| 世界      | S-1-1-0 |
| 当地的     | S-1-2-0 |
| 创建者/所有者 | S-1-3-0 |
| 创作者集团   | S-1-3-1 |

|           |         |
|-----------|---------|
| SID       | 缩写      |
| 创建者所有者服务器 | S-1-3-2 |
| 创建者组服务器   | S-1-3-3 |
| 非唯一ID     | S-1-4   |

NT 众所周知的 SID

表 4.36. NT 众所周知的 SID

|             |              |
|-------------|--------------|
| SID         | 缩写           |
| 北领地管理局      | S-1-5        |
| 拨号上网        | S-1-5-1      |
| 网络          | S-1-5-2      |
| 批           | S-1-5-3      |
| 交互的         | S-1-5-4      |
| 服务          | S-1-5-6      |
| 匿名登录（空登录）   | S-1-5-7      |
| 代理人         | S-1-5-8      |
| 服务器登录（域控制器） | S-1-5-9      |
| 自己          | S-1-5-10     |
| 已认证用户       | S-1-5-11     |
| 受限代码        | S-1-5-12     |
| 终端服务器       | S-1-5-13     |
| 登录 ID       | S-1-5-5-XY   |
| NT 非唯一 ID   | S-1-5-21-... |
| 内置域         | S-1-5-32     |

# 20. 概念 稀疏

## 20.1 概述

稀疏文件

修复 NT4（旧式）的数据运行页面  
13 b8 ae 04 ff 00 03 b8 ae 04 00  
NT4 稀疏数据运行中的坏簇  
使用 -1!

老的  
新的

---

# 第五章 尾声

## 1.待办事项

除非另有说明,否则每个项目都是重写/彻底修改。

紧迫的

- 安全
- 日志
- 索引根
- 属性 ID
- 文件记录

中等的

- 交叉引用 \$Secure
- 交叉引用 \$Quota
- 属性列表
- 已记录的公用流
- 压缩
- 数据运行
- 目录
- 文件
- 索引标题
- 索引记录
- 稀疏文件

低的

- 概述中的“住宅/非住宅”
- 表格 (P8)尺寸
- 数据
- 重解析点
- 文件参考

- 美国海军的混乱
- 删除链接?
- 重启
- SID
- 词汇表

## 2. 未解答的问题

本文档的最后一部分解答了所有未解之谜。有些问题与 Windows 对 NTFS 的使用有关,有些则非常技术性。

我们需要您的帮助来填写空白处。

- 为什么 NTFS 3.0+ 上的某些元数据文件仍然包含安全描述符?

在 NTFS 3.0 及更高版本中,\$Volume,\$AttrDef.dot 和 \$Boot 都包含安全描述符。这是为了节省启动时间吗?或许是为了减少需要解析的文件数量?还是与上一个问题相同?

- \$STANDARD\_INFORMATION:最大版本号、版本号和类 ID?

这三个字段中是否有使用?

- \$UsnJrnl 的 \$J 数据流大小是固定的吗?

它的大小是固定的吗?它会像 \$LogFile 那样循环吗?

- \$UsnJrnl 的 \$Max 数据流是做什么的?

其中包含时间戳、两个可能是旗标的字段和一个可能是长度的字段。

- 属性标头

什么情况下“已初始化”的大小与“实际”的大小不同?

- \$MountMgr数据库

这个直播流是什么格式?

- MFT (文件)记录

我们是否只能看到 inode 小于 23 的 MFT 扩展记录?元数据的序列号是否总是等于 inode 号?

- MFT 镜像

如果簇大小大于 4kB,那么这个值有多大?

- 索引记录

它们总是 4kB 吗?

- 整理

排序规则类型 ULONGS 是否等同于 GUID?

- 安全描述符

童年逆境经历是如何遗传的?

待办事项 :将问题复制到相关页面并添加交叉链接。

## 3. 历史

版本 0.6

- 转换为 DocBook 格式：
  - 将文件重新排序为章节和小节。
  - 删除/重新授权了与 html 格式相关的段落。
  - 带标题的章节和表格。
  - 换行符变成了新段落或被删除。
- 演示方式的改变：
  - 将“备注”和“其他信息”移至各部分的底部。
  - 从树的概念图中移除了插图（将来会重新添加）。
  - 删除了空白部分。
  - 数据运行示例现在以表格形式描述。

版本 0.5

- 新的：
  - 添加了 NTFS 常见问题解答的链接。
  - 添加了树的概念。
- 已整理、修改或重写：
  - 修复了第 3 个数据运行示例。
  - 将目录和索引视图标志添加到 FileName 属性。
  - 有关重新分析点的更多信息。
  - 有关 \$usnjrnl 的更多信息。
  - 更新了 \$boot。
  - 已更新 \$mftmirr。

- 更新了安全描述符属性。
- 修复了属性标题概念中的一个小错误。
- 修复了文件记录概念中的一个小错误。
- 修复了集群概念中的类型问题。
- 更新了感谢页面。
- HTML 改进：
  - 在 html 元数据中添加了图标
  - 将帮助菜单移至首页
  - 在页脚添加了旧金山标志和版权信息
  - 将联系邮箱地址改为图片。
  - 从词汇表中删除了指向已过时的 `property_set` 页面的链接。
  - 从页脚中删除了联系信息
  - 修复了指向 SourceForge 的链接（删除了 `www.` 前缀）。
  - CSS 更新。
  - 清理空白字符

#### 版本 0.4

- 新的：
  - 所有数据流和索引的列表
  - 页面:关于、排序和SID
  - 关于XP的一些信息
  - 关于 \$Q、\$O 和 \$R 的信息
  - 关于 MFT 区域的信息
  - 更多关于指数的信息
  - 大量新的词汇表条目
- 已整理、修改或重写：
  - 标准信息、文件名、修复
  - 规范四个时间字段的命名
  - 规范三个文件大小字段的命名
  - 对位图和配额进行了一些小改进

- HTML 改进：
  - 标准化表格
  - 每页均有脚注链接:验证 HTML\CSS 和在线代码
  - “下一页/上一页”链接循环浏览索引
  - 更好的 CSS 合规性
  - 添加关键词以帮助搜索引擎
  - 调整了字体

#### 版本 0.3

- 修改了 Anton 的头文件
- 新的排序页面
- 新增首页标题页
- \$UsnJrnl 的新页面
- 重新设计的索引记录页
- \$ObjId 的新信息
- 配额的新信息
- 关于 \$Secure 的新信息
- 关于 \$Reparse 的新信息
- \$MountMgrDatabase 已添加到点
- 重写了 \$MFT 页面
- 需要进行大量整理工作

#### 版本 0.2

- 将 SourceForge 上的所有内容都置于 CVS 控制之下
- 在每个文件的末尾添加了 \$Id CVS 标签
- 在每个文件的开头添加了完整路径
- 修复了 CSS,现在旧版 Netscape 应该可以正常显示了。
- 更新了 \$AttrDef
- 更新了 \$EA
- 更新了 \$EA\_INFORMATION

- 已更新 \$FILE\_NAME
- 更新了 \$STANDARD\_INFORMATION
- 更新了 \$VOLUME\_INFORMATION
- 为所有词汇表条目编写了词条
- 上一页和下一页的快捷键, 和。
- 修复了大量拼写错误

版本 0.1

- 首次公开发布,基于非常古老的“原始文档”

# 附录一:许可证

## 1. GNU自由文档许可证

### 1.1. 版本 1.1, 2000 年 3 月

版权所有 (C) 2000 自由软件基金会, 地址: 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA

任何人都可以复制和分发本许可文件的完整副本, 但不得对其进行更改。

#### 1.2.0. 前言

本许可旨在使手册、教科书或其他书面文件在意义上实现自由: 确保每个人都能有效地自由复制和分发该文件, 无论是否进行修改, 也无论用于商业用途还是非商业用途。其次, 本许可可为作者和出版商保留了获得署名权的途径, 同时免除其对他人修改内容的责任。

本许可证是一种反版权许可, 这意味着本文档的衍生作品本身也必须是同样意义上的自由软件。它与GNU通用公共许可证 (GNU General Public License) 相辅相成, 后者是一种专为自由软件设计的反版权许可。

我们设计此许可协议的目的是为了将其用于自由软件的手册, 因为自由软件需要自由文档: 自由程序应该附带手册, 并提供与软件本身相同的自由。但此许可协议并不局限于软件手册; 它可以用于任何文本作品, 无论其主题是什么, 也无论是否以印刷书籍的形式出版。我们主要推荐将此许可协议用于以教学或参考为目的的作品。

#### 1.3.1. 适用范围和定义

本许可适用于任何包含版权所有声明的手册或其他作品, 该声明表明该作品可根据本许可的条款进行分发。下文中的“文档”指的即为任何此类手册或作品。任何公众成员均为被许可人, 并被称为“您”。

文件的修改版本是指包含该文件或其一部分的任何作品, 无论是逐字复制, 还是经过修改和/或翻译成另一种语言。

次要章节是指文档中专门命名的附录或前言部分, 其内容仅涉及文档出版商或作者与文档整体主题 (或相关事项) 的关系, 而不包含任何直接属于该主题的内容。(例如, 如果文档部分内容是数学教科书, 则次要章节不得解释任何数学知识。) 这种关系可能涉及与主题或相关事项的历史渊源, 也可能涉及相关的法律、商业、哲学、伦理或政治立场。

不变章节是指某些次要章节, 其标题在声明该文档根据本许可证发布的通知中被指定为不变章节。

封面文字是指在声明该文档根据本许可发布时, 列为封面文字或封底文字的某些简短文字段落。

文档的透明副本是指以公开格式表示的机器可读副本, 其内容可以直接查看和编辑。

可以直接使用通用文本编辑器打开,或者 (对于像素图像)使用通用绘图程序打开,或者 (对于图形)使用一些广泛使用的绘图编辑器打开,并且该文本格式适合输入到文本格式化程序中,或者可以自动转换为各种适合输入到文本格式化程序的格式。如果使用透明文件格式创建的副本,其标记旨在阻止或妨碍读者进行后续修改,则该副本不透明。不透明的副本称为不透明副本。

适用于透明副本的格式示例包括:不含标记的纯 ASCII 格式、Texinfo 输入格式、LaTeX 输入格式、使用公开 DTD 的 SGML 或 XML 格式,以及符合标准且便于人工修改的简单 HTML 格式。不透明格式包括:PostScript、PDF、只能由专有文字处理软件读取和编辑的专有格式、DTD 和/或处理工具不常用的 SGML 或 XML 格式,以及某些文字处理软件仅用于输出目的的机器生成的 HTML 格式。

对于印刷书籍而言,“标题页”指标题页本身,以及为清晰地呈现本许可要求在标题页中出现的内容所需的后续页面。对于没有标题页的作品,“标题页”指作品标题最显著出现位置附近、正文开始之前的文本。

## 1.4.2. 逐字复制

您可以出于商业或非商业目的,以任何媒介复制和分发本文档,但前提是所有副本中均须包含本许可协议、版权声明以及声明本许可协议适用于本文档的许可声明,且您不得在本许可协议的条款之外添加任何其他条件。您不得使用任何技术手段来阻碍或控制他人阅读或进一步复制您制作或分发的副本。但是,您可以接受以副本换取报酬。如果您分发的副本数量足够多,则还必须遵守第 3 节中的条件。

您也可以按照上述条件出借副本,并且您可以公开展示副本。

## 1.5.3. 复印数量

如果您印刷的文档数量超过100份,且该文档的许可声明要求使用封面文字,则您必须将这些文档装入封面,封面必须清晰易读地印有所封面文字:封面文字印在封面上,封底文字印在封底上。两个封面还必须清晰易读地标明您是这些文档的出版商。封面必须完整显示文档标题,且标题中的所有文字都应同样醒目可见。您还可以在封面上添加其他内容。只要封面部分保留了文档标题并满足上述条件,则在其他方面可以视为逐字复制。

如果封面所需的文字太多,无法清晰地排版,则应将列出的前几条文字 (尽可能多地排版)放在封面上,其余文字则继续排版到相邻的页面上。

如果您出版或分发超过100份的文档不透明副本,则必须在每份不透明副本中附上一份机器可读的透明副本,或者在每份不透明副本中或随附一份透明副本,并注明一个可供公众访问的计算机网络位置,该位置包含一份完整的文档透明副本,不含任何附加材料,公众可以使用公共网络协议匿名免费下载该副本。如果您选择后一种方式,则在开始大量分发不透明副本时,您必须采取合理谨慎的措施,确保该透明副本在所述位置至少持续可访问,直至您最后一次 (直接或通过您的代理商或零售商)向公众分发该版本的不透明副本后至少一年。

建议您 (但不强制要求)在大量分发文档之前与文档作者联系,以便他们有机会为您提供文档的更新版本。

## 1.6.4.修改

您可以按照第 2 条的规定复制和分发本文件的修改版本。

以及上文第3点,前提是应以本许可协议发布修改版本,且修改版本充当文档的角色,从而授权任何拥有该修改版本副本的人分发和修改该版本。此外,您必须在修改版本中执行这些操作。

版本:

1. 在标题页 (以及封面,如有)上使用与文档标题不同的标题,并且与正文标题也不同。

先前版本的那些 (如果有的话,应该列在“历史记录”部分中) 文档)。如果原版本出版商允许,您可以使用与先前版本相同的标题。

2. 在标题页上列出对本书创作负有责任的一个或多个个人或实体作为作者。

修改版本中的修改,以及至少五位主要作者的修改文件 (如果作者少于五位,则需提供所有主要作者)。

3. 在标题页上注明修改版出版商的名称,作为出版商。

4. 保留文档的所有版权声明。

5. 在其他版权声明旁边,为您的修改添加适当的版权声明。

6. 在版权声明之后,紧接着添加许可声明,允许公众使用。  
根据本许可的条款,按照以下附录中所示的形式使用修改版本。

7. 在该许可声明中保留所有不变章节和所需封面文本的完整列表。  
文件的许可声明。

8. 附上本许可证的未修改副本。

9. 保留题为“历史”的部分及其标题,并在其中添加一条记录,至少应包含标题、年份等信息。

新作者,以及标题页上注明的修订版出版商。如果没有相关章节。

标题为“文档中的历史”,请创建一个包含标题、年份、作者和出版商的条目。

根据标题页上的信息,添加一个描述修改版本的条目,如上所示。

上一句。

10. 保留文档中提供的网络位置 (如有),以便公众通过透明方式访问。

文档副本,以及文档中列出的先前版本所依据的网络位置。这些信息可以放在“历史记录”部分。如果作品的出版时间比文档本身早至少四年,或者如果原始版本已发布,则可以省略其网络位置。

它所指版本的出版商已授予许可。

11. 在任何标题为“致谢”或“献词”的章节中,保留该章节的标题,并保留  
本节包含了所有贡献者致谢和/或献词的实质内容和语气。

12. 保留文档所有不变章节,其正文和标题均不得更改。章节编号或等效标识不视为章节标题的一部分。

13. 删除标题为“背书”的任何部分。此类部分不得包含在修改后的版本中。  
sion。

14. 不得将任何现有章节重新命名为“背书”,或使其标题与任何不变章节相冲突。

如果修改后的版本包含符合次要章节定义的新前言部分或附录,且不包含从原文档复制的任何内容,您可以选择指定部分或全部内容。

将这些章节标记为不变章节。为此,请将它们的标题添加到修改后的不变章节列表中。

版本许可声明。这些标题必须与其他任何章节标题区分开来。

您可以添加一个名为“认可”的部分,但其中只能包含各方对您的修改版本的认可。例如,同行评审声明,或者该文本已被某个组织批准为标准的权威定义。

您可以在修改版本封面文字列表的末尾添加一段不超过五个字的文字作为封面文字,以及一段不超过二十五个字的文字作为封底文字。任何实体(或通过其安排)只能添加一段封面文字和一段封底文字。如果文档中已包含同一封面的文字,且该文字此前已由您或您所代表的同一实体添加,则您不得再添加新的封面文字;但经先前添加该文字的出版商明确许可,您可以替换旧文字。

本文档的作者和出版商并未通过本许可授予任何人使用其姓名进行宣传或声明或暗示认可任何修改版本的许可。

## 1.7.5. 合并文件

您可以按照上文第 4 节中针对修改版本定义的条款,将本文档与根据本许可发布的其他文档合并,前提是必须将所有原始文档的所有不变部分包含在合并中,并且将它们全部列为合并作品的不变部分,并在其许可声明中列出。

合并后的作品只需包含一份本许可协议,多个相同的“不变章节”可以替换为一份。如果存在多个名称相同但内容不同的“不变章节”,则应在每个章节的标题末尾用括号添加该章节原作者或出版商的姓名(如果已知),否则添加一个唯一的编号,以确保每个章节标题的唯一性。

对合并作品的许可声明中不变章节列表中的章节标题进行同样的调整。

合并时,您必须将各原始文档中标题为“历史”的所有章节合并为一个标题为“历史”的章节;同样,将所有标题为“致谢”的章节和标题为“献词”的章节合并。您必须删除所有标题为“背书”的章节。

## 1.8.6. 文件集

您可以创建一个包含本文档和根据本许可发布的其他文档的集合,并将各个文档中的本许可的单独副本替换为包含在集合中的单个副本,前提是您所有其他方面都遵守本许可中关于逐字复制每个文档的规则。

您可以从这样的集合中提取单个文档,并根据本许可单独分发,前提是您将本许可的副本插入到提取的文档中,并在所有其他方面遵守本许可,包括对该文档的逐字复制。

## 1.9.7. 与独立作品的聚合

将本文档或其衍生作品与其他独立文档或作品汇编在一起,无论该汇编是存储在存储介质还是分发介质上,只要未对该汇编主张汇编版权,则该汇编整体不构成本文档的修改版本。此类汇编称为聚合体,本许可不适用于与本文档一同汇编的其他独立作品,除非它们本身并非本文档的衍生作品。

如果第 3 节的封面文本要求适用于这些文件副本,那么如果  
如果文件篇幅不足总篇幅的四分之一,则该文件的封面文字可以放在……

仅将文档包含在聚合体中的封面。否则,它们必须出现在封面上。  
围绕整个集合体。

### 1.10.8. 翻译

翻译被视为一种修改,因此您可以根据第 4 条的规定分发文档的译文。用译文替换不变章节需要获得特别许可。

您可以从版权所有处获得这些内容,但您可以除了保留这些不变章节的原始版本外,还包含部分或全部不变章节的译文。您也可以包含本许可协议的译文。  
前提是您也必须附上本许可协议的英文原版。如有任何分歧,  
本许可协议的翻译版本与英文原版之间,以英文原版为准。  
取得胜利。

### 1.11.9. 终止

除本许可明确规定外,您不得复制、修改、再许可或分发本文档。任何其他复制、修改、再许可或分发本文档的行为均属无效。

这将自动终止您在本许可项下的权利。但是,已收到副本的各方,  
只要相关方继续完全遵守本许可,您授予的任何权利或许可都不会被终止。

## 1.12. 10. 本许可的未来修订

自由软件基金会可能会不时发布新的、修订版的 GNU 自由文档许可证。这些新版本在精神上与当前版本相似,但可能存在差异。

详细说明以解决新问题或疑虑。请参阅 <http://www.gnu.org/copyleft/>。

许可证的每个版本都有一个唯一的版本号。如果文档中明确规定……

如果本许可证的特定版本号或任何后续版本适用于本文件,您可以选择遵循该指定版本或自由软件基金会已发布 (非草案)的任何后续版本的条款和条件。如果本文件未指定本许可证的版本号,您可以选择自由软件基金会已发布 (非草案)的任何版本。

基础。

# 词汇表

这是所有术语的词汇表。

有些条目会引用其他条目,例如“另请参阅”。

有些条目单独成页,例如“更多...”

# 词汇表

(参见 Dot,根目录)

参见 Dot,根目录。

门禁控制入口 (ACE)

门禁控制入口是最小的安全单元。它包含  
SID (用户或组)和权限信息。

权限将为“允许访问”、“拒绝访问”或“不允许访问”之一。  
系统审计。此对象包含用于确定权限继承方式的标志。

另请参阅安全标识符 (SID)、访问控制列表 (ACL)  
审计,审计。

访问控制列表 (ACL)

此安全结构包含一个ACE列表。  
另请参阅 \$SECURITY\_DESCRIPTOR、安全标识符 (SID)  
访问控制列表 (ACL)、审计、审核。

ACE (参见访问控制)  
入口)  
ACL (参见访问控制列表)

请参阅访问控制条目 (ACE)。

请参阅访问控制列表 (ACL)。

属性定义

此元数据文件包含所有属性的定义,这些属性如下:  
允许在 NTFS 卷上运行。

(更多的...)

属性

在磁盘上,文件以一组属性的形式存储:驻留属性/非驻留属性。

属性列表

当文件的属性无法放入单个元素中时,可以使用此属性。  
MFT 文件记录。它列出了所有属性及其位置。  
可以找到。

\$ATTRIBUTE\_LIST 始终存储在基本文件记录中。

(更多的...)  
另请参阅 FILE 记录、\$MFT、基本 FILE 记录。

审计,审计

作为文件安全权限的一部分,执行的任何操作  
可以记录在文件中。

例如,可能需要一个文件来记录所有尝试过的人员。  
想看,但没有权限。

B+树

B+树是二叉树的一种变体。

每个节点不是只有一个数据元素,而是有很多数据元素。

在 NTFS 文件系统中,实际数量取决于名称的长度和大小。

| 词汇表       |                                                                                                                                              |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------|
|           | <p>(簇的大小)。</p> <p>B+树保留了二叉树的效率,并且性能也相当出色。<br/>处理大量数据元素效果很好 (因为树状结构往往……)<br/>横向生长而非纵向生长)。<br/>另请参阅二叉树、平衡树。</p>                                |
| BAAD      | <p>在 chkdsk 期间,如果 NTFS 发现多扇区项 (MFT.INDEX) 块等)其中多扇区标头与值不匹配<br/>在该区域的末尾,它会用魔法数字标记物品。<br/>BAAD ,并用零填充 (每个字母末尾留一个短整型)。<br/>行业……)</p>             |
|           | <p>待修复<br/>“BAAD” == 损坏的记录<br/>“CHKD” == chkdsk ???<br/>“文件” == mft 条目<br/>“HOLE” == ??? (NTFS 3.0+?)<br/>“INDX” == 索引缓冲区<br/>RSTR 和 ???</p> |
|           | <p>另请参阅 chkdsk、fsck。</p>                                                                                                                     |
| 坏         | <p>这是代表不良集群的命名数据流<br/>体积。<br/>另请参阅 \$BadClus。</p>                                                                                            |
| \$BadClus | <p>此元数据文件列出了卷上所有不可读的簇。</p>                                                                                                                   |
|           | <p>(更多的…)</p>                                                                                                                                |
| 平衡树       | <p>二叉树经常会变得非常不均匀,通过重新组织……<br/>通过数据,可以平衡树,使得没有一个节点的左右两侧子节点的数量相似。<br/><br/>另请参阅 B+ 树、二叉树。</p>                                                  |
| 基本文件记录    | <p>如果属性无法放入单个 MFT 记录中,则使用基本记录。<br/>文件记录包含足够的信息来定位其他记录。<br/>另请参阅 \$ATTRIBUTE_LIST、FILE 记录、\$MFT。</p>                                          |
| 二进制       | <p>以二进制进行的数学运算。在本文中,某些标志<br/>为了清晰起见,字段以二进制形式表示。<br/><br/>例如 00001000<sub>2</sub>, 010000000<sub>2</sub>。<br/>另请参阅十进制、十六进制、十六进制、单位。</p>       |
| 二叉树       | <p>这是按顺序存储已排序数据的一种高效方法。<br/><br/>树中的每个节点代表一个数据元素。<br/><br/>左子节点是它前面所有元素的集合。<br/><br/>右子节点是所有传入元素的集合。<br/>之后。<br/>另请参阅 B+ 树、平衡树。</p>          |

| 词汇表       |                                                                                                                                                                         |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 少量        | 一个二进制数字,1 或 0。<br>另请参阅“单位”。                                                                                                                                             |
| \$Bitmap  | 此元数据文件用于跟踪哪些集群正在使用。<br>体积。<br><br>(更多的...)                                                                                                                              |
| 位图        | 此属性用于跟踪索引中正在使用的记录。<br><br>(更多的...)                                                                                                                                      |
| 堵塞        | 在Linux术语中,这被称为集群。块设备在Linux术语中被称为存储单元。<br><br>集群是最小的分配单元。<br><br>簇的大小是扇区大小的 2 的幂 (称为簇大小)。<br>(因子) ,其大小介于 512 字节到 4 KB 之间。<br>(有时是 64 KB,但 4 KB 是当前 NTFS 压缩引擎可以处理的最大簇大小。) |
|           | 该限制可能与英特尔处理器上使用的 4 KB 页面大小有关。<br>i386 CPU) 。                                                                                                                            |
|           | 此大小可通过 Windows NT 格式化实用程序进行设置,其默认值为:卷大小,簇大小 1 至 512 MB,扇区大小 512 MB<br>1GB 以内 1KB;1GB 到 2GB 2KB;超过 2GB 4KB                                                               |
| \$Boot    | 此元数据文件指向卷的引导扇区。<br><br>它包含有关体积大小、簇和……的信息<br>MFT。<br><br>(更多的...)                                                                                                        |
| 字节 (参见单位) | 参见单位。                                                                                                                                                                   |
| 检查磁盘      | 这是一个用于检查和修复文件系统的 DOS 和 Windows 实用程序。<br><br>它的名称是检查盘的缩写。<br>另请参阅 fsck。                                                                                                  |
| 簇         | 这是 NTFS 使用的最小磁盘单元,它是一个多维磁盘单元。<br>行业规模。<br><br>当卷被格式化且无法更改时,就会确定卷的格式。<br>然后。<br>另请参阅 Sector,\$Boot.Volume。                                                               |
| 压缩        | NTFS 支持文件级和目录级压缩。<br><br>压缩是在读取文件时透明地执行的。<br>书面。<br><br>压缩目录中的任何新文件都会自动压缩。<br><br>另请参阅压缩单元。                                                                             |

|                                 |                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 压缩单元                            | <p>每个标记为压缩的文件都被分成十六个簇。<br/>压缩块,也称为压缩单元。</p> <p>如果其中一个数据块无法压缩成十五个簇,或者<br/>压缩程度越低,压缩效果就越差。</p> <p>这种划分还有助于随机访问文件,即无需解压缩整个文件。</p> <p>另请参阅集群、压缩。</p>                            |
| \$DATA                          | <p>此属性包含文件的实际数据。</p> <p>这条溪流可能也有名字。</p> <p>(更多的...)</p>                                                                                                                     |
| 数据运行                            | <p>非驻留属性存储在称为运行的集群区间中。</p> <p>每次运行都由其起始簇及其长度表示。</p> <p>运行将文件的 VCN 映射到卷的 LCN。</p> <p>(更多的...)</p> <p>另请参阅属性、集群、逻辑集群编号 (LCN)、虚拟<br/>簇编号 (VCN) ,卷。</p>                         |
| 小数                              | <p>以十进制进行的数学运算。</p> <p>本文档中,既非十六进制也非二进制的数字,<br/>以十进制表示,例如 16 (十六) ,23 (二十三) 。<br/>另请参阅二进制、十六进制、十六进制、单位。</p>                                                                 |
| 目录                              | <p>NTFS 目录是一种索引属性。NTFS 使用索引属性。<br/>整理文件名。</p> <p>目录条目包含文件名和副本<br/>文件的标准信息属性 (时间戳信息) 。</p> <p>这种方法可以提升目录浏览的性能。<br/>因为 NTFS 不需要读取文件的 MFT 记录即可打印<br/>目录信息。</p> <p>(更多的...)</p> |
| DOS 文件权限 (参见文件)<br>权限)<br>点,根目录 | <p>请查看文件权限。</p> <p>磁盘的根目录</p> <p>(更多的...)</p>                                                                                                                               |
| 驱动器 (参见音量)                      | <p>参见卷号。</p>                                                                                                                                                                |
| 动态磁盘                            | <p>动态磁盘 SDS,win2k</p>                                                                                                                                                       |
| 每股                              | <p>此属性用于在 NTFS 下实现 HPFS 扩展属性。</p>                                                                                                                                           |

|                  |                                                                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | <p>它仅用于与 OS/2 兼容。</p> <p>(更多的...)</p>                                                                                                                     |
| \$EA_INFORMATION | <p>此属性用于在 NTFS 下实现 HPFS 扩展属性。</p>                                                                                                                         |
|                  | <p>它仅用于与 OS/2 兼容。</p> <p>(更多的...)</p>                                                                                                                     |
| \$EFS            | <p>\$EFS 是任何加密文件的命名 \$LOGGED_UTILITY_STREAM。</p> <p>另请参阅 \$LOGGED_UTILITY_STREAM。</p>                                                                     |
| \$Extend         | <p>此元数据目录包含元数据文件：</p> <ul style="list-style-type: none"> <li>· \$ObjId</li> <li>· 配额</li> <li>· \$Reparse</li> </ul> <p>(更多的...)</p>                      |
| 文件               | <p>在 NTFS 术语中,文件可以是普通文件,也可以是目录 (例如在 Linux 系统中)或系统文件。</p> <p>(更多的...)</p>                                                                                  |
| \$FILE_NAME      | <p>此属性表示文件名。</p> <p>一个文件可以有一个或多个名称,并且可以位于任何目录中。</p> <p>这相当于 NTFS 系统中的 Unix 硬链接。</p> <p>(更多的...)</p>                                                       |
| 文件名命名空间          | <p>并非所有字符都适用于 DOS 文件名。</p> <p>为了兼容性,NTFS 存储名称所属的命名空间。</p> <p>(更多的...)</p>                                                                                 |
| 文件权限             | <p>NTFS 支持标准的 DOS 文件权限集,即存档、系统、隐藏和只读。</p> <p>此外,NTFS 还支持压缩和加密。</p> <p>另请参阅 \$SECURITY_DESCRIPTOR,压缩。</p>                                                  |
| 文件记录             | <p>\$MFT 由 FILE 记录组成,之所以这样命名,是因为其包含数量惊人的 FILE 记录。</p> <p>每条记录都有一个标准标题和一个属性列表。</p> <p>如果属性数量不足以放入单个记录中,则会创建更多记录。</p> <p>将要使用,并且需要 \$ATTRIBUTE_LIST 属性。</p> |

|               |                                                                                                                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | 另请参阅属性、\$ATTRIBUTE_LIST、魔数、\$MFT。                                                                                                                                                                           |
| 文件记录段 (FRS)   | FRS = MFT 文件记录                                                                                                                                                                                              |
| 文件参考          | <p>每个文件记录都有一个唯一的标识编号。</p> <p>前 48 位是按顺序分配的数字,即 \$MFT 中的偏移量。</p> <p>最后 16 位是序列号。</p> <p>每次记录被修改时,这个数字都会递增。</p> <p>序列号可以帮助检测卷上的错误。<br/>另请参阅 FILE 记录、\$MFT、卷。</p>                                              |
| 文件运行 (参见数据运行) | 参见数据运行。                                                                                                                                                                                                     |
| 文件大小          | <p>NTFS 记录三种文件大小。</p> <p>它们各自存储字节数。</p> <ul style="list-style-type: none"> <li>· R) 实数。数据字节数。</li> <li>· A) 已分配空间。磁盘上占用的空间大小。</li> <li>· I) 已初始化。压缩文件大小。</li> </ul> <p>如果文件已压缩,则初始大小可能小于实际大小。<br/>实际尺寸。</p> |
| 文件系统          | <p>操作系统用于在存储单元上存储和组织文件的物理结构。</p> <p>常用的文件系统是 FAT (DOS 使用)。</p>                                                                                                                                              |
| 修复 (参见更新顺序)   | 参见更新顺序。                                                                                                                                                                                                     |
| 分支 (参见资源分支)   | 参见资源分支。                                                                                                                                                                                                     |
| 碎片化的          | (一个)文件                                                                                                                                                                                                      |
| FRS (参见文件记录段) | 请参阅文件记录段 (FRS)。                                                                                                                                                                                             |
| fsck          | <p>这是一个用于检查和修复文件系统的实用程序。</p> <p>它的名称是文件系统检查的缩写。</p>                                                                                                                                                         |
| GB (参见单位)     | 参见单位。                                                                                                                                                                                                       |
| GUID (参见单位)   | <p>GUID 的有效格式为 {XXXXXXXX-XXXX-XXXX-X}</p> <p>全球唯一标识符 (GUID)</p>                                                                                                                                             |

| 词汇表                                                |                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                    |                                                                                                                                                                                                                                                                                 |
|                                                    | GUID 结构存储全局唯一标识符 (GUID) ,它是一个 128 位值,由一组 8 个十六进制数字和三组各 4 个十六进制数字组成,最后是 12 个十六进制数字。GUID 是微软重要的分布式计算环境 (DCE) 的通用标识符。GUID 示例:                                                                                                                                                      |
|                                                    | 1F010768-5A73-BC91-0010A52216A7                                                                                                                                                                                                                                                 |
|                                                    | 订单存储在磁盘上吗?                                                                                                                                                                                                                                                                      |
|                                                    | 01020304-0506-0708-090A0B0C0D0E0F010                                                                                                                                                                                                                                            |
|                                                    | 0x00 04030201<br>0x04 0605<br>0x06 0807<br>0x08 090A0B0C0D0E0F010                                                                                                                                                                                                               |
| 十六进制                                               | 以十六进制进行的数学运算。<br><br>本文档中,许多数字以十六进制表示,例如 0x02E0,0xF100。<br>另请参阅二进制、十进制、单位。                                                                                                                                                                                                      |
| HFS (参见分层文件) 系统)<br>分层文件系统 (HFS)<br>高性能文件系统 (HPFS) | 参见分层文件系统 (HFS) 。<br><br>MacOS文件系统。<br><br>OS/2 文件系统。<br><br>请记住:曾几何时,OS/2 就是操作系统。<br>由 IBM 和微软共同开发。<br><br>两大巨头之间出现了裂痕。IBM继续发展 OS/2 (后来变成了 OS/2 Warp) ,这就解释了为什么 OS/2 知道如何执行 Windows 应用程序。微软决定这样做。<br>它自己的操作系统:Windows NT。<br><br>HPFS 的设计影响了 NTFS 的设计,因此这两个文件系统共享一些特性。<br>功能众多。 |
| HPFS (参见高性能文件系统) 文件系统)<br>130美元                    | 参见高性能文件系统 (HPFS) 。<br><br>这是目录使用的命名索引。<br><br>该名称指的是属性 0x30 (\$FILE_NAME)。<br>另请参阅属性、目录、\$FILE_NAME、索引。                                                                                                                                                                         |
| 指数                                                 | (仅仅是整个索引的概念)                                                                                                                                                                                                                                                                    |
| 索引分配                                               | 此属性包含构成索引的条目的位置。<br><br>(更多的...)                                                                                                                                                                                                                                                |
| \$INDEX_ROOT                                       | 此属性是索引的根。<br><br>索引以平衡二叉树的形式存储。                                                                                                                                                                                                                                                 |

| 词汇表              |                                                                                                                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  |                                                                                                                                                                                                                     |
|                  | 唯一建立索引的属性是 \$FILE_NAME 及其索引称为 \$I30。                                                                                                                                                                                |
|                  | (更多的...)                                                                                                                                                                                                            |
| INDX记录           | 索引记录被目录、\$Quota、\$Reparse 和安全。                                                                                                                                                                                      |
|                  | 索引内容取决于所保存的索引类型。                                                                                                                                                                                                    |
|                  | 目录存储 \$FILE_NAME 属性。                                                                                                                                                                                                |
|                  | (更多的...)<br>另请参阅 Directory、\$I30、\$Quota、\$Reparse、\$Secure。                                                                                                                                                        |
| 无限日志区域           | \$LogFile 中包含一些内容。它由一段 4KB 的序列组成。日志记录。<br>另请参阅 \$LogFile。                                                                                                                                                           |
| 索引节点             | inode 是文件系统对文件、目录、设备的表示形式。ETC。                                                                                                                                                                                      |
|                  | 在 NTFS 文件系统中,每个 inode 都由一个 MFT 文件记录表示。<br>另请参阅目录、文件、文件记录、文件系统。                                                                                                                                                      |
| \$J              | \$J 是元数据文件 \$UsnJrnl 的命名数据流。<br>另请参阅 \$UsnJrnl。                                                                                                                                                                     |
| 交汇点              | 微软术语,指挂载点,在 NT 5.0 中可用。                                                                                                                                                                                             |
| KB (参见单位)        | 参见单位。                                                                                                                                                                                                               |
| LCN (参见逻辑集群) 数字) | 请参阅逻辑簇号 (LCN)。                                                                                                                                                                                                      |
| 日志记录             | 无限日志区域中的一块 4KB 数据。一切始于魔法。编号为 “RCRD”并进行了修复,然后具有未记录的可变长度数据。[日志记录可能还会进一步细分 我无法想象]<br>如果只需要记录几个字节,它们就会浪费 4KB 的空间。Custer 提到了高级和低级 “记录”。高级记录包括: - 在 ode n 中分配内存, - 在目录 m 中创建目录项 foo。低级记录包括: -<br>使用 <1KB> 的新内容修改 inode n] |
| 日志文件             | 此元数据文件用于保证系统故障时的数据完整性。                                                                                                                                                                                              |
|                  | 它有两个重启区域和一个无限日志区域的副本。                                                                                                                                                                                               |
|                  | 日志文件位于卷的中间附近,就在第二个文件之后。<br>启动文件的集群。[最好用 “运行”而不是 “集群”。启动文件通常延伸到磁盘起始位置的几个簇,然后只运行一次集群 (启动副本)。扇形)。另外,它不是 “无限”吗?                                                                                                         |
|                  | 事务日志文件                                                                                                                                                                                                              |
|                  | (更多的...)                                                                                                                                                                                                            |

| 词汇表                                        |                                                                                                                                                   |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 是                                          | <p>此属性由加密文件使用。</p> <p>(更多的...)</p>                                                                                                                |
| 逻辑簇号 (LCN)                                 | <p>一个卷被分成若干个簇,并按顺序编号。从<br/>零开始。<br/>另请参阅集群、卷。</p>                                                                                                 |
| 逻辑序列号 (LSN)<br>LSN (参见逻辑序列)<br>数字)<br>神奇数字 | <p>用于识别 NTFS 日志记录的序列号。</p> <p>参见逻辑序列号 (LSN) 。</p> <p>NTFS 文件系统的大多数磁盘结构都有一个唯一的常量。识别它们。</p> <p>这个数字通常位于结构的开头,可以用作一种健全性检验。</p>                       |
| 主文件表                                       | <p>请参阅主文件表。</p>                                                                                                                                   |
| 最大                                         | <p>\$Max 是 \$UsnJrnl 的一个命名数据流。<br/>另请参阅 \$UsnJrnl。</p>                                                                                            |
| MB (参见单位)                                  | <p>参见单位。</p>                                                                                                                                      |
| 元数据                                        | <p>仅文件系统使用的存储单元上的数据,作为帧访问用户数据。</p> <p>元数据构成了文件系统的结构) 。</p> <p>各种文件系统的元数据示例包括 FAT 表和 inode。表、空闲块列表、空闲块位图、日志区域和超级块。</p>                             |
| 元数据                                        | <p>关于数据的数据。在数据处理中,元数据是指提供有关应用程序或环境的信息或文档的数据。</p> <p>例如,元数据会记录有关 d 属性 (名称、大小、数据类型等)和数据结构 (长度、字段、列等)以及 (其位置、关联方式、所有者)的数据,数据可能包括有关数据状态或特征的描述性信息。</p> |
| \$MFT                                      | <p>该元数据文件 (即主文件表)是所有文件的索引。音量。</p> <p>它包含每个文件的属性以及所有索引的根目录。</p> <p>(更多的...)</p>                                                                    |
| \$MFTMirr                                  | <p>此元数据文件存储了 \$MFT 的前四个记录的副本。</p> <p>这是一项安全措施,可能仅在 chkdsk 运行时才会使用。<br/>跑步。</p>                                                                    |

| 词汇表           |                                                                                                                          |
|---------------|--------------------------------------------------------------------------------------------------------------------------|
|               | (更多的...)                                                                                                                 |
| \$MountMgr数据库 | \$MountMgrDatabase 是 dot (根目录)的命名数据流。                                                                                    |
|               | 它包含已挂载卷的列表。<br>另请参阅 Dot、根目录。                                                                                             |
| MST (参见多部门转移) | 参见多部门转移支付。                                                                                                               |
| 多部门转移支付       | 多部门、维修、安全检查                                                                                                              |
| 蚕食            | 半个字节 (4 位) 。                                                                                                             |
| 北领地管理局        | NT 机构定义了安全标识符的范围。<br><br>数字0-4代表内部标识符，<br><br>例如:世界、本地、5 代表 NT 管理局。<br>另请参阅 NT 子机构、安全标识符 (SID)<br>\$SECURITY_DESCRIPTOR。 |
| NTFS          | NTFS 是 Windows NT、Windows 2000 和 Windows XP 的文件系统。                                                                       |
|               | (更多的...)<br>另请参阅文件系统。                                                                                                    |
| 北领地分局         | 子权限可以包含任意数量的字段 (通常为五个) 。                                                                                                 |
|               | 以 21 (0x15) 开头的子授权表示 NT 域<br>标识符。<br>另请参阅 NT Authority、安全标识符 (SID) ，<br>\$SECURITY_DESCRIPTOR。                           |
| \$O           | 这是属于 \$Quota 和 \$ObjId 的命名索引之一。<br>另请参阅索引、\$Q、\$ObjId、\$Quota。                                                           |
| \$OBJECT_ID   | 此属性存储 SID 和安全哈希之间的映射关系。                                                                                                  |
|               | (更多的...)                                                                                                                 |
| \$ObjId       | 此属性记录在使用分布式链接跟踪时分配给文件和目录的唯一标识符。                                                                                          |
|               | (更多的...)                                                                                                                 |
| 帕姆            | 可插拔身份验证模块 (PAM) 是一组用于……的库。<br>验证Linux系统的安全性。                                                                             |
| 分区 (参见卷)      | 参见卷号。                                                                                                                    |
| 分区表           | 分区表...                                                                                                                   |

|                              |                                                                                                                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              | SFS Win2K 动态磁盘                                                                                                                                                                                                                         |
| 权限                           | <p>NTFS 中有两种存储权限的机制。</p> <p>其中一个是 DOS 文件权限的超集,其中包括读取权限。仅限且隐藏。</p> <p>另一种方法基于 ACE,允许向特定用户授予特定权限。</p> <p>另请参阅访问控制条目 (ACE)、文件权限、<code>\$SECURITY_DESCRIPTOR</code>。</p>                                                                   |
| POSIX                        | <p>Portable Operating Systems Interface (便携式操作系统接口)的缩写 (发音类似 positive),由 Richard M. Stallman 提出。</p> <p>它是一套国际标准 (ISO/IEC 9945-1:1996(E)、ANSI/IEEE Std 1003.1 1996 版),用于与类 Unix 漏洞利用系统 (例如 Linux)进行接口。</p> <p>NTFS 不支持类 Unix 设备文件。</p> |
| <code>\$PROPERTY_SET</code>  | NT4 中已过时的属性 (0xF0)                                                                                                                                                                                                                     |
| <code>\$Q</code>             | <p>这是属于 <code>\$Quota</code> 的命名索引之一。</p> <p>另请参阅索引、<code>\$O</code>、<code>\$配额</code>。</p>                                                                                                                                            |
| 配额                           | <p>此元数据文件存储有关文件配额的信息。</p> <p>(更多的...)</p>                                                                                                                                                                                              |
| <code>\$R</code>             | <p>这是属于 <code>\$Reparse</code> 的命名索引。</p> <p>另请参阅索引、<code>\$Reparse</code>。</p>                                                                                                                                                        |
| RCRD记录                       | <p>此记录用于 <code>\$LogFile</code> 中。</p> <p>每一项都代表一个需要执行的原子交易。</p> <p>另请参阅 <code>\$LogFile</code>、<code>Transaction</code>。</p>                                                                                                          |
| 记录                           | <p>NTFS中有几种记录类型。</p> <p>FILE 记录用于 <code>\$MFT</code>,<code>INDX</code> 记录用于索引。<code>\$LogFile</code> 中的 RCRD 和 RSTR 记录。</p> <p>另请参阅 FILE 记录、<code>INDX</code> 记录、RCRD 记录、RSTR 记录。</p>                                                  |
| 递归                           | 参见递归。                                                                                                                                                                                                                                  |
| 参考                           | 文件 (还有其他的吗?)                                                                                                                                                                                                                           |
| <code>\$Reparse</code>       | <p>此元数据文件存储有关重解析点的信息。</p> <p>(更多的...)</p>                                                                                                                                                                                              |
| <code>\$REPARSE_POINT</code> | <p>此属性存储有关重解析点的信息。</p> <p>(更多的...)</p>                                                                                                                                                                                                 |

| 词汇表                                        |                                                                                                                                                                         |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 资源分叉                                       | <p>在 macOS 的文件系统 HFS 中,文件可以拥有多个数据流。</p> <p>这些被称为资源分支。</p> <p>另请参阅分层文件系统 (HFS)、流。</p>                                                                                    |
| 回滚                                         | <p>挂载 NTFS 卷时,会检查其是否处于一致状态。</p> <p>如果不是,则会查询 \$LogFile 并撤销事务,直到磁盘恢复到一致状态。</p> <p>这并不能保证数据完整性,只能保证磁盘完整性。</p> <p>另请参阅 \$LogFile、Transaction、Volume。</p>                   |
| 根目录 (参见点、根目录)<br>目录)<br>RSTR记录             | <p>参见 Dot,根目录。</p> <p>\$LogFile 中有两份此文件的副本。</p> <p>重启区域包含魔数“RSTR”,后面跟着修复程序和其他一些数据,包括三个 LSN。</p> <p>重启区域包含指向日志区域的指针,例如写入的第一条和最后一条日志记录以及写入的最后一条检查点记录。(一共三条 现在哪条对应哪条?)</p> |
| 运行 (参见数据运行)                                | <p>参见数据运行。</p>                                                                                                                                                          |
| \$SDH                                      | <p>这是属于 \$Secure 的命名索引之一。</p> <p>另请参阅 Index、\$SII、\$Secure。</p>                                                                                                         |
| \$SDS                                      | <p>这是属于 \$Secure 的命名数据流。</p> <p>另请参阅 \$Secure、Stream。</p>                                                                                                               |
| 部门                                         | <p>物理存储单元上的数据单元。</p> <p>存储控制器只能以该单位的倍数访问数据。</p> <p>一个扇区通常是 512 字节,但在某些亚洲硬盘上可能是 1 KB。</p>                                                                                |
| 安全                                         | <p>此元数据文件存储卷使用的安全描述符表。</p> <p>(更多的...)</p>                                                                                                                              |
| 安全                                         | <p>NTFS 文件系统具有两级安全保护。</p> <p>DOS 文件权限包括只读和隐藏等,以及 ACL 模型,该模型可以向特定用户授予特定权限。</p> <p>用户。</p> <p>另请参阅访问控制条目 (ACE)、访问控制列表 (ACL)、权限、\$SECURITY_DESCRIPTOR、安全标识符 (SID)。</p>     |
| \$SECURITY_DESCRIPTOR 此属性存储有关文件或目录的所有安全信息。 | <p>教区。</p> <p>它包含一个用于审计的访问控制列表 (ACL)、一个用于权限的访问控制列表 (ACL) 和一个安全标识符 (SID)。</p>                                                                                            |

| 词汇表                    |                                                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        | 显示所有者的用户和组。                                                                                                                                             |
|                        | (更多的...)<br>另请参阅属性、访问控制列表 (ACL)、访问控制入口 (ACE)、安全标识符 (SID)。                                                                                               |
| 安全标识符 (SID)            | 这个可变长度标识符用于唯一标识用户或组。<br>在NT域中,它用于安全权限。<br>另请参阅访问控制条目 (ACE)、访问控制列表 (ACL)。<br>\$SECURITY_DESCRIPTOR。                                                      |
| 序列数组 (参见更新顺序)          | 参见更新顺序。                                                                                                                                                 |
| SID (参见安全标识符)          | 参见更新顺序。                                                                                                                                                 |
| 捐赠                     | 这是属于 \$Secure 的命名索引之一。<br>另请参阅 Index、\$SDH、\$Secure。                                                                                                    |
| 稀疏文件                   | NTFS 支持稀疏文件。如果一个文件包含大的、连续的、如果是零块,NTFS 可以选择不浪费任何空间在磁盘上存储这些部分。<br><br>它们被表示为不包含任何内容的数据序列。<br><br>从磁盘读取数据时,NTFS 会直接用零替换。<br>另请参阅数据运行。                      |
| \$STANDARD_INFORMATION | 此属性包含有关文件的信息,例如文件权限和创建时间。                                                                                                                               |
| 在                      | (更多的...)                                                                                                                                                |
| 溪流                     | NTFS 上的所有数据都存储在数据流中,数据流可以有名称。<br><br>一个文件可以有多个数据流,但必须只有一个数据流。<br>没有名字。<br><br>文件的大小就是其未命名数据属性的大小。                                                       |
| \$SYMBOLIC_LINK        | 该属性与 \$VOLUME_VERSION 一样,在 NTFS v1.2 中就已存在。但并未被使用。<br><br>它在 NTFS v3.0+ 中已不再存在。                                                                         |
| TB (参见单位)              | 参见单位。                                                                                                                                                   |
| 时间戳                    | NTFS 存储文件和目录时会用到四个重要信息。<br><br>它们分别是:文件创建时间;最后修改时间;MFT 记录的最后修改时间;最后访问时间。<br><br>NTFS 存储日期为自 1601 年 1 月 1 日以来的 100ns 单位数。<br><br>Unix系统将日期存储为自1月1日以来的秒数。 |
|                        | 将 4 个时间字段标准化,名称和描述概念分别指代以下 4 个时间:<br><br>C 创建                                                                                                           |

词汇表

|                      |                                                                                                                                                                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | <div>修改</div> <div>M mft (mft 已改变)</div> <div>R 读取 (上次访问)</div>                                                                                                                                                                                                                                      |
|                      | <div>待修复:</div> <div>注意:关于 m 字段的信息存在冲突,但如下定义的含义已通过 Windows NT 上的实际实验验证为唯一正确的解释。</div>                                                                                                                                                                                                                |
|                      | <div>创建时间</div> <div>时间文件已创建。文件名更新时会更新。</div>                                                                                                                                                                                                                                                        |
|                      | <div>上次数据更改时间</div> <div>数据属性上次修改时间。</div>                                                                                                                                                                                                                                                           |
|                      | <div>last_mft_change_time</div> <div>此mft记录上次修改时间为。</div>                                                                                                                                                                                                                                            |
|                      | <div>上次访问时间</div> <div>文件上次访问的大致时间 (在只读卷上更新)。在 Windows 系统中,如果自上次访问以来经过了一定的时间差,则会访问此信息。</div>                                                                                                                                                                                                         |
|                      | <div>注意:关于 me 字段的信息存在冲突,但如下定义的含义已通过 Windows NT 上的实际实验验证为非常正确,并被认为是唯一正确的解释。</div>                                                                                                                                                                                                                     |
|                      | <div>另请参阅文件记录。</div>                                                                                                                                                                                                                                                                                 |
| 交易                   | <div>系统上的事务是指 (在该系统上)一系列操作的集合,这些操作构成一个整体。该整体不可分割。</div> <div>交易前,系统状态明确;交易过程中,系统状态未定义;交易后,系统状态明确。</div> <div>再次定义。</div> <div>事务不能只完成一半:如果没有操作失败,事务就完成。相反,如果一个或多个操作中发生错误,则事务将无法完成。</div> <div>操作过程中,交易未能实现。</div> <div>根据定义,一组 (即使是原子操作)也并非原子操作。</div> <div>事务是一种为这组数据提供原子性的模型。</div> <div>运营。</div> |
| 未分割的 (参见 Fragmented) | <div>参见“碎片化”。</div>                                                                                                                                                                                                                                                                                  |
| Unicode              | <div>国际字符集采用 16 位编码 (ASCII 采用 7 位编码)。</div> <div>Unicode 编码格式为 8 位二进制数 (拉丁字母 1 和 8 位拉丁字母 1)。Unicode 可以表示世界上几乎所有语言的每个符号。</div>                                                                                                                                                                        |
| 单位                   | <div>本文档中的所有大小均以字节为单位 (除非另有明确说明)。</div> <div>标记)。</div> <div>尺寸的缩写如下:</div>                                                                                                                                                                                                                          |

表122. 计量单位

| 缩写  | 姓名  | 正好10 2          | 大约               |
|-----|-----|-----------------|------------------|
| KB  | 千字节 |                 | 10 <sup>3</sup>  |
| MB  | 兆字节 | 2 <sup>20</sup> | 10 <sup>6</sup>  |
| 英国  | 技嘉  | 2 <sup>30</sup> | 10 <sup>9</sup>  |
| 结核病 | 太字节 | 2 <sup>40</sup> | 10 <sup>12</sup> |

另请参阅二进制、十进制、十六进制

注意 :严格来说,1024 字节的正确缩写是 KiB。  
它代表千字节。

上移

此元数据文件包含 128KB 的大写字母。

Unicode 字母表中的每个字符,在这个数据库中都有一个对应的条目。  
文件。

它用于比较和排序文件名。

(更多的...)

更新序列

NTFS 中的几个结构都包含序列号,需要进行检查。  
一致性错误。

它们是 FILE.INDX.RCRD 和 RSTR 记录。

在将记录写入磁盘之前,每个扇区的最后两个字节会被写入磁盘。  
复制到头部的一个数组中。

然后将更新序列号递增并写入。  
每个区域的末尾。

如果发生任何磁盘损坏,这种技术可以检测到它。

更新序列数组 (usa) 是一个数组,其中包含受更新保护的每个扇区的末尾数据。请注意,第一个条目是编号 (usn),它是一个循环计数器,用于记录数据写入磁盘的次数。每个扇区的最后一个 \_\_u16 行的值为 0 或 -1 (即 0xf),会在写入过程中将其设置为该值。如果不是,则表示在写入数据时发生了传输。

更新序列数组的最大大小为  
最大规模 = usa\_ofs + (usa\_count \* 2) =  
这 510 字节源于这样一个事实:最后一个 (显然)结束符在最后一个 \_\_u16 之前就已结束。  
该公式可用于一致性检查,(usa\_count \* 2) 必须小于或等于 51。

另请参阅 FILE 记录、INDX 记录、RCRD 记录、RSTR 记录。

\$UsnJrnl

词汇表

---

|                                                   |                                                                                      |
|---------------------------------------------------|--------------------------------------------------------------------------------------|
|                                                   | 用于日志记录                                                                               |
|                                                   | (更多的...)                                                                             |
| VCN (参见虚拟集群编号)                                    | 请参阅虚拟集群编号 (VCN)。                                                                     |
| 虚拟集群编号 (VCN) 在表示文件的数据运行时,集群会被赋予虚拟集群编号。            |                                                                                      |
|                                                   | 簇零指的是文件的第一个簇。                                                                        |
|                                                   | 数据运行将 VCN 映射到 LCN,以便可以在卷上找到文件。                                                       |
|                                                   | 另请参阅集群、逻辑集群号 (LCN)、卷。                                                                |
| 体积                                                | (=驱动器=分区) (扩展分区、条带化分区、镜像分区 (不支持) )                                                   |
|                                                   | 逻辑 NTFS 分区。它是一组物理分区 (参见 fdisk 工具,您可以设置镜像和条带化) ,这些物理分区作为一个整体运行 (有点像 Linux 的 md 块设备) 。 |
| 成交量                                               | 此元数据文件包含诸如名称、序列号以及卷是否需要检查错误等信息。                                                      |
|                                                   | (更多的...)                                                                             |
| \$VOLUME_INFORMATION 此属性包含序列号、创建时间以及卷是否需要检查错误等信息。 |                                                                                      |
|                                                   | (更多的...)                                                                             |
| \$VOLUME_NAME                                     | 此属性以 Unicode 编码存储卷的名称。                                                               |
|                                                   | (更多的...)                                                                             |
| \$VOLUME_VERSION                                  | 该属性与 \$SYMBOLIC_LINK 一样,在 NTFS v1.2 中存在,但未使用。                                        |
|                                                   | 它在 NTFS v3.0+ 中已不再存在。                                                                |